ĐẠI HỌC KINH TẾ QUỐC DÂN

VIỆN ĐÀO TẠO TIÊN TIẾN, CHẤT LƯỢNG CAO, POHE

BÀI TẬP NHÓM
KHOA HỌC DỮ LIỆU TRONG KINH TẾ VÀ KINH DOANH

ĐỀ TÀI: PHÂN TÍCH DỮ LIỆU VÀ CÁC YẾU TỐ LIÊN QUAN ĐẾN TÌNH TRẠNG VỠ NỢ TRONG HOẠT ĐỘNG CHO VAY

Sinh viên thực hiện	**:**	**Huỳnh Phương Thảo** Đoàn Ngọc Linh Đỗ Khánh Linh Bùi Hải Yến Phạm Nguyễn Minh Thư
Nhóm thực hiện	**:**	Nhóm 11
Lớp chuyên ngành	**:**	Kinh tế Đầu tư CLC 66B
Giảng viên hướng dẫn	**:**	TS. Trần Đức Minh

HÀ NỘI - 2026

REPOSITORY (GITHUB)

Toàn bộ source code và file báo cáo PDF được lưu trữ công khai tại kho chứa GitHub: https://github.com/khdl-group11-loan-default

BÁO CÁO PHÂN CÔNG NHIỆM VỤ

STT	Họ và tên	MSV	TỶ LỆ ĐÓNG GÓP
1	Huỳnh Phương Thảo	11245287	100%
2	Đoàn Ngọc Linh	11246190	100%
3	Đỗ Khánh Linh	11247644	100%
4	Bùi Hải Yến	11240212	100%
5	Phạm Nguyễn Minh Thư	11245693	100%

MỤC LỤC

LỜI MỞ ĐẦU **6**
CHƯƠNG 1. THU THẬP VÀ ĐÁNH GIÁ DỮ LIỆU BAN ĐẦU **8**
1.1. Giới thiệu nguồn dữ liệu 8
1.2. Khảo sát cấu trúc và thuộc tính dữ liệu 9
1.2.1. Kiểm tra số lượng quan sát và biến 9
1.2.2. Phân loại các nhóm thuộc tính 9
1.2.3. Xây dựng Data Dictionary 11
1.3. Đánh giá chất lượng dữ liệu ban đầu 14
1.3.1. Kiểm tra Missing Values 14
1.3.2. Kiểm tra bản ghi trùng lặp 19
1.3.3. Kiểm tra kiểu dữ liệu và tính hợp lệ 19
1.3.4. Kiểm tra phân phối sơ bộ biến mục tiêu 23
CHƯƠNG 2: TIỀN XỬ LÝ VÀ LÀM SẠCH DỮ LIỆU **25**
2.1. Chuẩn hóa cấu trúc dữ liệu 25
2.1.1. Chuẩn hóa tên biến 26
2.1.2. Chuẩn hóa giá trị biến phân loại 27
2.1.3. Loại bỏ biến không cung cấp thông tin phân tích 30
2.2. Xử lý dữ liệu thiếu (Missing Data Treatment) 33
2.2.1. Đánh giá mức độ thiếu dữ liệu 33
2.2.2. Phân tích cơ chế thiếu dữ liệu 35
2.2.3. Lựa chọn phương pháp xử lý 40
2.2.4. So sánh dữ liệu trước và sau xử lý 43
2.3. Phát hiện và xử lý dữ liệu bất thường 46
2.3.1. Kiểm tra giá trị không hợp lệ theo nghiệp vụ 47
2.3.2. Phát hiện ngoại lai bằng IQR 50
2.3.3. Trực quan hóa ngoại lai 53
2.3.4. Quyết định xử lý ngoại lai 58
2.4. Kiểm tra tính nhất quán giữa các biến 59
2.4.1. Kiểm tra tính hợp lý của LTV 59
2.4.2. Kiểm tra sự đồng nhất về thời điểm ghi nhận 61
2.4.3. Kiểm tra Data Leakage 62
2.5. Feature Engineering có chọn lọc 65
2.6. Mã hóa và chuẩn hóa dữ liệu 69
2.6.1. Xác định nhu cầu mã hóa 69
2.6.2. Xác định nhu cầu Scaling 72
2.7. Hoàn tất và lưu dữ liệu sạch 73

CHƯƠNG 3: PHÂN TÍCH KHÁM PHÁ DỮ LIỆU VÀ TRỰC QUAN HÓA.....78
3.1. Thống kê mô tả tổng quan..... 79
3.1.1. Phân phối biến mục tiêu.....79
3.1.2. Đặc điểm quy mô khoản vay..... 80
3.1.3. Đặc điểm tài chính người vay..... 80
3.1.4. Đặc điểm phân loại..... 81
3.2. Phân tích phân phối từng biến..... 82
3.2.1. Phân phối loan_amount.....82
3.2.2. Phân phối income.....83
3.2.3. Phân phối Credit_Score..... 83
3.2.4. Phân phối theo nhóm tuổi và loại khoản vay.....84
3.2.5. Phân phối các chỉ số LTV và DTI.....85
3.3. Phân tích chất lượng dữ liệu theo trạng thái vỡ nợ.....86
3.3.1. So sánh Missing Values giữa hai nhóm Status.....86
3.3.2. Phân tích sự khác biệt về mẫu quan sát..... 88
3.3.3. Ý nghĩa đối với kết quả phân tích.....92
3.4. Phân tích mối quan hệ giữa các biến và tình trạng vỡ nợ.....93
3.4.1. Tỷ lệ vỡ nợ theo nhóm tuổi.....93
3.4.2. Tỷ lệ vỡ nợ theo thu nhập..... 94
3.4.3. Điểm tín dụng và tình trạng vỡ nợ.....95
3.4.4. Quy mô khoản vay và tình trạng vỡ nợ.....96
3.4.5. Tỷ lệ LTV và tình trạng vỡ nợ..... 97
3.4.6. DTI và tình trạng vỡ nợ..... 99
3.4.7. Loại khoản vay và mục đích vay..... 100
3.5. Kiểm định các giả thuyết nghiên cứu..... 102
3.6. Phân tích mối quan hệ đa biến.....105
3.6.1. Ma trận tương quan.....105
3.6.2. Phát hiện mối liên hệ mạnh giữa các đặc trưng.....107
3.6.3. Liên hệ biến phân loại với status..... 108
3.6.4. Phân tích phân khúc kết hợp.....110
3.6.5. Kiểm tra độ nhạy của kết quả.....112
3.7. Tổng hợp Deep Insights.....114
CHƯƠNG 4. ĐÁNH GIÁ TÁC ĐỘNG VÀ KHUYẾN NGHỊ.....115
4.1. Tổng hợp những phát hiện chính..... 115
4.1.1. Phát hiện về chất lượng dữ liệu.....115
4.1.2. Phát hiện về đặc điểm các khoản vay..... 116
4.1.3. Các yếu tố liên hệ rõ nhất với tình trạng vỡ nợ.....117
4.2. Đánh giá tác động trong quản trị rủi ro tín dụng.....118

4.2.1. Đối với quá trình tiếp nhận và thẩm định hồ sơ.....118
4.2.2. Đối với phân khúc và giám sát danh mục khoản vay..... 119
4.2.3. Đối với quản lý chất lượng dữ liệu..... 119
4.3. Đề xuất các giải pháp ứng dụng.....120
4.3.1. Nhóm giải pháp cho tiếp nhận và thẩm định hồ sơ.....120
4.3.2. Nhóm giải pháp cho phân khúc và giám sát danh mục.....121
4.3.3. Nhóm giải pháp cho quản lý chất lượng dữ liệu.....122
4.4. Đánh giá giới hạn của nghiên cứu..... 122
4.4.1. Giới hạn về nguồn gốc và tính đại diện..... 122
4.4.2. Giới hạn về dữ liệu thiếu.....123
4.4.3. Giới hạn về quan hệ nhân quả.....123
4.4.4. Giới hạn về thời gian và mục tiêu.....123
4.5. Đề xuất hướng nghiên cứu tiếp theo.....123
4.6. Kết luận chung..... 124

LỜI MỞ ĐẦU

1. Đặt vấn đề

Trong lĩnh vực tài chính-ngân hàng, rủi ro tín dụng, đặc biệt là tình trạng vỡ nợ (loan default), luôn là một trong những thách thức lớn nhất ảnh hưởng trực tiếp đến hiệu quả hoạt động và sự ổn định của các tổ chức tín dụng. Việc cấp vốn cho những khách hàng không có khả năng hoàn trả không chỉ gây thất thoát tài sản mà còn làm tăng tỷ lệ nợ xấu, đe dọa đến tính thanh khoản của hệ thống. Do đó, việc xây dựng các hệ thống đánh giá rủi ro và dự báo khả năng vỡ nợ của khách hàng trước khi ra quyết định giải ngân là vô cùng cấp thiết.

Cùng với sự phát triển của khoa học dữ liệu, các mô hình học máy đang ngày càng chứng minh được sức mạnh trong việc khai phá dữ liệu tín dụng, giúp nhận diện sớm các hồ sơ có nguy cơ rủi ro cao. Tuy nhiên, hiệu quả của bất kỳ mô hình dự báo nào cũng phụ thuộc chặt chẽ vào chất lượng của dữ liệu đầu vào. Một tập dữ liệu chứa nhiều nhiễu, giá trị khuyết thiếu (missing values) hay bất thường (outliers) có thể dẫn đến những nhận định sai lệch và mô hình kém hiệu quả.

Xuất phát từ bối cảnh đó, nhóm chúng em thực hiện đề tài: **"**Phân tích dữ liệu và các yếu tố liên quan đến tình trạng vỡ nợ trong hoạt động cho vay".

2. Mục tiêu nghiên cứu

Hai mục tiêu cốt lõi được nhóm chú trọng thực hiện:

Trọng tâm về Dữ liệu: Đánh giá toàn diện chất lượng của tập dữ liệu tín dụng ban đầu, tập trung vào việc phát hiện và xử lý các vấn đề như dữ liệu khuyết thiếu, dữ liệu sai định dạng và các giá trị ngoại lệ. Quá trình làm sạch và tiền xử lý dữ liệu sẽ được thực hiện bài bản nhằm tạo ra một tập dữ liệu chuẩn mực.

Trọng tâm về Phân tích & Trực quan hóa: Khám phá và phân tích mối liên hệ giữa các đặc trưng của khách hàng/khoản vay (như thu nhập, giá trị tài sản thế chấp, tỷ lệ nợ,...) với tình trạng vỡ nợ (Status). Từ các biểu đồ trực quan và kiểm định giả thuyết, nhóm sẽ rút ra những hiểu biết sâu sắc (deep insights) về chân dung khách hàng rủi ro.

3. Phạm vi và Nguồn dữ liệu

Nguồn dữ liệu: Nghiên cứu sử dụng tập dữ liệu "Loan Default Dataset" được thu thập từ nền tảng Kaggle (https://www.kaggle.com/datasets/yasserh/loan-default-dataset).

Phạm vi dữ liệu: Tập dữ liệu bao gồm 148,670 quan sát (hồ sơ vay) với 34 thuộc tính khác nhau, bao gồm các thông tin nhân khẩu học, thông tin tài chính cá nhân, chi tiết khoản vay và biến mục tiêu là trạng thái vỡ nợ (Status).

4. Cấu trúc bài báo cáo

Ngoài phần Mở đầu và Kết luận, bài báo cáo được cấu trúc thành 4 chương chính như sau:

Chương 1. Thu thập và đánh giá dữ liệu ban đầu: Trình bày tổng quan về tập dữ liệu, từ điển dữ liệu (Data Dictionary) và nhận diện các vấn đề về chất lượng dữ liệu (như tỷ lệ missing values).

Chương 2. Tiền xử lý và làm sạch dữ liệu: Chi tiết các kỹ thuật được áp dụng để xử lý dữ liệu khuyết thiếu, ngoại lệ và chuẩn hóa dữ liệu, kèm theo lập luận cho từng phương pháp được chọn.

Chương 3. Phân tích khám phá dữ liệu và Trực quan hóa (EDA): Sử dụng thống kê mô tả, kiểm định giả thuyết và các biểu đồ trực quan để trả lời các câu hỏi kinh doanh và tìm ra nguyên nhân tiềm ẩn dẫn đến vỡ nợ.

Chương 4. Đánh giá tác động và Khuyến nghị: Tổng hợp các phát hiện quan trọng nhất, chỉ ra giới hạn của dữ liệu và đề xuất các giải pháp mang tính ứng dụng thực tế cho tổ chức tín dụng.

CHƯƠNG 1. THU THẬP VÀ ĐÁNH GIÁ DỮ LIỆU BAN ĐẦU

Chương 1 xác định bộ dữ liệu gồm 148.670 hồ sơ vay và 34 biến, không có bản ghi trùng, nhưng 33,96% hồ sơ thiếu ít nhất một ô và việc thiếu gắn chặt với biến mục tiêu Status. Đây là vấn đề trung tâm mà Chương 2 và 3 phải xử lý.

1.1. Giới thiệu nguồn dữ liệu

Thông tin	Mô tả
Tên bộ dữ liệu	Loan Default Dataset
Nguồn	Nền tảng Kaggle – kaggle.com/datasets/yasserh/loan-default-dataset
Định dạng	1 file Loan_Default.csv, khoảng 3,6 MB
Đơn vị quan sát	Mỗi dòng là một hồ sơ vay thế chấp
Nội dung	Đặc điểm người vay, điều khoản khoản vay, tài sản đảm bảo và kết quả vỡ nợ
Biến mục tiêu	Status (1 = vỡ nợ, 0 = không vỡ nợ)
Hạn chế	Kaggle không cung cấp từ điển dữ liệu chính thức, không nêu tổ chức cho vay và quốc gia; ý nghĩa một số mã (p1–p4, type1–type3, cf/ncf, l1/l2) do nhóm suy luận từ tên biến và giá trị

Bối cảnh. Cho vay là nguồn thu chính của ngân hàng, nhưng khoản vay vỡ nợ gây tổn thất trực tiếp. Bộ dữ liệu này cho phép đánh giá chất lượng dữ liệu tín dụng và tìm các đặc điểm có liên hệ với tình trạng vỡ nợ.

Mục tiêu nghiên cứu: Đánh giá chất lượng dữ liệu ban đầu, phát hiện các khiếm khuyết kỹ thuật (khuyết thiếu, ngoại lệ, mất cân bằng nhãn) và phân loại thuộc tính làm cơ sở xây dựng mô hình dự báo xác suất vỡ nợ (Status).

Code:

```
from google.colab import drive
drive.mount('/content/drive')
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/Colab
Notebooks/Loan_Default.csv')
print(df.shape)    # kết quả đúng phải là (148670, 34)
df.head()          # xem 5 dòng đầu
```

Kết quả: `(148670, 34)`, đúng kích thước mong đợi. `df.head()` hiển thị 5 dòng đầu với đủ 34 cột, gồm cả cột số (`loan_amount`, `income`) và cột chữ (`Gender`, `loan_type`).

Nhận xét: Tên cột chưa thống nhất cách viết: có cột viết hoa (`Status`, `Credit_Score`), có cột viết thường (`loan_amount`), có cột dùng gạch nối (`co-applicant_credit_type`).

Kết luận: Dữ liệu sẵn sàng cho bước khảo sát cấu trúc. Tên cột sẽ được chuẩn hóa ở mục 2.1.1.

1.2. Khảo sát cấu trúc và thuộc tính dữ liệu

1.2.1. Kiểm tra số lượng quan sát và biến

Code:

```
print("Kích thước:", df.shape)
print("Số hồ sơ vay (dòng):", df.shape[0])
print("Số thuộc tính (cột):", df.shape[1])
# Thống kê kiểu dữ liệu
print('Dtype:', df.dtypes.value_counts())
df.head()
# Kiểm tra xem có dòng nào bị trùng lặp hoàn toàn không
df.duplicated().sum()
```

Kết quả:

Chỉ tiêu	Giá trị
Số hồ sơ vay (dòng)	148.670
Số thuộc tính (cột)	34
Cột kiểu object (chữ)	21
Cột kiểu float64 (số thực)	8
Cột kiểu int64 (số nguyên)	5

Nhận xét: Mỗi dòng là một hồ sơ vay, mỗi cột là một đặc điểm được ghi nhận. Gần 2/3 số cột (21/34) lưu dạng chữ.

Kết luận: Với 148.670 quan sát, cỡ mẫu đủ lớn để so sánh nhiều phân nhóm. Tuy nhiên, mẫu lớn cũng làm p-value rất nhỏ, nên các chương sau cần đọc kèm kích thước hiệu ứng.

1.2.2. Phân loại các nhóm thuộc tính

Code:

```
bien_so = df.select_dtypes(include='number').columns.tolist()
bien_chu = df.select_dtypes(include='object').columns.tolist()
print("Số biến định lượng:", len(bien_so), bien_so)
print("Số biến định tính:", len(bien_chu), bien_chu)

# Phân loại các biến theo vai trò
id_cols = ['ID']
time_cols = ['year']
target_cols = ['Status']

# Biến phân loại được lưu dưới dạng object
categorical_cols = df.select_dtypes(
    include='object'
).columns.tolist()

# Biến định lượng thực sự (loại trừ các cột ID, year, và Status)
numeric_cols = [
    c for c in df.select_dtypes(include='number').columns
    if c not in id_cols + time_cols + target_cols
    ]

print("Biến định danh:", id_cols)
print("Biến thời gian:", time_cols)
print("Biến mục tiêu:", target_cols)
print("Biến phân loại:", len(categorical_cols), categorical_cols)
print("Biến định lượng đưa vào phân tích:", len(numeric_cols),
numeric_cols)

# Kiểm tra đã phân loại đủ 34 biến
all_cols = id_cols + time_cols + target_cols + categorical_cols +
numeric_cols
assert len(all_cols) == df.shape[1]
assert len(set(all_cols)) == df.shape[1]
```

Kết quả (a) – theo kiểu lưu trữ: 13 biến lưu dạng số và 21 biến lưu dạng chữ.

Kết quả (b) – theo vai trò và nhóm nghiệp vụ:

Nhóm nghiệp vụ	Số biến	Biến định lượng	Biến định tính
Người vay	4	income	Gender, age, Region
Tín dụng	5	Credit_Score	credit_type, co-applicant_credit_type, Credit_Worthiness, open_credit
Khoản vay	8	loan_amount, term	loan_limit, loan_type, loan_purpose, approv_in_adv, business_or_commercial, submission_of_application

Nhóm nghiệp vụ	Số biến	Biến định lượng	Biến định tính
Chi phí vay	3	rate_of_interest, Interest_rate_spread, Upfront_charges	–
Cấu trúc trả nợ	3	–	Neg_ammortization, interest_only, lump_sum_payment
Tài sản thế chấp	6	property_value	construction_type, occupancy_type, Secured_by, total_units, Security_Type
Tỷ lệ rủi ro	2	LTV, dtir1	–
Định danh, thời gian, mục tiêu	3	–	ID, year (không phân tích); Status (biến mục tiêu)
Tổng	**34**	**10**	**21 + 3**

Nhận xét:

- Kiểu lưu trữ chưa phản ánh đúng bản chất của 3 biến số: ID là mã định danh, year là biến thời gian, Status là nhãn nhị phân. Sau khi tách ra, còn 10 biến định lượng thực sự.
- Hai biến chữ có thứ bậc: age (nhóm tuổi <25 đến >74) và total_units (1U–4U). Cần giữ thứ tự khi trình bày và vẽ biểu đồ.
- Nhóm Khoản vay và Tài sản thế chấp chiếm 14/34 biến, phù hợp đặc thù vay thế chấp.

Kết luận: Phân tích sẽ dùng 10 biến định lượng, 21 biến định tính và biến mục tiêu Status. ID và year được xem xét loại bỏ ở Chương 2.

1.2.3. Xây dựng Data Dictionary

```
# Tạo bảng Data Dictionary sơ bộ cho 34 biến
data_dictionary = pd.DataFrame({
    "Tên biến": df.columns,
    "Kiểu dữ liệu": df.dtypes.astype(str).values,
    "Số giá trị thiếu": df.isna().sum().values,
    "Tỷ lệ thiếu (%)": (
        df.isna().mean().values * 100
    ).round(2),
    "Số giá trị phân biệt": df.nunique().values,
    "Giá trị ví dụ": [
```

```
        df[c].dropna().unique()[:5].tolist()
        for c in df.columns
    ]
})

display(data_dictionary)
```

Kết quả:

Tên biến	Kiểu dữ liệu	Số giá trị thiếu	Tỷ lệ thiếu (%)	Số giá trị phân biệt	Giá trị ví dụ
ID	int64	0	0.0	148670	[24890, 24891, 24892, 24893, 24894]
year	int64	0	0.0	1	[2019]
loan_limit	object	3344	2.25	2	['cf', 'ncf']
Gender	object	0	0.0	4	['Sex Not Available', 'Male', 'Joint', 'Female']
approv_in_adv	object	908	0.61	2	['nopre', 'pre']
loan_type	object	0	0.0	3	['type1', 'type2', 'type3']
loan_purpose	object	134	0.09	4	['p1', 'p4', 'p3', 'p2']
Credit_Worthiness	object	0	0.0	2	['l1', 'l2']
open_credit	object	0	0.0	2	['nopc', 'opc']
business_or_commercial	object	0	0.0	2	['nob/c', 'b/c']
loan_amount	int64	0	0.0	211	[116500, 206500, 406500, 456500, 696500]
rate_of_interest	float64	36439	24.51	131	[4.56, 4.25, 4.0, 3.99, 4.5]
Interest_rate_spread	float64	36639	24.64	22516	[0.2, 0.681, 0.3042, 0.1523, 0.9998]
Upfront_charges	float64	39642	26.66	58271	[595.0, 0.0, 370.0, 5120.0, 5609.88]

term	float64	41	0.03	26	[360.0, 300.0, 180.0, 312.0, 144.0]
Neg_ammortization	object	121	0.08	2	['not_neg', 'neg_amm']
interest_only	object	0	0.0	2	['not_int', 'int_only']
lump_sum_payment	object	0	0.0	2	['not_lpsm', 'lpsm']
property_value	float64	15098	10.16	385	[118000.0, 508000.0, 658000.0, 758000.0, 1008000.0]
construction_type	object	0	0.0	2	['sb', 'mh']
occupancy_type	object	0	0.0	3	['pr', 'sr', 'ir']
Secured_by	object	0	0.0	2	['home', 'land']
total_units	object	0	0.0	4	['1U', '2U', '3U', '4U']
income	float64	9150	6.15	1001	[1740.0, 4980.0, 9480.0, 11880.0, 10440.0]
credit_type	object	0	0.0	4	['EXP', 'EQUI', 'CRIF', 'CIB']
Credit_Score	int64	0	0.0	401	[758, 552, 834, 587, 602]
co-applicant_credit_type	object	0	0.0	2	['CIB', 'EXP']
age	object	200	0.13	7	['25-34', '55-64', '35-44', '45-54', '65-74']
submission_of_application	object	200	0.13	2	['to_inst', 'not_inst']
LTV	float64	15098	10.16	8484	[98.72, 80.01, 69.37, 91.88, 70.08]
Region	object	0	0.0	4	['south', 'North', 'central', 'North-East']
Security_Type	object	0	0.0	2	['direct', 'Indriect']
Status	int64	0	0.0	2	[1, 0]
dtir1	float64	24121	16.22	57	[45.0, 46.0, 42.0, 39.0, 40.0]

Nhận xét:

Kích thước tập dữ liệu: Biến ID có chính xác 148,670 giá trị phân biệt và không có dữ liệu thiếu, cho thấy tập dữ liệu bao gồm 148,670 mẫu (dòng) ghi nhận các khoản vay.

Vấn đề dữ liệu thiếu: Một số biến tài chính quan trọng có tỷ lệ thiếu hụt rất cao và cần có chiến lược xử lý đặc biệt trước khi đưa vào mô hình học máy: Upfront_charges (26.66%), Interest_rate_spread (24.64%), rate_of_interest (24.51%) và dtir1 (16.22%).

Biến không có giá trị phân tích: Biến year chỉ có duy nhất 1 giá trị (2019). Vì nó là hằng số, biến này hoàn toàn không có ý nghĩa phân loại hay dự báo và nên được loại bỏ khỏi tập dữ liệu.

Biến mục tiêu: Biến Status là dạng nhị phân (0 hoặc 1), phù hợp với định dạng chuẩn của các bài toán phân lớp (Classification) dự đoán khả năng vỡ nợ.

Biến phân loại ẩn dưới dạng chuỗi: Cột Gender chứa các giá trị như "Sex Not Available" thay vì để trống (NaN). Trong quá trình làm sạch dữ liệu, cần cân nhắc chuyển các giá trị này thành dữ liệu khuyết hoặc nhóm chúng vào một nhóm riêng để thuật toán xử lý chính xác hơn.

1.3. Đánh giá chất lượng dữ liệu ban đầu

1.3.1. Kiểm tra Missing Values

```
# Thống kê dữ liệu thiếu theo cột và tỷ lệ trên tổng số bản ghi
miss = df.isna().sum().sort_values(ascending=False)
summary = pd.DataFrame({'Số ô thiếu': miss, 'Tỷ lệ %': (miss / len(df) *
100).round(3)})
print(summary.loc[summary['Số ô thiếu'] > 0]) # Chỉ hiện cột có thiếu
print('Số dòng có ít nhất một ô thiếu:', df.isna().any(axis=1).sum())
```

Kết quả: cột Mức độ do nhóm tự xếp: Thấp dưới 5%, Trung bình 5–20%, Cao trên 20%

Cột	Số lượng thiếu	Tỷ lệ (%)	Mức độ
Upfront_charges	39.642	26,664	Cao
Interest_rate_spread	36.639	24,645	Cao
rate_of_interest	36.439	24,510	Cao
dtir1	24.121	16,225	Trung bình
property_value	15.098	10,155	Trung bình
LTV	15.098	10,155	Trung bình

income	9.150	6,155	Trung bình
loan_limit	3.344	2,249	Thấp
approv_in_adv	908	0,611	Thấp
age	200	0,135	Thấp
submission_of_application	200	0,135	Thấp
loan_purpose	134	0,09	Thấp
Neg_ammortization	121	0,081	Thấp
term	41	0,28	Thấp

Số dòng có ít nhất một ô thiếu: 50,483

Tỷ lệ dòng bị thiếu dữ liệu: 33.96%

Nhận xét:

Sự phân bố của Missing Values: Có tổng cộng 14 biến chứa giá trị khuyết thiếu. Trong đó, nhóm các biến liên quan trực tiếp đến thông tin lãi suất và chi phí khoản vay có tỷ lệ thiếu hụt cao nhất (trên 24%), bao gồm: Upfront_charges (26.66%), Interest_rate_spread (24.65%), và rate_of_interest (24.51%).

Mối liên hệ giữa các biến thiếu: Đáng chú ý là LTV (Loan-to-Value) và property_value (Giá trị tài sản) có số lượng ô thiếu hoàn toàn khớp nhau (15,098 ô, chiếm 10.15%). Điều này hoàn toàn hợp lý vì LTV được tính dựa trên số tiền vay và giá trị tài sản; nếu không có giá trị tài sản thì LTV cũng không thể tính toán được. Tương tự, age và submission_of_application cũng cùng khuyết 200 giá trị.

Mức độ ảnh hưởng tổng thể: Có tới **33.96%** (50,483) số dòng trong tập dữ liệu chứa ít nhất một giá trị thiếu. Đây là một tỷ lệ rất lớn. Nếu chúng ta chọn phương pháp loại bỏ dòng (Drop Rows) đối với tất cả các Missing Values, tập dữ liệu sẽ bị mất đi hơn 1/3 thông tin, gây lãng phí dữ liệu và có thể làm sai lệch mô hình. Thay vào đó, cần áp dụng các kỹ thuật Imputation (điền khuyết) phù hợp cho từng biến, đặc biệt ưu tiên xử lý cẩn thận nhóm biến tài chính có tỷ lệ thiếu hụt >20%.

Kết luận chung: Dữ liệu thiếu ở mức đáng kể, tập trung ở nhóm chi phí vay và các tỷ lệ tài chính. Mỗi mức thiếu cần một cách xử lý riêng ở Chương 2.

Dữ liệu thiếu có gắn với vỡ nợ không?

```
# Kiểm tra sự liên hệ giữa ô thiếu và biến mục tiêu Status
```

```
cols =
['rate_of_interest','Interest_rate_spread','Upfront_charges','property_val
ue','LTV','dtir1','income']
for c in cols:
print(c, df.groupby('Status')[c].apply(lambda s:
round(s.isna().mean()*100,2)).to_dict())
# Lấy toàn bộ các cột có giá trị thiếu
missing_cols = df.columns[df.isna().any()].tolist()

# Tỷ lệ thiếu theo từng nhóm Status
missing_by_status = pd.DataFrame({
   status: df.loc[df['Status'] == status, missing_cols]
             .isna().mean() * 100
   for status in sorted(df['Status'].dropna().unique())
})

display(missing_by_status.round(2))
```

Kết quả:

	Số ô thiếu	Tỷ lệ phần trăm
Upfront_charges	39642	26.664
Interest_rate_spread	36639	24.645
rate_of_interest	36439	24.510
dtir1	24121	16.225
property_value	15098	10.155
LTV	15098	10.155
income	9150	6.155
loan_limit	3344	2.249
approv_in_adv	908	0.611
age	200	0.135
submission_of_application	200	0.135
loan_purpose	134	0.090
Neg_ammortization	121	0.081
term	41	0.028

Số dòng có ít nhất một ô thiếu: 50483

Nhận xét:
Mức độ khuyết thiếu lớn: Có tới 50.483 dòng chứa ít nhất một giá trị thiếu, chiếm khoảng 1/3 tổng quy mô tập dữ liệu, cho thấy chất lượng dữ liệu thô chưa thực sự hoàn chỉnh.

Trọng tâm khuyết thiếu: Các biến phản ánh chi phí và lãi suất khoản vay (Upfront_charges, Interest_rate_spread, rate_of_interest) bị thiếu nghiêm trọng nhất, đều vượt mức 24%.

Ràng buộc logic nghiệp vụ: Hai biến property_value (Giá trị tài sản) và LTV (Tỷ lệ khoản vay/giá trị) có số ô khuyết hoàn toàn bằng nhau (15.098 ô), minh chứng cho việc khi thiếu giá trị tài sản thì hệ thống không thể tính toán được chỉ số LTV.

Kết luận chung: Bộ dữ liệu có tỷ lệ khuyết thiếu khá lớn và mang tính hệ thống chứ không hoàn toàn ngẫu nhiên. Việc thiếu hụt tập trung chủ yếu vào các thuộc tính tài chính cốt lõi và các biến có tính ràng buộc nghiệp vụ chặt chẽ với nhau, do đó dữ liệu này yêu cầu các bước làm sạch chuyên sâu ở giai đoạn sau để không làm biến dạng thông tin khi xây dựng mô hình dự báo rủi ro vỡ nợ.

Kiểm tra sự liên hệ giữa ô thiếu và biến mục tiêu Status:

```
cols=['rate_of_interest','Interest_rate_spread','Upfront_charges','propert
y_value','LTV','dtir1','income']
for c in cols:
print(c, df.groupby('Status')[c].apply(lambda s:
round(s.isna().mean()*100,2)).to_dict())
# Lấy toàn bộ các cột có giá trị thiếu
missing_cols = df.columns[df.isna().any()].tolist()

# Tỷ lệ thiếu theo từng nhóm Status
missing_by_status = pd.DataFrame({
   status: df.loc[df['Status'] == status, missing_cols]
             .isna().mean() * 100
   for status in sorted(df['Status'].dropna().unique())
})
display(missing_by_status.round(2))
```

Kết quả:

Tỷ lệ thiếu (%) cho từng biến theo nhóm Status:

rate_of_interest {0: 0.0, 1: 99.45}
Interest_rate_spread {0: 0.0, 1: 100.0}
Upfront_charges {0: 2.82, 1: 99.58}
property_value {0: 0.0, 1: 41.2}
LTV {0: 0.0, 1: 41.2}

dtir1 {0: 6.97, 1: 44.52}
income {0: 7.06, 1: 3.38}

Bảng tỷ lệ thiếu chi tiết theo nhóm Status:

Biến số	% vỡ nợ khi thiếu (Status 0)	% vỡ nợ khi không thiếu (Status 0)
loan_limit	2.20	2.40
approv_in_adv	0.60	0.66
loan_purpose	0.09	0.10
rate_of_interest	0.00	99.45
Interest_rate_spread	0.00	100.00
Upfront_charges	2.82	99.58
term	0.02	0.04
Neg_ammortization	0.08	0.09
property_value	0.00	41.20
income	7.06	3.38
age	0.00	0.55
submission_of_application	0.00	0.55
LTV	0.00	41.20
dtir1	6.97	44.52

Nhận xét:

Kết quả cho thấy tình trạng thiếu dữ liệu có mối liên hệ thống kê rất mạnh với biến mục tiêu `Status`, đặc biệt ở các biến chi phí vay. Điều này cho thấy giả định thiếu hoàn toàn ngẫu nhiên (MCAR) không phù hợp, nhưng nghiên cứu chưa đủ cơ sở xác định chính xác cơ chế thiếu dữ liệu.

Dấu hiệu "Data Leakage" (Rò rỉ dữ liệu) ở các biến lãi suất và chi phí:

Interest_rate_spread: Khuyết **100%** ở nhóm Status = 1 và **0%** ở nhóm Status = 0.

rate_of_interest: Khuyết **99.45%** ở nhóm Status = 1 và **0%** ở nhóm Status = 0.

Upfront_charges: Khuyết **99.58%** ở nhóm Status = 1.

Tình trạng thiếu của một số biến lãi suất và chi phí có mối liên hệ gần như tuyệt đối với `Status`. Nếu các thông tin này chỉ được xác định hoặc cập nhật sau thời điểm cần dự báo, việc đưa chúng vào mô hình có thể gây rò rỉ thông tin mục tiêu, khiến kết quả đánh giá trở nên quá lạc quan. Tuy nhiên, do chưa xác minh được định nghĩa của `Status` và thời điểm ghi nhận từng biến, nghiên cứu chưa thể kết luận chắc chắn đã xảy ra Data Leakage.

Sự khuyết thiếu có hệ thống ở tài sản đảm bảo:

property_value và LTV (Loan-to-Value) đều không bị thiếu ở nhóm Status = 0 (0%), nhưng lại khuyết tới **41.2%** ở nhóm Status = 1. Tương tự, biến tỷ lệ nợ trên thu nhập dtir1 khuyết **44.52%** ở nhóm Status = 1 nhưng chỉ khuyết gần 7% ở nhóm Status = 0. Điều này cho thấy hồ sơ của những người thuộc nhóm Status = 1 thường chưa hoàn thiện các bước định giá tài sản hoặc đánh giá tín dụng chi tiết.

Kết luận: Dữ liệu thiếu ở mức đáng kể và không ngẫu nhiên, gắn với kết quả vỡ nợ. Nhóm không xóa các dòng này mà xử lý theo mức độ thiếu ở Chương 2.

1.3.2. Kiểm tra bản ghi trùng lặp

```
print('Trùng hàng hoàn toàn:', df.duplicated().sum()) # Trùng đủ 34 cột
print('Trùng mã ID:', df['ID'].duplicated().sum()) # Trùng khóa hồ sơ
print('Trùng sau bỏ ID:', df.drop(columns=['ID']).duplicated().sum())
```

Kết quả:
Trùng hàng hoàn toàn: 0
Trùng mã ID: 0
Trùng sau bỏ ID: 0

1.3.3. Kiểm tra kiểu dữ liệu và tính hợp lệ

```
# Kiểm tra giá trị bất thường trên các biến số
checks = {
    "loan_amount <= 0":
        (df["loan_amount"] <= 0).sum(),
    "property_value <= 0":
        (df["property_value"] <= 0).sum(),
    "income < 0":
        (df["income"] < 0).sum(),
    "income = 0":
        (df["income"] == 0).sum(),
    "term <= 0":
        (df["term"] <= 0).sum(),
    "rate_of_interest < 0":
        (df["rate_of_interest"] < 0).sum(),
    "rate_of_interest = 0":
        (df["rate_of_interest"] == 0).sum(),
    "Credit_Score ngoài khoảng 300-900":
```

```
        ((df["Credit_Score"] < 300) |
         (df["Credit_Score"] > 900)).sum(),
    "dtir1 ngoài khoảng 0–100":
        ((df["dtir1"] < 0) |
         (df["dtir1"] > 100)).sum(),
    "LTV > 100":
        (df["LTV"] > 100).sum(),
    "LTV > 150":
        (df["LTV"] > 150).sum(),
    "Status không thuộc {0, 1}":
        (~df["Status"].isin([0, 1])).sum(),
}
display(
    pd.Series(checks, name="Số bản ghi")
      .to_frame()
)
```

Kết quả:

	Số bản ghi
loan_amount <= 0	0
property_value <= 0	0
income < 0	0
income = 0	1260
term <= 0	0
rate_of_interest < 0	0
rate_of_interest = 0	1
Credit_Score ngoài khoảng 300–900	0
dtir1 ngoài khoảng 0–100	0
LTV > 100	1799
LTV > 150	33
Status không thuộc {0, 1}	0

```
# Kiểm tra các giá trị phân loại gốc
for col in categorical_cols:
    print(f"\nBiến: {col}")
    print(df[col].value_counts(dropna=False))
```

Kết quả:

Biến: loan_limit
loan_limit
cf 135348
ncf 9978
NaN 3344
Name: count, dtype: int64

Biến: Gender
Gender
Male 42346
Joint 41399
Sex Not Available 37659
Female 27266
Name: count, dtype: int64

Biến: approv_in_adv
approv_in_adv
nopre 124621
pre 23141
NaN 908
Name: count, dtype: int64

Biến: loan_type
loan_type
type1 113173
type2 20762
type3 14735
Name: count, dtype: int64

Biến: loan_purpose
loan_purpose
p3 55934
p4 54799
p1 34529
p2 3274
NaN 134
Name: count, dtype: int64

Biến: Credit_Worthiness
Credit_Worthiness
l1 142344
l2 6326
Name: count, dtype: int64

Biến: open_credit
open_credit
nopc 148114
opc 556
Name: count, dtype: int64

Biến: business_or_commercial
business_or_commercial
nob/c 127908
b/c 20762
Name: count, dtype: int64

Biến: Neg_ammortization
Neg_ammortization
not_neg 133420
neg_amm 15129
NaN 121
Name: count, dtype: int64

Biến: interest_only
interest_only
not_int 141560
int_only 7110
Name: count, dtype: int64

Biến: lump_sum_payment
lump_sum_payment
not_lpsm 145286
lpsm 3384
Name: count, dtype: int64

Biến: construction_type
construction_type
sb 148637
mh 33
Name: count, dtype: int64

Biến: occupancy_type
occupancy_type
pr 138201
ir 7340
sr 3129
Name: count, dtype: int64

Biến: Secured_by
Secured_by

home 148637
land 33
Name: count, dtype: int64

Biến: total_units
total_units
1U 146480
2U 1477
3U 393
4U 320
Name: count, dtype: int64

Biến: credit_type
credit_type
CIB 48152
CRIF 43901
EXP 41319
EQUI 15298
Name: count, dtype: int64

Biến: co-applicant_credit_type
co-applicant_credit_type
CIB 74392
EXP 74278
Name: count, dtype: int64

Biến: age
age
45-54 34720
35-44 32818
55-64 32534
65-74 20744
25-34 19142
>74 7175
<25 1337
NaN 200
Name: count, dtype: int64

Biến: submission_of_application
submission_of_application
to_inst 95814
not_inst 52656
NaN 200
Name: count, dtype: int64

Biến: Region
Region
North 74722
south 64016
central 8697
North-East 1235
Name: count, dtype: int64

Biến: Security_Type
Security_Type
direct 148637
Indriect 33
Name: count, dtype: int64

```
# Kiểm tra công thức LTV trên các dòng có đủ thông tin
valid = df[['loan_amount','property_value','LTV']].dropna()
difference = (valid['LTV'] - 100 * valid['loan_amount'] /
valid['property_value']).abs()
print('Số dòng kiểm tra:', len(valid), 'Sai lệch lớn nhất:',
difference.max())
```

Kết quả:
Số dòng kiểm tra: 133572 Sai lệch lớn nhất: 4.948452669850667e-08 \

Nhận xét:

- **Biến số:** Không có giá trị âm hay ngoài phạm vi hợp lệ; Status chỉ gồm 0 và 1. Tuy nhiên có 1.260 hồ sơ thu nhập bằng 0, 1 hồ sơ lãi suất bằng 0 và 1.799 hồ sơ LTV vượt 100% (33 hồ sơ vượt 150%). Đây là các trường hợp bất thường về nghiệp vụ.

- **Biến phân loại:**
  + Gender có 37.659 hồ sơ (25,33%) mang nhãn "Sex Not Available", thực chất là dữ liệu thiếu ngụy trang.
  + Có lỗi nhập liệu: Security_Type sai chính tả "Indriect", Region viết hoa không thống nhất.
  + construction_type, Secured_by và Security_Type gần như hằng số. Giá trị thiểu số của cả ba biến đều có đúng 33 hồ sơ, nhiều khả năng là cùng một nhóm.
- **Công thức LTV:** Sai lệch lớn nhất trên 133.572 hồ sơ chỉ khoảng $4{,}9 \times 10^{-8}$, tức LTV hoàn toàn nhất quán với loan_amount và property_value.

Kết luận: Dữ liệu nhìn chung đáp ứng yêu cầu về tính hợp lệ: không có giá trị âm, các biến nằm trong phạm vi cho phép và công thức tính toán giữa các biến nhất quán. Tuy nhiên, dữ liệu vẫn tồn tại các vấn đề cần xử lý ở Chương 2, bao gồm:

- dữ liệu thiếu ngụy trang ở Gender
- lỗi chính tả và cách viết không thống nhất ở các biến phân loại
- các biến gần như hằng số có thể không mang giá trị phân tích
- các giá trị bất thường ở income, LTV và rate_of_interest cần được rà soát bằng phương pháp phát hiện ngoại lai.

1.3.4. Kiểm tra phân phối sơ bộ biến mục tiêu

```
# Thống kê số lượng và tỷ lệ phân phối biến Status
count = df['Status'].value_counts().sort_index()
rate = (count / len(df) * 100).round(2)
print(pd.DataFrame({'Số hồ sơ': count, 'Tỷ lệ %': rate}))
```

Kết quả:

Status	Số hồ sơ	Tỷ lệ %
0	112031	75.36
1	36639	24.64

```
# Bảng tổng hợp chất lượng dữ liệu ban đầu
quality_summary = pd.DataFrame({
    "Chỉ tiêu": [
        "Tổng số dòng",
        "Tổng số cột",
        "Số cột có Missing Values",
        "Số dòng có Missing Values",
        "Số dòng trùng hoàn toàn",
        "Số ID trùng",
        "Số cột chỉ có một giá trị",
        "Số giá trị Status bị thiếu"
    ],
    "Kết quả": [
        df.shape[0],
```

```
        df.shape[1],
        df.isna().any().sum(),
        df.isna().any(axis=1).sum(),
        df.duplicated().sum(),
        df["ID"].duplicated().sum(),
        (df.nunique(dropna=False) == 1).sum(),
        df["Status"].isna().sum()
    ]
})
display(quality_summary)

# Khởi tạo lại data_dictionary đề phòng trường hợp ô 1.2.3 chưa được chạy
data_dictionary = pd.DataFrame({
    "Tên biến": df.columns,
    "Kiểu dữ liệu": df.dtypes.astype(str).values,
    "Số giá trị thiếu": df.isna().sum().values,
    "Tỷ lệ thiếu (%)": (
        df.isna().mean().values * 100
    ).round(2),
    "Số giá trị phân biệt": df.nunique().values,
    "Giá trị ví dụ": [
        df[c].dropna().unique()[:5].tolist()
        for c in df.columns
    ]
})
```

Kết quả:

	Chỉ tiêu	Kết quả
0	Tổng số dòng	148670
1	Tổng số cột	34
2	Số cột có Missing Values	14
3	Số dòng có Missing Values	50483
4	Số dòng trùng hoàn toàn	0
5	Số ID trùng	0
6	Số cột chỉ có một giá trị	1
7	Số giá trị Status bị thiếu	0

Nhận xét: Bộ dữ liệu gồm 148.670 hồ sơ và 34 biến. Trong đó có 36.639 hồ sơ vỡ nợ, chiếm 24,64%, so với 75,36% hồ sơ không vỡ nợ. Tỷ lệ khoảng 3 : 1 này cho thấy dữ liệu mất cân bằng ở mức vừa phải. Biến mục tiêu Status đầy đủ, không có dòng hay ID trùng

lặp. Tuy nhiên, 14 cột chứa giá trị thiếu, ảnh hưởng đến 50.483 hồ sơ (khoảng 33,96%). Ngoài ra, year là cột duy nhất chỉ có một giá trị (2019). Đáng chú ý, số hồ sơ vỡ nợ trùng khớp đúng với số ô thiếu của Interest_rate_spread, củng cố nhận định về mối liên hệ giữa dữ liệu thiếu và tình trạng vỡ nợ ở mục 1.3.1.

Kết luận: Dữ liệu đảm bảo về quy mô, tính duy nhất và tính đầy đủ của biến mục tiêu. Do dữ liệu mất cân bằng, Chương 3 cần so sánh theo tỷ lệ vỡ nợ (%), lấy mức 24,64% làm mốc nền. Với hơn 1/3 số hồ sơ bị thiếu thông tin, không nên loại bỏ các dòng thiếu. Ưu tiên ở Chương 2 là xử lý dữ liệu thiếu một cách thận trọng và loại bỏ các biến không mang thông tin phân tích.

CHƯƠNG 2: TIỀN XỬ LÝ VÀ LÀM SẠCH DỮ LIỆU

2.1. Chuẩn hóa cấu trúc dữ liệu

Dữ liệu đầu vào là bản gốc ở Chương 1: 148.670 hồ sơ, 34 biến, biến mục tiêu Status (112.031 hồ sơ không vỡ nợ, 36.639 hồ sơ vỡ nợ, tỷ lệ 24,64%). Bản gốc được giữ nguyên trong df_raw, mọi bước xử lý chạy trên bản sao.

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns
from google.colab import drive
# Cấu hình hiển thị bảng và biểu đồ.
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_rows", 70)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")
sns.set_theme(style="whitegrid")
DATA_PATH = "/content/drive/MyDrive/Colab Notebooks/Loan_Default.csv"
df_raw = df.copy()
df = df_raw.copy()
print("Kích thước Chương 1:", df_raw.shape)
print("Số cột có missing:", int(df_raw.isna().any().sum()))
print("Dòng trùng hoàn toàn:", int(df_raw.duplicated().sum()))
print("Phân phối Status:",
df_raw["Status"].value_counts().sort_index().to_dict())
assert df_raw.shape == (148670, 34), "Kích thước file không khớp bộ dữ
liệu của Chương 1."
assert df_raw["Status"].isin([0, 1]).all(), "Biến mục tiêu khác 0/1; cần
kiểm tra lại."
```

Kết quả: Kích thước Chương 1: (148670, 34)

Số cột có missing: 14

Dòng trùng hoàn toàn: 0

Phân phối Status: {0: 112031, 1: 36639}

2.1.1. Chuẩn hóa tên biến

```
# Tạo hàm chuẩn hóa tên: bỏ khoảng trắng, hạ chữ, đổi dấu cách/gạch nối
thành gạch dưới.
def standardize_name(name):
    name = name.strip().lower()
    name = re.sub(r"[\s\-]+", "_", name)
    name = re.sub(r"_+", "_", name)
    return name.strip("_")
# Ghi bảng đối chiếu trước/sau để các chương sau biết tên cột mới.
old_cols = df.columns.tolist()
new_cols = [standardize_name(name) for name in old_cols]
if len(set(new_cols)) != len(new_cols):
    raise ValueError("Chuẩn hóa tên gây trùng cột; phải xử lý tên trùng
trước.")
rename_table = pd.DataFrame({"Tên cột trước": old_cols, "Tên cột sau":
new_cols})
df.columns = new_cols
print("Các tên cột thay đổi:")
display(rename_table.query('`Tên cột trước` != `Tên cột sau`'))
print("Số cột trước/sau:", len(old_cols), "/", len(df.columns))
print("Tên biến mục tiêu:", "status")
```

Kết quả: Các tên cột thay đổi:

	Tên cột trước	Tên cột sau
0	ID	id
3	Gender	gender
7	Credit_Worthiness	credit_worthiness
12	Interest_rate_spread	interest_rate_spread
13	Upfront_charges	upfront_charges
15	Neg_ammortization	neg_ammortization
21	Secured_by	secured_by
25	Credit_Score	credit_score
26	co-applicant_credit_type	co_applicant_credit_type

29	LTV	ltv
30	Region	region
31	Security_Type	security_type
32	Status	status

Số cột trước/sau: 34 / 34
Tên biến mục tiêu: status
Kết luận: 34 tên cột đã thống nhất về chữ thường, dùng gạch dưới, không còn ký tự đặc biệt. Số dòng và số cột không đổi. Biến mục tiêu từ đây là `status`

2.1.2. Chuẩn hóa giá trị biến phân loại

```
#Xác định các biến phân loại dạng ký tự.
cat_cols = df.select_dtypes(include=["object", "string"]).columns.tolist()
cat_before = df[cat_cols].copy()
#Chuẩn hóa từng biến: bỏ khoảng trắng hai đầu, gộp nhiều khoảng trắng
thành một, viết thường.
for col in cat_cols:
   df[col] = (df[col].astype("string")
                        .str.strip()
                        .str.replace(r"\s+", " ", regex=True)
                        .str.lower()
                        .replace("", pd.NA))
#Sửa lỗi chính tả xác định được từ danh sách nhãn gốc.
df["security_type"] = df["security_type"].replace({"indriect":
"indirect"})
#Thống kê tên biến, số nhóm trước/sau và số ô có thay đổi cách viết.
cat_changes = []
for col in cat_cols:
   before = cat_before[col].astype("string").fillna("<NA>")
   after = df[col].fillna("<NA>")
   cat_changes.append({
       "Biến": col,
       "Số nhóm trước": int(cat_before[col].nunique(dropna=True)),
       "Số nhóm sau": int(df[col].nunique(dropna=True)),
       "Số ô chỉnh cách viết": int((before != after).sum()),
   })
cat_changes = pd.DataFrame(cat_changes)
print("Bảng tổng hợp chuẩn hóa biến phân loại:")
display(cat_changes)
#In một số biến để kiểm tra các nhãn nhạy cảm về ý nghĩa.
for col in ["region", "security_type", "gender", "loan_type", "age"]:
```

```
    print(f"\n{col}:")
    display(df[col].value_counts(dropna=False).rename("Số hồ
sơ").to_frame())
#Đảm bảo không sinh thêm NaN ngoài các ô đã thiếu trước chuẩn hóa.
assert df[cat_cols].isna().sum().sum() == cat_before.isna().sum().sum()
```

Kết quả: Bảng tổng hợp chuẩn hóa biến phân loại:

	Biến	Số nhóm trước	Số nhóm sau	Số ô chỉnh cách viết
0	loan_limit	2	2	0
1	gender	4	4	148670
2	approv_in_adv	2	2	0
3	loan_type	3	3	0
4	loan_purpose	4	4	0
5	credit_worthiness	2	2	0
6	open_credit	2	2	0
7	business_or_commercial	2	2	0
8	neg_ammortization	2	2	0
9	interest_only	2	2	0
10	lump_sum_payment	2	2	0
11	construction_type	2	2	0
12	occupancy_type	3	3	0
13	secured_by	2	2	0
14	total_units	4	4	148670
15	credit_type	4	4	148670
16	co_applicant_credit_type	2	2	148670
17	age	7	7	0

18	submission_of_application	2	2	0
19	region	4	4	75957
20	security_type	2	2	33

region:

region	Số hồ sơ
north	74722
south	64016
central	8697
north-east	1235

security_type:

security_type	Số hồ sơ
direct	148637
indirect	33

gender:

gender	Số hồ sơ
male	42346
joint	41399
sex not available	37659
female	27266

loan_type:

loan_type	Số hồ sơ
type1	113173
type2	20762
type3	14735

age:

age	Số hồ sơ
45-54	34720
35-44	32818
55-64	32534
65-74	20744
25-34	19142
>74	7175
<25	1337
<NA>	200

Giải thích. Với 21 biến dạng chữ, code bỏ khoảng trắng thừa, gộp nhiều khoảng trắng thành một và chuyển về chữ thường. Lỗi chính tả `Indriect` được sửa thành `indirect`. Bảng tổng hợp so sánh số nhóm trước/sau và số ô bị chỉnh sửa. Một lệnh `assert` bảo đảm bước này không tạo thêm ô thiếu.

Nhận xét. Số nhóm của cả 21 biến giữ nguyên, nên không nhãn nào bị gộp. Thay đổi chỉ nằm ở cách viết:

- `gender`, `total_units`, `credit_type` và `co_applicant_credit_type` có 148.670 ô đổi, do chuyển sang chữ thường.
- `region` có 75.957 ô đổi, gồm `North` và `North-East`. Biến này trước đó trộn chữ hoa và chữ thường.
- `security_type` có 33 ô đổi, do sửa lỗi `Indriect`.

Biến `gender` vẫn có bốn mức. `joint` và `sex not available` được giữ riêng vì khác nghĩa nghiệp vụ với `male` và `female`.

Kết luận. Các biến phân loại đã nhất quán về cách viết mà không làm mất hay gộp nhầm thông tin. Không có ô thiếu mới.

2.1.3. Loại bỏ biến không cung cấp thông tin phân tích

```
#Kiểm tra cột có một giá trị chiếm trên 99.9% dòng (bao gồm cả giá trị
thiếu nếu có).
near_constant = []
for col in df.columns:
   freq = df[col].value_counts(normalize=True, dropna=False)
```

```
    dominant = float(freq.iloc[0])
    if dominant > 0.999:
        near_constant.append({"Biến": col, "Giá trị phổ biến":
str(freq.index[0]),
                              "Tỷ lệ phổ biến (%)": dominant * 100,
                              "Số nhóm": df[col].nunique(dropna=False)})
display(pd.DataFrame(near_constant))
#'business_or_commercial' được mã hóa hoàn toàn bởi loan_type: b/c tương
ứng type2.
redundancy_check = (df["business_or_commercial"].eq("b/c") ==
df["loan_type"].eq("type2"))
print("Tỷ lệ quy tắc business_or_commercial ↔ loan_type khớp:",
f"{redundancy_check.mean():.2%}")
display(pd.crosstab(df["business_or_commercial"], df["loan_type"]))
#3 cột dạng gần như hằng số cùng đánh dấu đúng 33 dòng hiếm.
rare_a = df["construction_type"].eq("mh")
rare_b = df["secured_by"].eq("land")
rare_c = df["security_type"].eq("indirect")
print("3 biến cùng nhận diện đúng một tập 33 hồ sơ:",
bool(rare_a.equals(rare_b) and rare_b.equals(rare_c)))
print("Số hồ sơ hiếm:", int(rare_a.sum()))
print("Status của nhóm hiếm:", df.loc[rare_a,
"status"].value_counts().to_dict())
```

Kết quả:

	Biến	Giá trị phổ biến	Tỷ lệ phổ biến (%)	Số nhóm
0	**year**	2019	100.000	1
1	**construction_type**	sb	99.978	2
2	**secured_by**	home	99.978	2
3	**security_type**	direct	99.978	2

Tỷ lệ quy tắc business_or_commercial ↔ loan_type khớp: **100.00%**

loan_type	type1	type2	type3
business_or_commercial			
b/c	**0**	**20762**	**0**
nob/c	**113173**	**0**	**14735**

3 biến cùng nhận diện đúng một tập 33 hồ sơ: True

Số hồ sơ hiếm: 33

Status của nhóm hiếm: {1: 33}

```
# Danh sách 6 cột loại khỏi tập phân tích tổng thể sau khi đã kiểm tra.
columns_to_drop = {
   "id": "Mã định danh, không dùng làm biến giải thích",
   "year": "Chỉ có năm 2019, không có biến thiên thời gian",
   "business_or_commercial": "Có thể suy ra hoàn toàn từ loan_type",
   "construction_type": "99.978% cùng một nhãn; 33 dòng hiếm trùng với
secured_by/security_type",
   "secured_by": "Gần hằng số và cùng phân tách 33 dòng với
construction_type",
   "security_type": "Gần hằng số và cùng phân tách 33 dòng với
construction_type",
}
# Lưu danh sách lý do để minh bạch, sau đó bỏ cột, không bỏ dòng.
drop_summary = pd.DataFrame([{"Biến loại": k, "Lý do": v} for k, v in
columns_to_drop.items()])
display(drop_summary)
df = df.drop(columns=list(columns_to_drop.keys()))
print("Kích thước sau 2.1:", df.shape)
assert df.shape == (148670, 28)
assert "status" in df.columns
```

Giải thích. Code tìm biến gần như hằng số (một giá trị chiếm hơn 99,9%) và biến dư thừa. Kết quả:

- `year` chỉ có một giá trị, 2019.
- `construction_type`, `secured_by` và `security_type` có cùng một giá trị phổ biến, chiếm 99,978%. Ba biến này cùng đánh dấu đúng một tập 33 hồ sơ.
- `business_or_commercial` trùng khớp 100% với `loan_type`: `b/c` ứng với `type2`, `nob/c` ứng với `type1` và `type3`.
- `id` chỉ là mã định danh, không có giá trị giải thích.

Nhận xét. Sáu biến bị loại (`id`, `year`, `business_or_commercial`, `construction_type`, `secured_by`, `security_type`). Dữ liệu còn 148.670 dòng và 28 cột, và chỉ bỏ cột, không bỏ dòng. Cả 33 hồ sơ thuộc nhóm hiếm đều có `status = 1`. Tuy vậy, nhóm này quá nhỏ (0,02% mẫu) và ba biến mô tả trùng thông tin, nên chưa đủ cơ sở để coi là một phân khúc rủi ro ổn định. Ba biến construction_type, secured_by và security_type cùng xác định một nhóm 33 hồ sơ, tất cả đều có status = 1. Nhóm hiếm chỉ có 33 hồ sơ, ba biến cung cấp

thông tin trùng lặp và chưa đủ cơ sở để diễn giải thành một phân khúc rủi ro ổn định. Việc loại chúng là hợp lý vì nhóm quá hiếm và ba biến trùng thông tin.

Kết luận. Việc loại 6 biến làm gọn dữ liệu mà không mất thông tin phân tích. Nhóm 33 hồ sơ hiếm được ghi nhận là hạn chế của dữ liệu.

2.2. Xử lý dữ liệu thiếu (Missing Data Treatment)

2.2.1. Đánh giá mức độ thiếu dữ liệu

```
# Kế thừa trực tiếp dữ liệu sau mục 2.1
assert df.shape == (148670, 28)
# Tổng số ô thiếu và số dòng có ít nhất 1 ô thiếu.
missing_counts_before = df.isna().sum()
missing_table = pd.DataFrame({
   "Số ô thiếu": missing_counts_before,
   "Tỷ lệ thiếu (%)": df.isna().mean().mul(100),
})
missing_table = missing_table[missing_table["Số ô thiếu"] >
0].sort_values(
   "Số ô thiếu", ascending=False)
print("Số dòng:", f"{len(df):,}")
print("Số cột có missing:", len(missing_table))
print("Tổng số ô thiếu:", f"{int(missing_counts_before.sum()):,}")
print("Số dòng có ≥ 1 ô thiếu:", f"{int(df.isna().any(axis=1).sum()):,}")
print("Tỷ lệ dòng có ≥ 1 ô thiếu:",
f"{100*df.isna().any(axis=1).mean():.2f}%")
display(missing_table.round(3))
# Phân phối số ô thiếu trên một hồ sơ — kiểm tra mức tập trung của
missing.
missing_per_row = df.isna().sum(axis=1)
display(missing_per_row.value_counts().sort_index().rename_axis("Ô thiếu
/ hồ sơ").rename("Số hồ sơ").to_frame())
# Biểu đồ tỷ lệ thiếu từ ít đến nhiều
plt.figure(figsize=(10, 6))
plot_missing = missing_table.sort_values("Tỷ lệ thiếu (%)")
plt.barh(plot_missing.index, plot_missing["Tỷ lệ thiếu (%)"],
color="#3976b4")
plt.xlabel("Tỷ lệ ô thiếu (%)")
plt.title("Hình 2.1. Tỷ lệ dữ liệu thiếu theo biến")
plt.tight_layout()
```

Kết quả:

Số dòng: 148,670

Số cột có missing: 14

Tổng số ô thiếu: 181,135

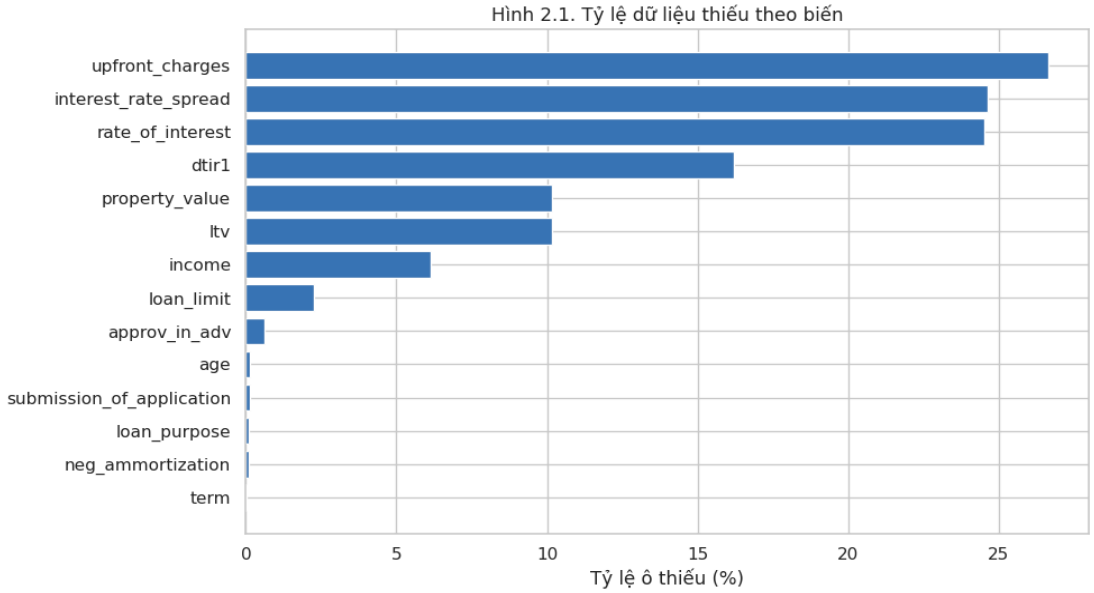
Số dòng có ≥ 1 ô thiếu: 50,483

Tỷ lệ dòng có ≥ 1 ô thiếu: 33.96%

	Số ô thiếu	Tỷ lệ thiếu (%)
upfront_charges	39642	26.664
interest_rate_spread	36639	24.645
rate_of_interest	36439	24.510
dtir1	24121	16.225
property_value	15098	10.155
ltv	15098	10.155
income	9150	6.155
loan_limit	3344	2.249
approv_in_adv	908	0.611
submission_of_application	200	0.135
age	200	0.135
loan_purpose	134	0.090
neg_ammortization	121	0.081
term	41	0.028

Ô thiếu / hồ sơ	Số hồ sơ
0	98187
1	5934
2	7449
3	20097
4	693

5	1147
6	14636
7	527



Hình 2.1. Tỷ lệ dữ liệu thiếu theo biến

Giải thích. Code thống kê số ô thiếu và tỷ lệ thiếu theo từng biến, số dòng có ít nhất một ô thiếu, và phân phối số ô thiếu trên mỗi hồ sơ (Hình 2.1).

Nhận xét. 14/28 biến có ô thiếu, tổng cộng 181.135 ô. Có 50.483 hồ sơ (33,96%) thiếu ít nhất một ô. Tỷ lệ thiếu theo biến:

- Nhóm thiếu nhiều: `upfront_charges` 26,66%, `interest_rate_spread` 24,65%, `rate_of_interest` 24,51%, `dtir1` 16,23%, `property_value` và `ltv` cùng 10,16%, `income` 6,16%.

- Nhóm thiếu rất ít: `loan_limit` 2,25%, `approv_in_adv` 0,61%, `age` và `submission_of_application` cùng 0,14%, `loan_purpose` 0,09%, `neg_ammortization` 0,08%, `term` 0,03%.

Số ô thiếu trên mỗi hồ sơ tập trung ở mức 3 ô (20.097 hồ sơ) và 6 ô (14.636 hồ sơ). Điều này gợi ý các biến thiếu cùng nhau theo cụm, không rải ngẫu nhiên từng ô.

Kết luận. Dữ liệu thiếu rõ rệt, nhưng phần lớn chỉ tập trung ở bảy biến tài chính. Mức độ thiếu cao nên cần phân tích cơ chế thiếu trước khi chọn cách xử lý.

2.2.2. Phân tích cơ chế thiếu dữ liệu

```
# Tạo bảng 2x2 giữa tình trạng thiếu và status cho từng cột bị thiếu.
```

```
mechanism_rows = []
for col in missing_table.index:
    is_missing = df[col].isna()
    table = pd.crosstab(df["status"], is_missing).reindex(index=[0, 1],
columns=[False, True], fill_value=0)
    # Chi-square kiểm định H0: tình trạng thiếu độc lập với status.
    chi2, p_value, _, _ = chi2_contingency(table, correction=False)
    # Cramér's V cho bảng 2x2 = sqrt(chi-square / n), đo cường độ liên hệ.
    cramers_v = np.sqrt(chi2 / table.values.sum())
    mechanism_rows.append({
        "Biến": col,
        "Thiếu chung (%)": is_missing.mean() * 100,
        "Thiếu | Status=0 (%)": is_missing[df["status"] == 0].mean() *
100,
        "Thiếu | Status=1 (%)": is_missing[df["status"] == 1].mean() *
100,
        "Status=1 | thiếu (%)": df.loc[is_missing, "status"].mean() * 100,
        "Cramér's V": cramers_v,
        "p-value (Chi-square)": p_value
    })
mechanism_table = pd.DataFrame(mechanism_rows).set_index("Biến")
display(mechanism_table.round(4))
print("Tỷ lệ Status=1 toàn bộ:", f"{df['status'].mean()*100:.2f}%")
print("Ghi chú: p-value hiển thị bằng 0 do giới hạn số học = rất nhỏ,
không phải xác suất bằng 0 tuyệt đối.")
```

Kết quả:

Biến	Thiếu chung (%)	Thiếu | Status=0 (%)	Thiếu | Status=1 (%)	Status=1 | thiếu (%)	Cramér's V	p-value (Chi-square)
upfront_charges	26.664	2.817	99.582	92.039	0.943	0.000
interest_rate_spread	24.645	0.000	100.000	100.000	1.000	0.000
rate_of_interest	24.510	0.000	99.454	100.000	0.996	0.000
dtir1	16.224	6.972	44.515	67.617	0.439	0.000
property_value	10.155	0.002	41.202	99.987	0.588	0.000
ltv	10.155	0.002	41.202	99.987	0.588	0.000
income	6.155	7.061	3.382	13.541	0.066	0.000
loan_limit	2.249	2.199	2.405	26.346	0.006	0.021

approv_in_adv	0.611	0.595	0.658	26.542	0.004	0.183
submission_of_application	0.135	0.000	0.546	100.000	0.064	0.000
age	0.135	0.000	0.546	100.000	0.064	0.000
loan_purpose	0.090	0.088	0.096	26.119	0.001	0.692
neg_ammortization	0.081	0.079	0.087	26.446	0.001	0.645
term	0.028	0.023	0.041	36.585	0.005	0.076

Tỷ lệ Status=1 toàn bộ: 24.64%

Ghi chú: p-value hiển thị bằng 0 do giới hạn số học = rất nhỏ, không phải xác suất bằng 0 tuyệt đối.

```
# Trực quan hóa các biến có nhiều dữ liệu thiếu trong hai nhóm status.
compare_cols = ["interest_rate_spread", "rate_of_interest",
"upfront_charges",
                "property_value", "ltv", "dtir1", "income"]
compare_missing = mechanism_table.loc[compare_cols,
                                      ["Thiếu | Status=0 (%)", "Thiếu |
Status=1 (%)"]]
ax = compare_missing.plot(kind="bar", figsize=(12, 6),
                          color=["#467caf", "#d77c47"], width=0.78)
ax.set_title("Hình 2.2. Tỷ lệ missing theo nhóm Status")
ax.set_xlabel("Biến")
ax.set_ylabel("Tỷ lệ thiếu (%)")
ax.legend(["Status=0", "Status=1"], title="Nhóm")
plt.xticks(rotation=32, ha="right")
plt.tight_layout()
# Kiểm tra các cặp cột thiếu đúng cùng một tập hồ sơ.
shared_missing = [
    ("ltv", "property_value"),
    ("age", "submission_of_application"),
    ("rate_of_interest", "interest_rate_spread"),
]
for col_a, col_b in shared_missing:
    equal_mask = df[col_a].isna().equals(df[col_b].isna())
    print(f"{col_a} ↔ {col_b}: trùng hoàn toàn cờ thiếu: {equal_mask}")
    display(pd.crosstab(df[col_a].isna(), df[col_b].isna(),
                        rownames=[f"{col_a} thiếu"], colnames=[f"{col_b}
thiếu"]))
```

Kết quả:

ltv ↔ property_value: trùng hoàn toàn cờ thiếu: True

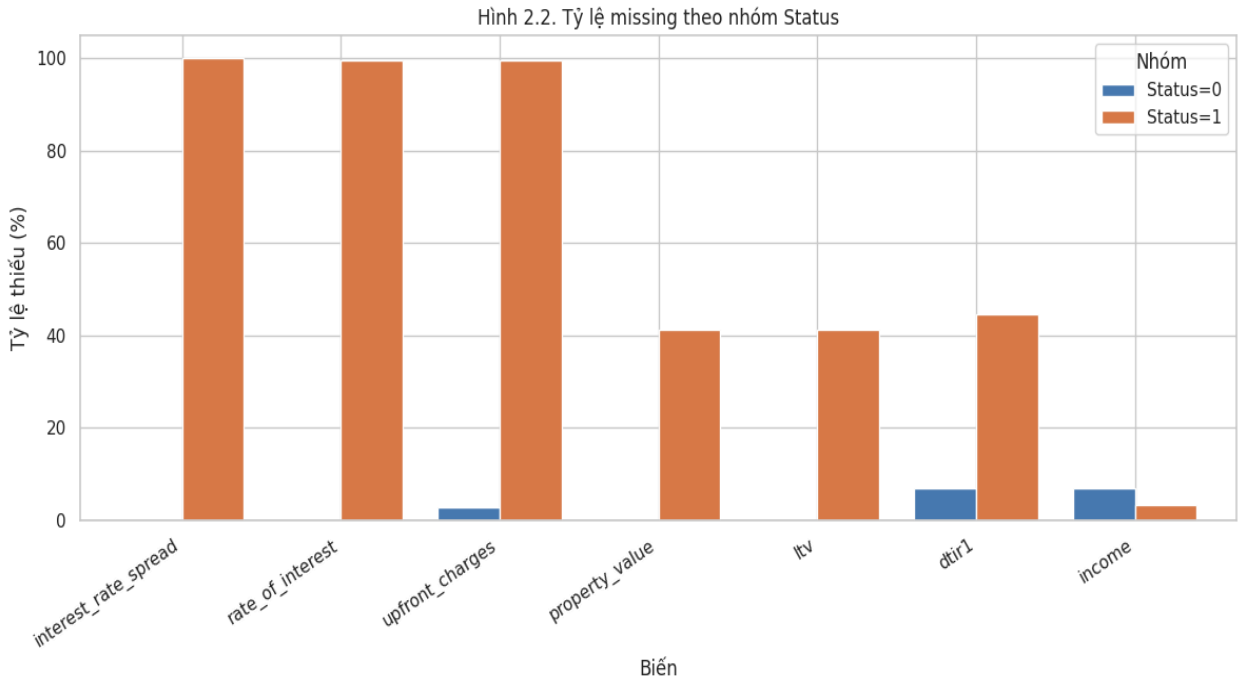
property_value thiếu	False	True
ltv thiếu		
False	133572	0
True	0	15098

age ↔ submission_of_application: trùng hoàn toàn cờ thiếu: True

submission_of_application thiếu	False	True
age thiếu		
False	148470	0
True	0	200

rate_of_interest ↔ interest_rate_spread: trùng hoàn toàn cờ thiếu: False

interest_rate_spread thiếu	False	True
rate_of_interest thiếu		
False	112031	200
True	0	36439



Giải thích. Với mỗi biến thiếu, code lập bảng chéo 2×2 giữa "thiếu hay không" và `status`. Sau đó tính tỷ lệ thiếu theo từng nhóm `status`, xác suất vỡ nợ trong nhóm thiếu, kiểm định Chi-square và hệ số Cramér's V (đo độ mạnh liên hệ). Code cũng kiểm tra các cặp biến thiếu cùng một tập hồ sơ.

Nhận xét. Có thể chia thành bốn nhóm:

Nhóm	Biến	Tỷ lệ thiếu ở Status=0 / Status=1	Cramér's V
Gần như trùng nhãn	interest_rate_spread	0% / 100%	1,000
	rate_of_interest	0% / 99,45%	0,996
	upfront_charges	2,82% / 99,58%	0,943
Liên hệ mạnh	property_value, ltv	0,002% / 41,20%	0,588
	dtir1	6,97% / 44,52%	0,439
Liên hệ yếu	income	7,06% / 3,38%	0,066
Gần như không liên hệ	loan_limit, approv_in_adv, loan_purpose, neg_ammortization, term	xấp xỉ nhau ở hai nhóm	0,001 đến 0,006

Khi `interest_rate_spread` thiếu, 100% hồ sơ vỡ nợ. Với `property_value` và `ltv`, xác suất vỡ nợ trong nhóm thiếu là 99,987%. Hai cặp `ltv`–`property_value` và `age`–`submission_of_application` thiếu đúng cùng một tập hồ sơ (15.098 và 200 hồ sơ). Riêng 200 hồ sơ thiếu `age` đều có `status = 1`. Với hơn 148 nghìn quan sát, p-value gần như luôn rất nhỏ, nên đánh giá dựa trên mức chênh lệch tỷ lệ thiếu và Cramér's V.

Kết luận. Dữ liệu thiếu của các biến tài chính không ngẫu nhiên hoàn toàn (không phải MCAR) mà phụ thuộc vào `status`. Chỉ nhóm biến thiếu rất ít là gần như độc lập với nhãn.

2.2.3. Lựa chọn phương pháp xử lý

Code:

```
# Xem dữ liệu số có lệch phải không: mean, median, skewness của từng cột.
numerical_review = ["loan_amount", "property_value", "income", "ltv",
                    "dtir1", "upfront_charges", "term"]
distribution_table = pd.DataFrame({
    "Mean": df[numerical_review].mean(),
    "Median": df[numerical_review].median(),
    "Skewness": df[numerical_review].skew(),
    "Thiếu (%)": df[numerical_review].isna().mean() * 100,
})
display(distribution_table.round(3))
print("Giá trị mode của term:", df["term"].mode(dropna=True).iloc[0])
```

Kết quả:

	Mean	Median	Skewness	Thiếu (%)
loan_amount	331,117.744	296,500.000	1.667	0.000
property_value	497,893.466	418,000.000	4.586	10.155
income	6,957.339	5,760.000	17.308	6.155
ltv	72.746	75.136	120.615	10.155
dtir1	37.733	39.000	-0.551	16.225
upfront_charges	3,224.996	2,596.450	1.754	26.664
term	335.137	360.000	-2.175	0.028

Giá trị mode của term: 360.0

Giải thích. Bảng này đối chiếu trung bình, trung vị, độ lệch (skewness) và tỷ lệ thiếu của bảy biến số. Mục đích là chọn thống kê phù hợp để điền khuyết. Với phân phối lệch, trung bình bị đuôi kéo đi, còn trung vị ổn định hơn. Code cũng in mode của `term` (360).

Biến	Trung bình	Trung vị	Skewness	Thiếu (%)
loan_amount	331.118	296.500	1,67	0
property_value	497.893	418.000	4,59	10,16
income	6.957	5.760	17,31	6,16
ltv	72,75	75,14	120,62	10,16
dtir1	37,73	39	−0,55	16,23
upfront_charges	3.225	2.596	1,75	26,66
term	335,14	360	−2,18	0,03

Nhận xét.

- Bốn biến loan_amount, property_value, income và upfront_charges đều có trung bình lớn hơn trung vị (lần lượt khoảng 12%, 19%, 21% và 24% so với trung vị) và skewness dương. Đây là phân phối lệch phải. income lệch mạnh nhất (17,31).

- ltv có skewness 120,62 nhưng trung bình (72,75) lại nhỏ hơn trung vị (75,14). Độ lệch này không đến từ một đuôi dày mà từ sáu giá trị cực trị trên 1.000% (cao nhất 7.831%) ở 2.3.1. Vì vậy chỉ số skewness của ltv bị chi phối bởi vài điểm.

- dtir1 gần đối xứng (skewness −0,55, trung bình 37,73 gần trung vị 39), nhưng thiếu tới 16,23%.

- term lệch trái (−2,18) vì hầu hết khoản vay có kỳ hạn 360 tháng, còn một số ít ngắn hơn. Trung vị và mode đều là 360.

Kết luận. Điền trung bình không phù hợp với các biến lệch phải. Với term, mode 360 trùng trung vị nên là lựa chọn hợp lý. Riêng dtir1, tính đối xứng cho phép điền bằng trung bình hoặc trung vị mà ít méo.

```
df_clean = df.copy()
# Nhóm biến phân loại cần giữ nhãn 'unknown' cho những hồ sơ không ghi
nhận.
categorical_impute_cols = ["loan_limit", "approv_in_adv", "loan_purpose",
                           "neg_ammortization", "age",
"submission_of_application"]
filled_categorical =
int(df_clean[categorical_impute_cols].isna().sum().sum())
for col in categorical_impute_cols:
```

```
    df_clean[col] = df_clean[col].fillna("unknown")
# Mode dùng để điền số ít ô thiếu của kỳ hạn vay (term).
term_mode = df_clean["term"].mode(dropna=True).iloc[0]
filled_term = int(df_clean["term"].isna().sum())
df_clean["term"] = df_clean["term"].fillna(term_mode)
# Các biến số này CỐ Ý giữ missing phục vụ phân tích phân phối/so sánh
đúng trên dữ liệu quan sát.
keep_nan_cols = ["rate_of_interest", "interest_rate_spread",
"upfront_charges",
                 "property_value", "ltv", "dtir1", "income"]
print("Số ô phân loại điền 'unknown':", f"{filled_categorical:,}")
print("Số ô term điền mode:", filled_term, "| mode:", term_mode)
print("7 cột được giữ NaN:", keep_nan_cols)
assert df_clean.shape == df.shape
assert (df_clean["status"].to_numpy() == df["status"].to_numpy()).all()
assert int(df_clean[categorical_impute_cols +
["term"]].isna().sum().sum()) == 0
# Hai nhãn unknown có thể tiết lộ nhãn; chỉ dùng đánh giá chất lượng dữ
liệu.
unknown_review = pd.DataFrame([{
    'Biến': col,
    'Số unknown': int(df_clean[col].eq('unknown').sum()),
    'Status=1 trong unknown (%)': 100 *
df_clean.loc[df_clean[col].eq('unknown'), 'status'].mean()
} for col in ['age', 'submission_of_application']])
display(unknown_review)
assert (unknown_review['Status=1 trong unknown (%)'] == 100).all()
```

Kết quả: Số ô phân loại điền 'unknown': 4,907

Số ô term điền mode: 41 | mode: 360.0

7 cột được giữ NaN: ['rate_of_interest', 'interest_rate_spread', 'upfront_charges', 'property_value', 'ltv', 'dtir1', 'income']

	Biến	Số unknown	Status=1 trong unknown (%)
0	age	200	100.000
1	submission_of_application	200	100.000

Nhận xét và kết luận: Các biến phân loại thiếu dữ liệu (`loan_limit`, `approv_in_adv`, `loan_purpose`, `neg_ammortization`, `age`, `submission_of_application`) được gán nhãn `unknown` với tổng 4.907 ô, nhằm bảo toàn số lượng hồ sơ và tránh suy đoán giá trị.

Riêng biến `term` chỉ thiếu 41 trường hợp (0,03%) nên được điền bằng mode 360 tháng cho mục đích EDA. Bảy biến số `rate_of_interest`, `interest_rate_spread`, `upfront_charges`, `property_value`, `ltv`, `dtir1` và `income` được cố ý giữ `NaN`. Với sáu biến đầu, cơ chế thiếu có liên hệ đáng kể với `status`, nên việc điền trung vị trên toàn bộ dữ liệu có thể làm sai lệch phân phối. Riêng `income` có liên hệ yếu hơn với nhãn (Cramér's V = 0,066) nhưng phân phối lệch phải rất mạnh (skewness 17,31), nên cũng không điền ở giai đoạn EDA. Trong nhánh mô hình, các phép điền thiếu được thực hiện bằng Pipeline chỉ fit trên tập Train.

Kết quả kiểm tra cho thấy 200 hồ sơ `unknown` của `age` và `submission_of_application` đều có `status = 1` (100%, so với mức nền 24,64%). Đây là dấu hiệu bất thường của dữ liệu, không phải một phân khúc khách hàng có rủi ro thực tế, nên nhóm này chỉ dùng để đánh giá chất lượng dữ liệu và bị loại khỏi nhánh mô hình.

2.2.4. So sánh dữ liệu trước và sau xử lý

Code:

```
before_missing = df.isna().sum()
after_missing = df_clean.isna().sum()
comparison_table = pd.DataFrame({
    "Thiếu trước (ô)": before_missing,
    "Thiếu sau (ô)": after_missing,
    "Đã xử lý (ô)": before_missing - after_missing,
    "Tỷ lệ thiếu trước (%)": before_missing / len(df) * 100,
    "Tỷ lệ thiếu sau (%)": after_missing / len(df_clean) * 100
}).loc[missing_table.index]
display(comparison_table.round(3))
# Chỉ số đánh giá chất lượng ở các mốc: gốc, sau cấu trúc, sau xử lý missing.
summary_before_after = pd.DataFrame([
    {"Giai đoạn": "Gốc Chương 1", "Số dòng": len(df_raw), "Số cột": df_raw.shape[1],
     "Số cột thiếu": int(df_raw.isna().any().sum()), "Tổng ô thiếu": int(df_raw.isna().sum().sum()),
     "Dòng có ô thiếu": int(df_raw.isna().any(axis=1).sum())},
    {"Giai đoạn": "Sau 2.1", "Số dòng": len(df), "Số cột": df.shape[1],
     "Số cột thiếu": int(df.isna().any().sum()), "Tổng ô thiếu": int(df.isna().sum().sum()),
     "Dòng có ô thiếu": int(df.isna().any(axis=1).sum())},
    {"Giai đoạn": "Sau 2.2", "Số dòng": len(df_clean), "Số cột": df_clean.shape[1],
     "Số cột thiếu": int(df_clean.isna().any().sum()),
     "Tổng ô thiếu": int(df_clean.isna().sum().sum()),
     "Dòng có ô thiếu": int(df_clean.isna().any(axis=1).sum())},
```

```
])
display(summary_before_after)
# Đảm bảo số ô được điền bằng đúng mức giảm missing; không bỏ hồ sơ hoặc đổi biến mục tiêu.
assert int(before_missing.sum() - after_missing.sum()) ==
filled_categorical + filled_term
assert filled_categorical == 4907 and filled_term == 41
assert int(after_missing.sum()) == 176187
assert len(df_raw) == len(df) == len(df_clean)
for col in keep_nan_cols:
   pd.testing.assert_series_equal(df_clean[col], df[col])
print("Kiểm tra tự động: KHỚP – không mất hồ sơ, không đổi 7 cột số được giữ NaN.")
```

Kết quả:

	Thiếu trước (ô)	Thiếu sau (ô)	Đã xử lý (ô)	Tỷ lệ thiếu trước (%)	Tỷ lệ thiếu sau (%)
upfront_charges	39642	39642	0	26.664	26.664
interest_rate_spread	36639	36639	0	24.645	24.645
rate_of_interest	36439	36439	0	24.510	24.510
dtir1	24121	24121	0	16.225	16.225
property_value	15098	15098	0	10.155	10.155
ltv	15098	15098	0	10.155	10.155
income	9150	9150	0	6.155	6.155
loan_limit	3344	0	3344	2.249	0.000
approv_in_adv	908	0	908	0.611	0.000
submission_of_application	200	0	200	0.135	0.000
age	200	0	200	0.135	0.000
loan_purpose	134	0	134	0.090	0.000
neg_ammortization	121	0	121	0.081	0.000

term	41	0	41	0.028	0.000

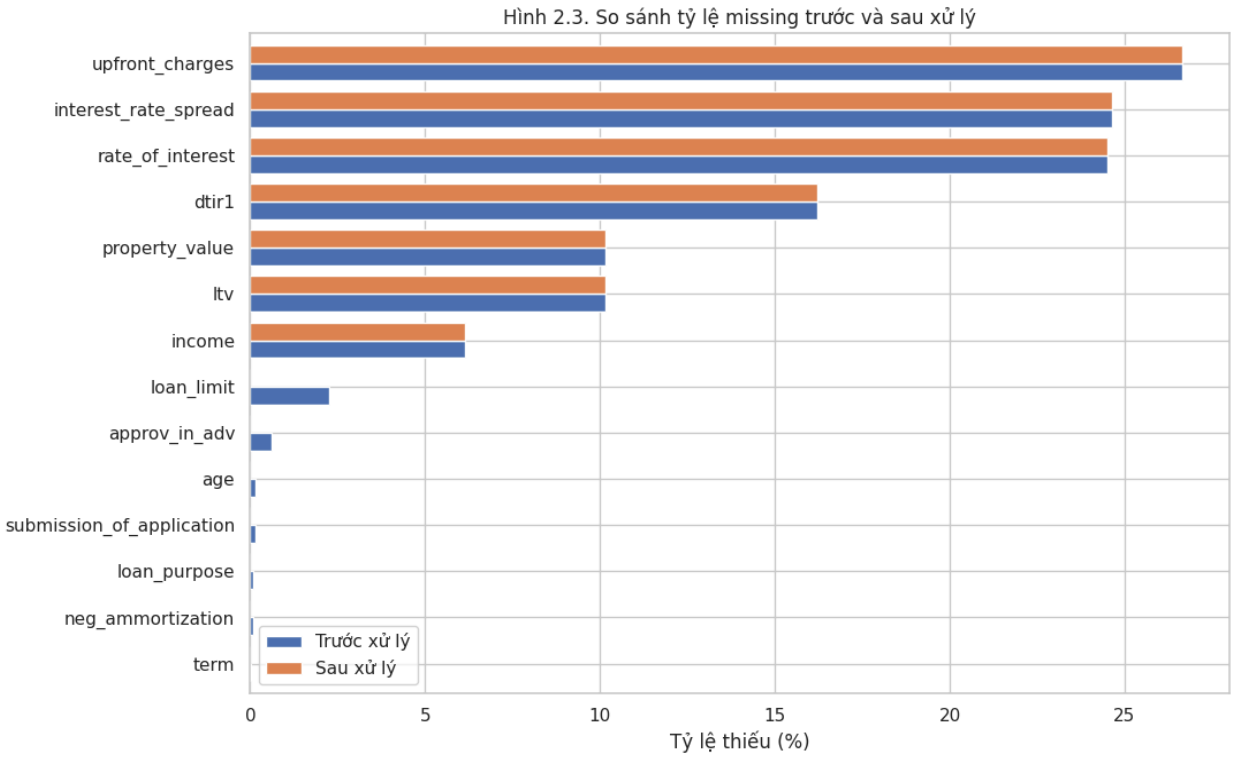
	Giai đoạn	Số dòng	Số cột	Số cột thiếu	Tổng ô thiếu	Dòng có ô thiếu
0	Gốc Chương 1	148670	34	14	181135	50483
1	Sau 2.1	148670	28	14	181135	50483
2	Sau 2.2	148670	28	7	176187	47337

Kiểm tra tự động: KHỚP – không mất hồ sơ, không đổi 7 cột số được giữ NaN.

```
# Biểu đồ so sánh tỷ lệ thiếu trước và sau; 7 biến số còn thiếu được giữ cố ý.
plot_compare = comparison_table[["Tỷ lệ thiếu trước (%)", "Tỷ lệ thiếu sau (%)"]].sort_values("Tỷ lệ thiếu trước (%)")
ax = plot_compare.plot(kind="barh", figsize=(11, 7), width=0.77)
ax.set_xlabel("Tỷ lệ thiếu (%)")
ax.set_title("Hình 2.3. So sánh tỷ lệ missing trước và sau xử lý")
ax.legend(["Trước xử lý", "Sau xử lý"])
plt.tight_layout()
# Minh họa rủi ro gây sai lệch nếu điền median vào property_value trong bộ phân tích.
observed = df["property_value"]
median_filled_demo = observed.fillna(observed.median())  # Chỉ thử nghiệm trên bản Series tách biệt.
median_impact = pd.DataFrame({
    "Status": [0, 1],
    "Mean property_value quan sát": [observed[df.status == s].mean() for s in [0, 1]],
    "Mean nếu điền median toàn bộ": [median_filled_demo[df.status == s].mean() for s in [0, 1]],
    "N thiếu": [int(observed[df.status == s].isna().sum()) for s in [0, 1]]
})
display(median_impact.round(2))
```

Kết quả:

Status	Mean property_value quan sát	Mean nếu điền median toàn bộ	N thiếu
0	505,606.070	505,604.500	2
1	457,786.010	441,393.380	15096




```
# Bàn giao dữ liệu trong bộ nhớ cho phần 2.3
print("Dữ liệu cuối 2.2 (df_clean):", df_clean.shape)
print("Số ô thiếu còn lại:", int(df_clean.isna().sum().sum()))
```

Kết quả:

Dữ liệu cuối 2.2 (df_clean): (148670, 28)

Số ô thiếu còn lại: 176187

Giải thích. Code so sánh số ô thiếu, tỷ lệ thiếu trước/sau và số dòng, số cột ở các mốc xử lý (Hình 2.3). Các lệnh `assert` kiểm tra không mất hồ sơ, không đổi `status` và bảy cột số giữ nguyên.

Nhận xét. Số cột có thiếu giảm từ 14 xuống 7. Tổng ô thiếu giảm từ 181.135 xuống 176.187, tức giảm 4.948 ô (gồm 4.907 ô phân loại và 41 ô `term`). Số hồ sơ có ô thiếu giảm từ 50.483 xuống 47.337. Số hồ sơ vẫn là 148.670, và 176.187 ô thiếu còn lại đều thuộc bảy cột được giữ có chủ đích.

Kết luận. Quá trình xử lý bảo toàn toàn bộ hồ sơ và biến mục tiêu. Dữ liệu thiếu còn lại là có chủ đích và sẽ được xử lý trong Pipeline.

2.3. Phát hiện và xử lý dữ liệu bất thường

Code:

```
df_before = df_clean.copy()
df = df_before.copy()
assert df.shape == (148670, 28), "Kích thước khác đầu ra 2.2."
assert int(df.isna().sum().sum()) == 176187
assert df["status"].isin([0, 1]).all()
print("Đầu vào 2.3 từ df_clean:", df.shape)
print("Ô thiếu còn lại:", int(df.isna().sum().sum()))
```

Kết quả:

Đầu vào 2.3 từ df_clean: (148670, 28)

Ô thiếu còn lại: 176187

2.3.1. Kiểm tra giá trị không hợp lệ theo nghiệp vụ

Kết quả:

```
# Hàm đếm trường hợp vi phạm, bỏ qua NaN đã được xử lý riêng ở phần 2.2.
def count_invalid(condition, col):
    return int((condition & df[col].notna()).sum())
# Các quy tắc kiểm tra trực tiếp về kiểu đo lường và tính dương/không
âm.
checks = {
    "status ngoài {0,1}": count_invalid(~df["status"].isin([0, 1]),
"status"),
    "loan_amount <= 0": count_invalid(df["loan_amount"] <= 0,
"loan_amount"),
    "property_value <= 0": count_invalid(df["property_value"] <= 0,
"property_value"),
    "income < 0": count_invalid(df["income"] < 0, "income"),
    "upfront_charges < 0": count_invalid(df["upfront_charges"] < 0,
"upfront_charges"),
    "term <= 0": count_invalid(df["term"] <= 0, "term"),
    "term không nguyên tháng": count_invalid(df["term"].mod(1).ne(0),
"term"),
    "rate_of_interest ngoài [0,100]":
count_invalid(~df["rate_of_interest"].between(0, 100),
"rate_of_interest"),
    "dtir1 ngoài [0,100]": count_invalid(~df["dtir1"].between(0, 100),
"dtir1"),
    "ltv <= 0": count_invalid(df["ltv"] <= 0, "ltv"),
    "credit_score < 0": count_invalid(df["credit_score"] < 0,
"credit_score"),
}
# Kiểm tra bổ sung các giá trị vô hạn ở những cột định lượng.
num_cols = df.select_dtypes(include=["number"]).columns.tolist()
```

```
checks["Giá trị số vô hạn +/-inf"] =
int(np.isinf(df[num_cols].to_numpy(dtype=float)).sum())
# Tạo và hiển thị bảng kiểm tra điều kiện nghiệp vụ.
invalid_table = pd.DataFrame({"Điều kiện kiểm tra": list(checks), "Số vi
phạm": list(checks.values())})
display(invalid_table)
print("Tổng vi phạm các quy tắc kiểm tra:", int(invalid_table["Số vi
phạm"].sum()))
```

Kết quả:

	Điều kiện kiểm tra	Số vi phạm
0	status ngoài {0,1}	0
1	loan_amount <= 0	0
2	property_value <= 0	0
3	income < 0	0
4	upfront_charges < 0	0
5	term <= 0	0
6	term không nguyên tháng	0
7	rate_of_interest ngoài [0,100]	0
8	dtir1 ngoài [0,100]	0
9	ltv <= 0	0
10	credit_score < 0	0
11	Giá trị số vô hạn +/-inf	0

Tổng vi phạm các quy tắc kiểm tra: 0

Code:

```
# Tạo những cảnh báo dữ liệu có thể là ngoại lệ nghiệp vụ, nhưng CHƯA
chứng minh là lỗi.
warning_masks = {
    "LTV > 100%": df["ltv"].gt(100),
    "LTV > 1.000%": df["ltv"].gt(1000),
    "Thu nhập bằng 0": df["income"].eq(0),
    "Lãi suất danh nghĩa bằng 0": df["rate_of_interest"].eq(0),
```

```
    "Kỳ hạn không chia hết cho 12": df["term"].mod(12).ne(0),
}
warning_table = pd.DataFrame({"Điểm cần kiểm tra": list(warning_masks),
                              "Số hồ sơ": [int(m.sum()) for m in
warning_masks.values()]})
print("Các trường hợp cần xác minh thêm theo nghiệp vụ:")
display(warning_table)
print("Các kỳ hạn lẻ tháng:", df.loc[warning_masks["Kỳ hạn không chia hết
cho 12"], "term"].value_counts().to_dict())
print("6 hồ sơ LTV > 1.000% (giữ nguyên, KHÔNG tự sửa):")
display(df.loc[warning_masks["LTV > 1.000%"],
               ["loan_amount", "property_value", "ltv", "status"]])
```

Kết quả:

Các trường hợp cần xác minh thêm theo nghiệp vụ:

	Điểm cần kiểm tra	Số hồ sơ
0	LTV > 100%	1799
1	LTV > 1.000%	6
2	Thu nhập bằng 0	1260
3	Lãi suất danh nghĩa bằng 0	1
4	Kỳ hạn không chia hết cho 12	3

Các kỳ hạn lẻ tháng: {165.0: 1, 280.0: 1, 322.0: 1}

6 hồ sơ LTV > 1.000% (giữ nguyên, KHÔNG tự sửa):

	loan_amount	property_value	ltv	status
16951	626500	8,000.000	7,831.250	0
46287	236500	8,000.000	2,956.250	1
47807	416500	8,000.000	5,206.250	0
55286	536500	8,000.000	6,706.250	0
65238	376500	8,000.000	4,706.250	0
123343	186500	8,000.000	2,331.250	0

Nhận xét. Kết quả là 0 vi phạm ở cả 12 quy tắc. Các cảnh báo cần xác minh thêm là: 1.799 hồ sơ có LTV > 100%, 1.260 hồ sơ có `income` = 0, 1 hồ sơ có lãi suất bằng 0, 3 hồ sơ có `term` không chia hết cho 12 (165, 280 và 322 tháng). Đáng chú ý là 6 hồ sơ có LTV > 1.000%. Cả sáu đều có `property_value` đúng bằng 8.000 trong khi khoản vay từ 186.500 đến 626.500, và đều thiếu `income`. Điều này gợi ý lỗi nhập giá trị tài sản, nhưng chưa thể khẳng định nên được giữ nguyên.

2.3.2. Phát hiện ngoại lai bằng IQR

Code:

```
# Chọn các biến số phù hợp để dò ngoại lai; loại status và term khỏi phép tính IQR.
iqr_cols = ["loan_amount", "rate_of_interest", "interest_rate_spread",
            "upfront_charges", "property_value", "income",
            "credit_score", "ltv", "dtir1"]
# Lưu cờ ngoại lai cho từng hồ sơ và thống kê theo mỗi biến.
outlier_flags = pd.DataFrame(False, index=df.index, columns=iqr_cols)
rows = []
for col in iqr_cols:
    # Pandas bỏ qua NaN khi tính phân vị và khi so sánh điều kiện.
    series = df[col]
    q1 = float(series.quantile(0.25))
    q3 = float(series.quantile(0.75))
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    flag = (series < lower) | (series > upper)
    outlier_flags[col] = flag.fillna(False)
    n_valid = int(series.notna().sum())
    rows.append({"Biến": col, "N có dữ liệu": n_valid,
                 "Q1": q1, "Q3": q3, "IQR": iqr,
                 "Ngưỡng dưới": lower, "Ngưỡng trên": upper,
                 "Ngoại lai thấp": int((series < lower).sum()),
                 "Ngoại lai cao": int((series > upper).sum()),
                 "Tổng ngoại lai": int(flag.sum()),
                 "Tỷ lệ/giá trị hợp lệ (%)": 100 * flag.sum() / n_valid})
iqr_table = pd.DataFrame(rows)
print("Bảng tính các khoảng IQR (không bao gồm biến status/term):")
display(iqr_table.round(3))
print("Số dòng có ít nhất một ngoại lai IQR:",
int(outlier_flags.any(axis=1).sum()))
print("Tổng lượt gắn cờ IQR (có thể trùng nhau trên cùng hồ sơ):",
int(outlier_flags.sum().sum()))
```

Kết quả:

Bảng tính các khoảng IQR (không bao gồm biến status/term):

	Biến	N có dữ liệu	Q1	Q3	IQR	Ngưỡng dưới	Ngưỡng trên	Ngoại lai thấp	Ngoại lai cao	Tổng ngoại lai	Tỷ lệ/giá trị hợp lệ (%)
0	loan_amount	148670	196,500.000	436,500.000	240,000.000	-163,500.000	796,500.000	0	1895	1895	1.275
1	rate_of_interest	112231	3.625	4.375	0.750	2.500	5.500	10	846	856	0.763
2	interest_rate_spread	112031	0.076	0.775	0.699	-0.973	1.824	31	414	445	0.397
3	upfront_charges	109028	581.490	4,812.500	4,231.010	-5,765.025	11,159.015	0	2880	2880	2.642
4	property_value	133572	268,000.000	628,000.000	360,000.000	-272,000.000	1,168,000.000	0	5266	5266	3.942
5	income	139520	3,720.000	8,520.000	4,800.000	-3,480.000	15,720.000	0	6546	6546	4.692
6	credit_score	148670	599.000	800.000	201.000	297.500	1,101.500	0	0	0	0.000
7	ltv	133572	60.475	86.184	25.709	21.911	124.748	1762	120	1882	1.409
8	dtir1	124549	31.000	45.000	14.000	10.000	66.000	2013	0	2013	1.616

Số dòng có ít nhất một ngoại lai IQR: 16400

Tổng lượt gắn cờ IQR (có thể trùng nhau trên cùng hồ sơ): 21783

Code:

```
# Phân tích riêng thời hạn vay bằng tần suất, tránh việc IQR = 0 làm sai kết luận.
term_q1 = float(df["term"].quantile(0.25))
```

```
term_q3 = float(df["term"].quantile(0.75))
term_diff_360 = int(df["term"].ne(360).sum())
print(f"term: Q1={term_q1:g}, Q3={term_q3:g}, IQR={term_q3 - term_q1:g}.")
print(f"Có {term_diff_360:,} khoản vay với term khác 360 tháng; không kết luận những hồ sơ này là lỗi.")
print("Các kỳ hạn phổ biến:")
display(df["term"].value_counts().head(8).rename("Số hồ sơ").to_frame())
```

Kết quả:

term: Q1=360, Q3=360, IQR=0.

Có 26,944 khoản vay với term khác 360 tháng; không kết luận những hồ sơ này là lỗi.

Các kỳ hạn phổ biến:

term	Số hồ sơ
360.000	121726
180.000	12981
240.000	5859
300.000	2822
324.000	2766
120.000	510
144.000	263
348.000	260

Giải thích. Với chín biến số, ngưỡng ngoại lai là Q1 − 1,5×IQR và Q3 + 1,5×IQR, tính trên các giá trị quan sát được. `term` được phân tích riêng bằng tần suất vì Q1 = Q3 = 360 làm IQR = 0.

Nhận xét. Số ngoại lai theo biến: `income` 6.546 (4,69%), `property_value` 5.266 (3,94%), `upfront_charges` 2.880 (2,64%), `dtir1` 2.013 (1,62%, toàn bộ ở phía thấp), `loan_amount` 1.895 (1,28%, toàn bộ ở phía cao), `ltv` 1.882 (1,41%, gồm 1.762 thấp và 120 cao), `rate_of_interest` 856 (0,76%), `interest_rate_spread` 445 (0,40%). `credit_score` không có ngoại lai (ngưỡng 297,5 đến 1.101,5). Tổng cộng 21.783 lượt gắn cờ, rơi vào 16.400 hồ sơ. Về `term`, 121.726 khoản vay (81,9%) có kỳ hạn 360 tháng, 26.944

khoản khác 360 tháng. Các kỳ hạn này (180, 240, 300, 324 tháng...) là kỳ hạn hợp lệ, không phải lỗi.

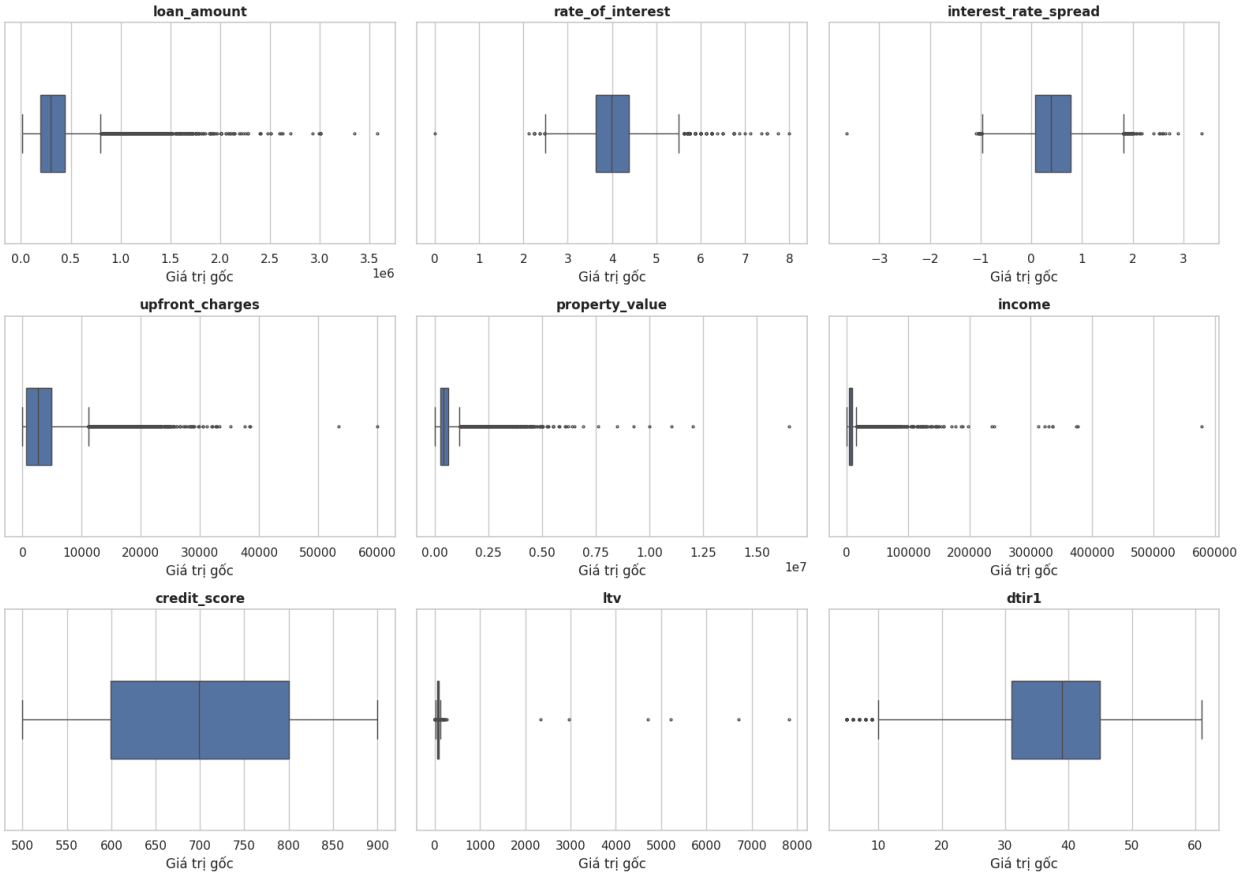
Kết luận. Ngoại lai tập trung ở các biến tiền tệ lệch phải (`income`, `property_value`, `loan_amount`, `upfront_charges`). Phần lớn là giá trị lớn nhưng có thể có thật.

2.3.3. Trực quan hóa ngoại lai

Code:

```
# Hình 2.4: Vẽ boxplot 9 biến, chia theo từng ô vì đơn vị đo khác nhau.
fig, axes = plt.subplots(3, 3, figsize=(16, 12))
for ax, col in zip(axes.flat, iqr_cols):
   sns.boxplot(x=df[col].dropna(), ax=ax, fliersize=2,
               linewidth=1, width=.35)
   ax.set_title(col, fontsize=12, fontweight="bold")
   ax.set_xlabel("Giá trị gốc")
fig.suptitle("Hình 2.4 - Boxplot các biến định lượng (giữ nguyên ngoại
lai)",
             fontsize=15, fontweight="bold", y=1.01)
fig.tight_layout()
```

Hình 2.4 - Boxplot các biến định lượng (giữ nguyên ngoại lai)



Nhận xét. Hình gồm chín boxplot, mỗi biến một ô vì đơn vị khác nhau. Có thể chia thành ba nhóm.

- Nhóm biến tiền tệ lệch phải mạnh (`loan_amount`, `upfront_charges`, `property_value`, `income`).

+ Hộp bị nén sát gốc trục, kèm một dải điểm ngoại lai dày đặc kéo dài về bên phải.
+ `loan_amount` có ngoại lai từ khoảng 0,8 triệu đến hơn 3,5 triệu.
+ `upfront_charges` có dải ngoại lai từ khoảng 11 nghìn đến gần 40 nghìn, kèm hai điểm xa ở khoảng 53 nghìn và 60 nghìn.
+ `property_value` có một điểm cách xa tới khoảng 16,5 triệu.
+ `income` có điểm xa nhất khoảng 578 nghìn.
+ Do trục kéo dài theo các điểm cực trị, phần trung tâm phân phối (hộp) gần như không đọc được. Đây là lý do cần biến đổi log ở mục 2.5.

- Nhóm biến tương đối cân đối (`rate_of_interest`, `interest_rate_spread`, `dtir1`, `credit_score`).

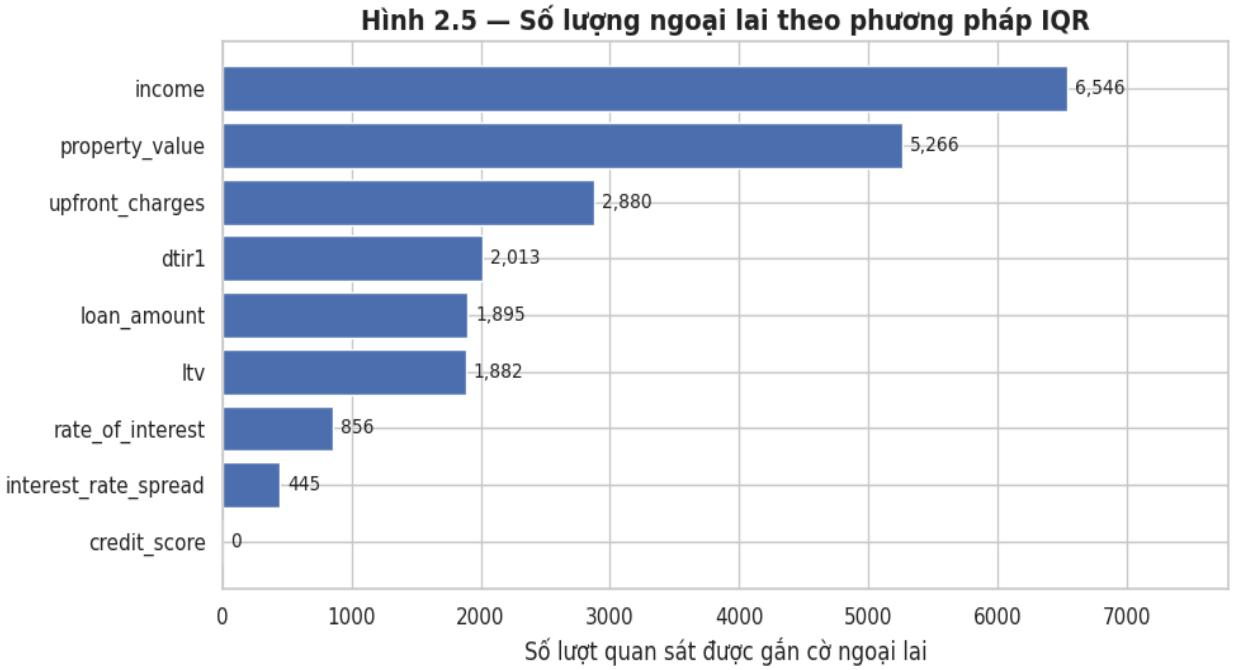
+ `rate_of_interest` có hộp quanh 3,6 đến 4,4, trung vị khoảng 4. Ngoại lai chủ yếu ở phía cao (đến 8), kèm một điểm ở 0 (hồ sơ lãi suất bằng 0 đã nêu ở 2.3.1).
+ `interest_rate_spread` có hộp gọn quanh 0 đến 0,8, và một điểm tách biệt ở khoảng −3,7.
+ `dtir1` có hộp từ 31 đến 45, trung vị 39. Ngoại lai chỉ nằm ở phía thấp (nhóm điểm dưới 10) và râu phải chạm 61 mà không có điểm nào vượt. Điều này khớp với bảng IQR (2.013 ngoại lai, toàn bộ ở phía thấp).
+ `credit_score` không có ngoại lai. Hộp rộng (600 đến 800) và râu phủ trọn khoảng 500 đến 900, cho thấy điểm tín dụng phân bố khá đều.

- **`ltv`**. Hộp và râu bị nén thành một khối rất nhỏ sát gốc, trong khi sáu điểm cô lập rải từ khoảng 2.300% đến hơn 7.800%. Sáu điểm này chính là sáu hồ sơ LTV > 1.000% đã nêu ở 2.3.1. Vì chúng kéo giãn trục nên cần xem thêm Hình 2.6 (phóng vùng đến P99) mới thấy được phân phối thật.

Kết luận. Ngoại lai tập trung ở đuôi phải của các biến tiền tệ, còn `credit_score` sạch hoàn toàn. Các điểm cực đoan nhất (`ltv`, `property_value`, `income`) là các điểm cô lập, không phải một cụm lớn, nên cần rà soát thay vì xóa. Hình cũng là lý do chọn biến đổi log và `RobustScaler` ở các mục sau.

```
#Hình 2.5: So sánh số lượng ngoại lai IQR trên cùng một đồ thị.
fig, ax = plt.subplots(figsize=(10, 5))
plot_counts = iqr_table.sort_values("Tổng ngoại lai", ascending=True)
ax.barh(plot_counts["Biến"], plot_counts["Tổng ngoại lai"])
for i, v in enumerate(plot_counts["Tổng ngoại lai"]):
    ax.text(v + 60, i, f"{v:,}", va="center", fontsize=10)
ax.set_xlim(0, max(plot_counts["Tổng ngoại lai"]) * 1.19)
ax.set_xlabel("Số lượt quan sát được gắn cờ ngoại lai")
ax.set_title("Hình 2.5 — Số lượng ngoại lai theo phương pháp IQR",
fontsize=14, fontweight="bold")
fig.tight_layout()
```

Kết quả:



Nhận xét. Biểu đồ cột ngang xếp giảm dần số lượt gắn cờ: `income` 6.546, `property_value` 5.266, `upfront_charges` 2.880, `dtir1` 2.013, `loan_amount` 1.895, `ltv` 1.882, `rate_of_interest` 856, `interest_rate_spread` 445 và `credit_score` 0.

- Hai biến `income` và `property_value` chiếm khoảng 54% trong tổng 21.783 lượt gắn cờ. Cộng thêm `upfront_charges` thì ba biến này chiếm khoảng 67%.
- Đây là số lượng tuyệt đối, chưa tính đến cỡ mẫu từng biến. Theo tỷ lệ trên số giá trị hợp lệ, thứ tự có đổi ở cuối bảng: `ltv` (1,41%) nhỉnh hơn `loan_amount` (1,28%), dù số lượng của `ltv` (1.882) thấp hơn. Nguyên nhân là `ltv` có ít giá trị hợp lệ hơn (133.572 so với 148.670).

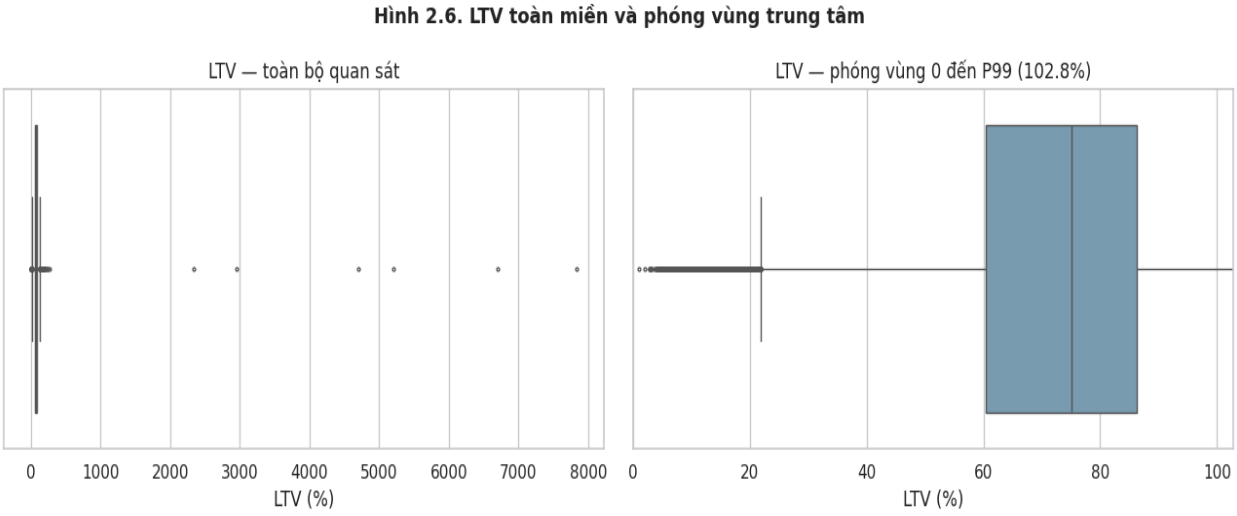
  Hình không phân biệt ngoại lai phía thấp và phía cao. Ví dụ, ngoại lai của `dtir1` đều ở phía thấp còn của `loan_amount` đều ở phía cao.
- Con số là lượt gắn cờ, một hồ sơ có thể bị gắn cờ ở nhiều biến. 21.783 lượt tương ứng 16.400 hồ sơ.

Kết luận. Ngoại lai chủ yếu nằm ở `income` và `property_value`, hai biến tiền tệ lệch phải mạnh, trong khi `credit_score` không có ngoại lai. Số lượng lớn này phản ánh đặc tính đuôi dài của dữ liệu tài chính hơn là lỗi nhập liệu, nên chúng được giữ lại và giảm ảnh hưởng bằng biến đổi ở các mục sau.

```
# Hình 2.6: so sánh toàn bộ LTV với vùng phóng đến P99
ltv_nonnull = df['ltv'].dropna()
ltv_p99 = float(ltv_nonnull.quantile(.99))
print('Số LTV cực trị >1.000%:', int(ltv_nonnull.gt(1000).sum()))
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
sns.boxplot(x=ltv_nonnull, ax=axes[0], color='#6E9DBB', fliersize=2)
axes[0].set_title('LTV — toàn bộ quan sát')
axes[0].set_xlabel('LTV (%)')
sns.boxplot(x=ltv_nonnull, ax=axes[1], color='#6E9DBB', fliersize=2)
axes[1].set_xlim(0, ltv_p99)
axes[1].set_title(f'LTV — phóng vùng 0 đến P99 ({ltv_p99:.1f}%)')
axes[1].set_xlabel('LTV (%)')
fig.suptitle('Hình 2.6. LTV toàn miền và phóng vùng trung tâm',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

Kết quả:

Số LTV cực trị >1.000%: 6



Nhận xét.

- Ô trái (toàn bộ quan sát, thang 0 đến hơn 8.000%). Gần như toàn bộ dữ liệu bị nén thành một vệt sát gốc trục, không đọc được hộp hay trung vị. Sáu điểm cô lập nằm rất xa, ở khoảng 2.300%, 3.000%, 4.700%, 5.200%, 6.700% và 7.800%. Đó chính là sáu hồ sơ LTV > 1.000% đã nêu ở 2.3.1, đều có `property_value` bằng 8.000. Hình này cho thấy chúng là các điểm tách biệt hẳn khỏi phần còn lại, không thuộc một cụm nào.

- Ô phải (phóng vùng 0 đến P99, tức khoảng 102,8%). Phần trung tâm hiện rõ.

+ Hộp từ khoảng 60,5% đến 86,2%, trung vị khoảng 75%. Các số này khớp với Q1 = 60,48, Q3 = 86,18 và trung vị 75,14 trong các bảng trước.
+ Râu trái dừng ở khoảng 22% (ngưỡng dưới IQR là 21,9), và phía dưới râu có một dải điểm ngoại lai dày đặc từ khoảng 1% đến 22%.
+ Râu phải kéo dài đến sát mép cửa sổ, vì ngưỡng trên IQR (124,7%) lớn hơn P99. Vì vậy 120 ngoại lai phía cao (LTV trên 124,7%) nằm ngoài khung nhìn của ô này.

- Phân phối trung tâm khá cân đối, nhưng đa số ngoại lai IQR của `ltv` nằm ở phía thấp (1.762 điểm) chứ không phải phía cao (120 điểm). LTV thấp nghĩa là khoản vay nhỏ so với giá trị tài sản, một tình huống hợp lý, không phải lỗi.

- Điều này giải thích vì sao skewness của `ltv` ở 2.2.3 lên tới 120,62 trong khi trung bình (72,75) và trung vị (75,14) gần nhau: chỉ số này bị sáu điểm cực trị chi phối.

Kết luận. Hình 2.6 cho thấy phân phối LTV thực chất tập trung quanh 60% đến 86%. Ngoại lai IQR chủ yếu là các khoản vay có LTV thấp (hợp lệ), còn các giá trị bất thường thật sự chỉ là sáu điểm trên 1.000%, được giữ nguyên và gắn cờ để rà soát. Việc dùng hai thang đo cạnh nhau là cần thiết, vì chỉ riêng thang toàn miền sẽ che mất hình dạng thật của phân phối.

2.3.4. Quyết định xử lý ngoại lai

Code:

```
df_after = df.copy()
# Tạo danh sách hồ sơ cần rà soát
audit_mask = outlier_flags.any(axis=1) | pd.concat(warning_masks,
axis=1).any(axis=1)
audit = pd.DataFrame(index=df.index)
audit["dong_trong_file_2_2"] = df.index + 2
audit["status"] = df["status"]
for col in iqr_cols:
   audit[f"iqr_{col}"] = outlier_flags[col].astype(int)
for label, flag in warning_masks.items():
   audit["canh_bao_" + {"LTV > 100%":"ltv_gt_100", "LTV >
1.000%":"ltv_gt_1000",
                "Thu nhập bằng 0":"income_eq_0", "Lãi suất danh nghĩa bằng
0":"rate_eq_0",
                "Kỳ hạn không chia hết cho
12":"term_not_multiple_12"}[label]] = flag.astype(int)
# Ghi chỉ những dòng có ít nhất một cờ để dễ kiểm tra nguồn.
audit_flagged = audit.loc[audit_mask].copy()

# Kiểm chứng: không xóa dòng, không đổi cột, không sửa dữ liệu đã xử lý ở
2.2.
pd.testing.assert_frame_equal(df_before, df_after, check_dtype=True,
check_exact=True)
```

```
assert df_after.shape == (148670, 28)
assert int(df_after.isna().sum().sum()) == 176187
assert len(audit_flagged) == int(audit_mask.sum())
# In bảng so sánh trước và sau để minh bạch toàn bộ lựa chọn.
compare = pd.DataFrame({
   "Chỉ tiêu": ["Số hồ sơ", "Số biến", "Số ô thiếu", "Số hồ sơ có ngoại
lai IQR",
                "Số hồ sơ bị loại", "Số giá trị bị sửa"],
   "Trước mục 2.3": [len(df_before), df_before.shape[1],
int(df_before.isna().sum().sum()),
                     int(outlier_flags.any(axis=1).sum()), 0, 0],
   "Sau mục 2.3": [len(df_after), df_after.shape[1],
int(df_after.isna().sum().sum()),
                   int(outlier_flags.any(axis=1).sum()), 0, 0],
})
display(compare)
print("Số hồ sơ trong danh sách kiểm tra (cờ IQR HOẶC cảnh báo nghiệp
vụ):", len(audit_flagged))
```

Kết luận:

	Chỉ tiêu	Trước mục 2.3	Sau mục 2.3
0	Số hồ sơ	148670	148670
1	Số biến	28	28
2	Số ô thiếu	176187	176187
3	Số hồ sơ có ngoại lai IQR	16400	16400
4	Số hồ sơ bị loại	0	0
5	Số giá trị bị sửa	0	0

Số hồ sơ trong danh sách kiểm tra (cờ IQR HOẶC cảnh báo nghiệp vụ): 19130

2.4. Kiểm tra tính nhất quán giữa các biến

Code:

```
df_input = df_after.copy()
df = df_input.copy()
assert df.shape == (148670, 28)
assert int(df.isna().sum().sum()) == 176187
assert set(df["status"].unique()) == {0, 1}
print("Đầu vào 2.4 từ df_after:", df.shape)
```

```
print("Số hồ sơ status=1:", int(df["status"].eq(1).sum()))
```

Kết quả:

Đầu vào 2.4 từ df_after: (148670, 28)

Số hồ sơ status=1: 36639

2.4.1. Kiểm tra tính hợp lý của LTV

Code:

```
# Chỉ so sánh công thức ở những hồ sơ có đủ 3 biến và tài sản thế chấp
dương.
valid = df[['loan_amount', 'property_value', 'ltv']].notna().all(axis=1) &
df['property_value'].gt(0)
ltv_calculated = df.loc[valid, 'loan_amount'] / df.loc[valid,
'property_value'] * 100
ltv_error = (df.loc[valid, 'ltv'] - ltv_calculated).abs()

# Xác định số dòng sai khác ngoài sai số làm tròn máy tính.
tolerance = 1e-6
ltv_summary = pd.DataFrame([
   {'Chỉ tiêu': 'Số hồ sơ có đủ ba biến để kiểm tra', 'Giá trị':
int(valid.sum())},
   {'Chỉ tiêu': 'LTV lệch công thức > 1e-6', 'Giá trị':
int(ltv_error.gt(tolerance).sum())},
   {'Chỉ tiêu': 'Sai số tuyệt đối lớn nhất', 'Giá trị':
float(ltv_error.max())},
   {'Chỉ tiêu': 'Cả property_value và ltv đều thiếu', 'Giá trị':
int((df['property_value'].isna() & df['ltv'].isna()).sum())},
   {'Chỉ tiêu': 'Chỉ một trong hai biến thiếu', 'Giá trị':
int((df['property_value'].isna() ^ df['ltv'].isna()).sum())},
   {'Chỉ tiêu': 'LTV > 100%', 'Giá trị': int(df['ltv'].gt(100).sum())},
   {'Chỉ tiêu': 'LTV > 1.000%', 'Giá trị': int(df['ltv'].gt(1000).sum())},
])
print('Bảng 2.4.1 — Đối chiếu LTV:')
display(ltv_summary)
# Sáu hồ sơ cực trị chỉ chuyển sang danh sách kiểm tra, không tự sửa hoặc
xóa.
ltv_extreme = df.loc[df['ltv'].gt(1000),
   ['loan_amount', 'property_value', 'ltv', 'income', 'status']].copy()
ltv_extreme.insert(0, 'dong_du_lieu', ltv_extreme.index + 2)
print('Sáu dòng LTV > 1000%:')
display(ltv_extreme)
assert int(ltv_error.gt(tolerance).sum()) == 0
assert int((df['property_value'].isna() ^ df['ltv'].isna()).sum()) == 0
```

Kết quả:

Bảng 2.4.1 - Đối chiếu LTV:

	Chỉ tiêu	Giá trị
0	Số hồ sơ có đủ ba biến để kiểm tra	133,572.000
1	LTV lệch công thức > 1e-6	0.000
2	Sai số tuyệt đối lớn nhất	0.000
3	Cả property_value và ltv đều thiếu	15,098.000
4	Chỉ một trong hai biến thiếu	0.000
5	LTV > 100%	1,799.000
6	LTV > 1.000%	6.000

Sáu dòng LTV > 1000%:

	dong_du_lieu	loan_amount	property_value	ltv	income	status
16951	16953	626500	8,000.000	7,831.250	NaN	0
46287	46289	236500	8,000.000	2,956.250	NaN	1
47807	47809	416500	8,000.000	5,206.250	NaN	0
55286	55288	536500	8,000.000	6,706.250	NaN	0
65238	65240	376500	8,000.000	4,706.250	NaN	0
123343	123345	186500	8,000.000	2,331.250	NaN	0

Giải thích. Code tính lại LTV = `loan_amount` / `property_value` × 100 và so với cột `ltv` trên các hồ sơ có đủ ba biến, với dung sai 1e-6.

Nhận xét. Có 133.572 hồ sơ đủ ba biến, 0 hồ sơ lệch quá dung sai, sai số lớn nhất bằng 0. 15.098 hồ sơ thiếu cả `property_value` lẫn `ltv`, và không có hồ sơ nào chỉ thiếu một trong hai. Hệ quả: LTV là biến suy ra hoàn toàn từ hai biến kia. Khi thiếu cả hai, không thể dùng công thức để khôi phục. Khi đưa cả ba vào mô hình tuyến tính sẽ gây đa cộng tuyến. Thêm 1.799 hồ sơ có LTV > 100%, trong đó 6 hồ sơ vượt 1.000% (đã nêu ở 2.3.1).

Kết luận. Cột `ltv` nhất quán tuyệt đối với công thức. Do đó chỉ cần giữ một biểu diễn trong mô hình, và dữ liệu thiếu của hai biến này không thể điền bằng công thức.

2.4.2. Kiểm tra sự đồng nhất về thời điểm ghi nhận

```
# Bộ dữ liệu chỉ có cùng năm 2019 và không có trường ngày để đối chiếu trình tự.
assert df_raw['year'].nunique(dropna=False) == 1
print('Năm duy nhất:', df_raw['year'].iloc[0])
print('Không có mốc ngày tiếp nhận, giải ngân và phát sinh status: chưa xác minh được thứ tự thời gian.')
```

Kết quả:

Năm duy nhất: 2019

Không có mốc ngày tiếp nhận, giải ngân và phát sinh status: chưa xác minh được thứ tự thời gian.

Giải thích. Code kiểm tra biến `year` và tìm các trường ngày tháng để đối chiếu thứ tự thời gian.

Nhận xét. Toàn bộ dữ liệu thuộc năm 2019 và không có mốc ngày tiếp nhận, giải ngân hay phát sinh `status`. Vì vậy không thể xác minh biến nào được ghi nhận trước hoặc sau khi biết kết quả vỡ nợ.

Kết luận. Không kiểm chứng được thứ tự thời gian. Do đó ở các bước sau áp dụng nguyên tắc thận trọng: biến chưa rõ thời điểm ghi nhận được loại khỏi nhánh mô hình.

2.4.3. Kiểm tra Data Leakage

Code:

```
# Đối chiếu xác suất status=1 trong nhóm thiếu với mức nền toàn mẫu.
missing_cols = ['interest_rate_spread', 'rate_of_interest',
'upfront_charges',
                'property_value', 'ltv', 'dtir1', 'income']
baseline = 100 * df['status'].mean()
leakage_rows = []
for col in missing_cols:
   missing = df[col].isna()
   pct0 = 100 * missing[df['status'].eq(0)].mean()
   pct1 = 100 * missing[df['status'].eq(1)].mean()
   p_default_missing = 100 * df.loc[missing, 'status'].mean()
   leakage_rows.append({
```

```
            'Biến': col,
            'Thiếu khi status=0 (%)': pct0,
            'Thiếu khi status=1 (%)': pct1,
            'P(status=1 | thiếu) (%)': p_default_missing,
            'Mức nền status=1 (%)': baseline,
            'Cramér V': mechanism_table.loc[col, "Cramér's V"]
        })
leakage_table = pd.DataFrame(leakage_rows)
print('Bảng 2.4.3 — Liên hệ giữa thiếu dữ liệu và status:')
display(leakage_table.round(3))
# Kiểm tra mối liên hệ hoàn hảo ở cột interest_rate_spread.
spread_missing = df['interest_rate_spread'].isna()
display(pd.crosstab(df['status'], spread_missing))
assert spread_missing.astype(int).eq(df['status']).all()
# Hai nhãn unknown: 200 hồ sơ và tất cả status=1, KHÔNG dùng nhóm này để
giải thích EDA.
for col in ['age', 'submission_of_application']:
    mask = df[col].eq('unknown')
    print(col, ':', int(mask.sum()), 'unknown; P(status=1 | unknown) =',
          round(100 * df.loc[mask, 'status'].mean(), 2), '%')
```

Kết quả:

Bảng 2.4.3 - Liên hệ giữa thiếu dữ liệu và status:

	Biến	Thiếu khi status=0 (%)	Thiếu khi status=1 (%)	P(status=1 | thiếu) (%)	Mức nền status=1 (%)	Cramér V
0	interest_rate_spread	0.000	100.000	100.000	24.645	1.000
1	rate_of_interest	0.000	99.454	100.000	24.645	0.996
2	upfront_charges	2.817	99.582	92.039	24.645	0.943
3	property_value	0.002	41.202	99.987	24.645	0.588
4	ltv	0.002	41.202	99.987	24.645	0.588
5	dtir1	6.972	44.515	67.617	24.645	0.439
6	income	7.061	3.382	13.541	24.645	0.066

interest_rate_spread	False	True
status		

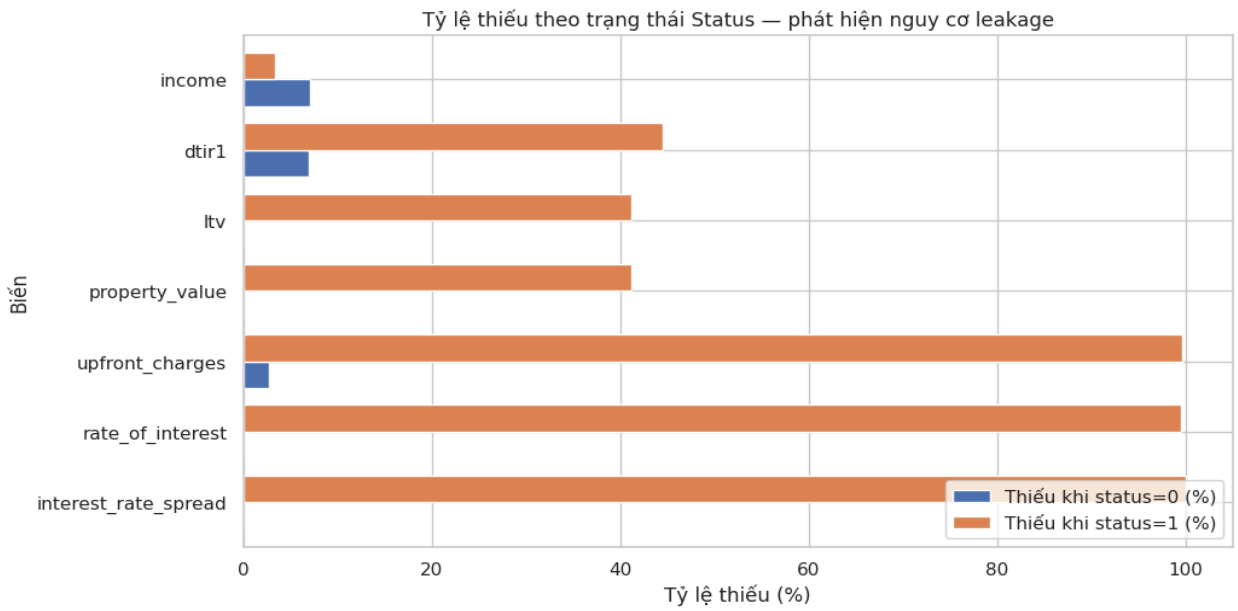
0	112031	0
1	0	36639

age : 200 unknown; P(status=1 | unknown) = 100.0 %

submission_of_application : 200 unknown; P(status=1 | unknown) = 100.0 %

```
# Vẽ tỷ lệ thiếu của các biến trọng điểm theo status
plot_cols = ['interest_rate_spread', 'rate_of_interest',
'upfront_charges',
             'property_value', 'ltv', 'dtir1', 'income']
chart_data = (leakage_table.set_index('Biến')
              .loc[plot_cols, ['Thiếu khi status=0 (%)', 'Thiếu khi
status=1 (%)']])
ax = chart_data.plot(kind='barh', figsize=(10.5, 5.8), width=0.76)
ax.set_xlabel('Tỷ lệ thiếu (%)')
ax.set_ylabel('Biến')
ax.set_xlim(0, 105)
ax.set_title('Tỷ lệ thiếu theo trạng thái Status — phát hiện nguy cơ
leakage')
ax.legend(loc='lower right')
plt.tight_layout()
```

Kết quả:



Bảng 2.4.3 cho thấy ba biến `interest_rate_spread`, `rate_of_interest` và `upfront_charges` có tình trạng thiếu gần như trùng hoàn toàn với `status`. Với `interest_rate_spread`, toàn bộ 112.031 hồ sơ `status = 0` đều có giá trị và toàn bộ 36.639 hồ sơ `status = 1` đều thiếu (Cramér's V = 1,000), nên biết ô này thiếu hay không là

biết luôn nhãn. `rate_of_interest` (thiếu ở 99,45% hồ sơ vỡ nợ, xác suất vỡ nợ khi thiếu là 100%) và `upfront_charges` (thiếu ở 99,58% hồ sơ vỡ nợ, so với 2,82% ở nhóm còn lại) có biểu hiện tương tự. Vì vậy ba biến này có nguy cơ rò rỉ nhãn nghiêm trọng, không được đưa vào mô hình dự báo, kể cả các cờ báo thiếu tương ứng.

`property_value` và `ltv` thiếu ở 41,20% hồ sơ vỡ nợ nhưng chỉ 0,002% hồ sơ không vỡ nợ. Khi hai biến này thiếu, 99,987% hồ sơ vỡ nợ (Cramér's V = 0,588), cao hơn nhiều so với mức nền 24,64%. `dtir1` có mức liên hệ thấp hơn nhưng vẫn rõ: thiếu ở 44,52% hồ sơ vỡ nợ so với 6,97% ở nhóm còn lại, và xác suất vỡ nợ khi thiếu là 67,62%. Do cơ chế thiếu liên quan một phần đến `status`, ba biến này được giữ để phân tích mô tả nhưng tạm loại khỏi mô hình do chưa xác minh thời điểm ghi nhận. Dữ liệu không có trường ngày tháng (mục 2.4.2) nên chưa thể kiểm chứng thứ tự thời gian. Ngược lại, `income` thiếu ở 3,38% hồ sơ vỡ nợ và 7,06% hồ sơ không vỡ nợ, xác suất vỡ nợ khi thiếu (13,54%) thấp hơn mức nền, nên không có dấu hiệu rò rỉ.

Hai nhãn `unknown` thuộc `age` và `submission_of_application` đều xuất hiện ở cùng 200 hồ sơ có `status = 1` (100%). Chúng được coi là dấu hiệu bất thường của dữ liệu, không phải một nhóm khách hàng có rủi ro thực tế, nên không dùng để giải thích trong EDA và không đưa vào mô hình.

2.5. Feature Engineering có chọn lọc

Code:

```
df_fe = df.copy()

# Tránh chia cho 0; giữ NaN nếu income thiếu hoặc income=0 (không tự
đoán thu nhập).
positive_income = df_fe['income'].gt(0)
df_fe['loan_income_ratio'] = (df_fe['loan_amount'] /
                              df_fe['income'].where(positive_income))

# Dùng log loan_amount; income=0 không có log xác định, giữ NaN nhất quán
với loan_income_ratio.
df_fe['log_loan_amount'] = np.log1p(df_fe['loan_amount'])
df_fe['log_income'] = np.log(df_fe['income'].where(df_fe['income'] > 0))

# Cờ kỳ hạn 360 tháng, có tính đến 41 giá trị term đã được điền mode ở
mục 2.2.
df_fe['term_360_flag'] = df_fe['term'].eq(360).astype('int8')

# Các khoảng điểm cố định để trình bày EDA, không dùng ngưỡng này làm
chuẩn đánh giá tín dụng.
score_bins = [499, 599, 699, 799, 900]
score_labels = ['500-599', '600-699', '700-799', '800-900']
```

```
df_fe['credit_score_band'] = pd.cut(df_fe['credit_score'],
bins=score_bins,
                                    labels=score_labels,
include_lowest=True)

# Tóm tắt dữ liệu tạo mới.
features = ['loan_income_ratio', 'log_loan_amount', 'log_income',
            'term_360_flag', 'credit_score_band']
summary_features = pd.DataFrame({
    'Biến tạo mới': features,
    'Số quan sát không thiếu': [int(df_fe[c].notna().sum()) for c in
features],
    'Số giá trị thiếu': [int(df_fe[c].isna().sum()) for c in features],
})
print('Số biến mới:', len(features), '| Kích thước sau tạo đặc trưng:',
df_fe.shape)
display(summary_features)
print('Đặc điểm tỷ lệ vay / thu nhập (chỉ tính income dương):')
display(df_fe['loan_income_ratio'].describe(percentiles=[.25, .5, .75,
.95, .99]).round(3))
print('Phân phối nhóm điểm tín dụng:')
display(df_fe['credit_score_band'].value_counts(sort=False).to_frame('Số
hồ sơ'))
print('Số kỳ hạn 360 tháng:', int(df_fe['term_360_flag'].sum()))
print('Độ lệch (skewness) trước/sau log:')
display(pd.DataFrame({'Biến': ['loan_amount → log_loan_amount', 'income →
log_income'],
                      'Skew gốc': [df['loan_amount'].skew(),
df['income'].skew()],
                      'Skew sau log': [df_fe['log_loan_amount'].skew(),
df_fe['log_income'].skew()]}).round(3))
```

Kết quả:

Số biến mới: 5 | Kích thước sau tạo đặc trưng: (148670, 33)

	Biến tạo mới	Số quan sát không thiếu	Số giá trị thiếu
0	loan_income_ratio	138260	10410
1	log_loan_amount	148670	0
2	log_income	138260	10410
3	term_360_flag	148670	0
4	credit_score_band	148670	0

Đặc điểm tỷ lệ vay / thu nhập (chỉ tính income dương):

	loan_income_ratio
count	138,260.000
mean	56.542
std	95.487
min	0.599
25%	37.216
50%	52.292
75%	69.845
95%	97.817
99%	127.632
max	25,275.000

dtype: float64

Phân phối nhóm điểm tín dụng:

credit_score_band	Số hồ sơ
500–599	37190
600–699	37200
700–799	36811
800–900	37469

Số kỳ hạn 360 tháng: 121726

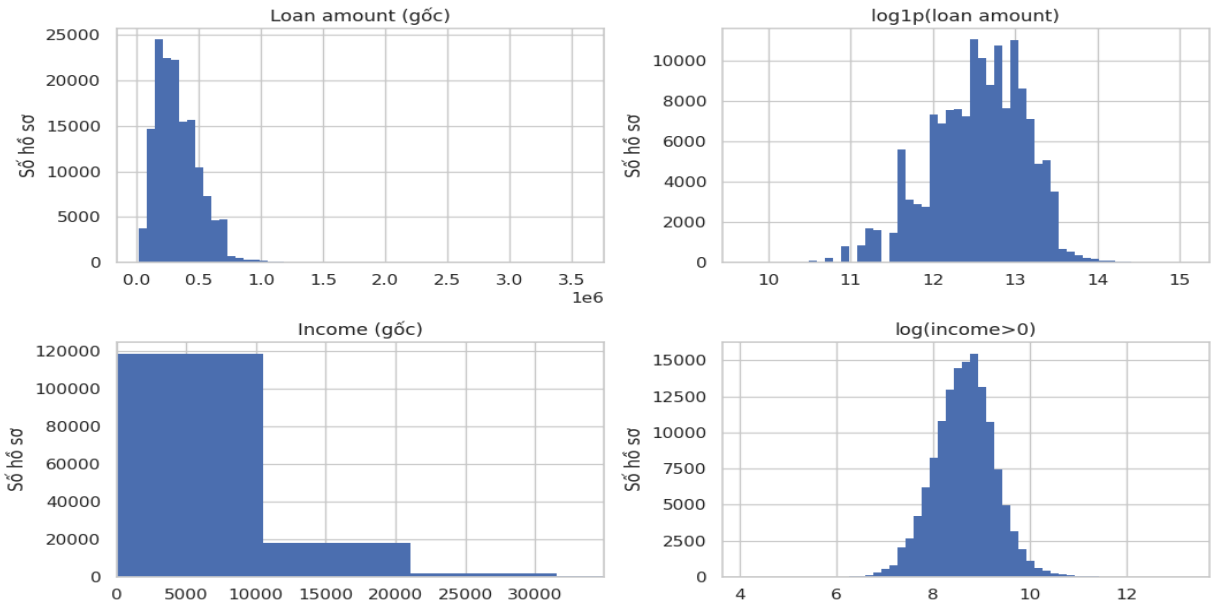
Độ lệch (skewness) trước/sau log:

	Biến	Skew gốc	Skew sau log
0	loan_amount → log_loan_amount	1.667	-0.397
1	income → log_income	17.308	0.006

Code:

```
# Biểu đồ minh họa sự thay đổi hình dạng phân phối sau phép biến đổi log.
fig, axs = plt.subplots(2, 2, figsize=(10.8, 7))
for ax, col, ttl in [(axs[0,0], 'loan_amount', 'Loan amount (gốc)'),
                     (axs[0,1], 'log_loan_amount', 'log1p(loan amount)'),
                     (axs[1,0], 'income', 'Income (gốc)'),
                     (axs[1,1], 'log_income', 'log(income>0)')]:
    vals = df_fe[col].dropna()
    ax.hist(vals, bins=55, edgecolor='none')
    if col == 'income':
        ax.set_xlim(0, vals.quantile(.995))  # Trục hiển thị phóng to, dữ
liệu không bị xóa.
    ax.set_title(ttl)
    ax.set_ylabel('Số hồ sơ')
plt.tight_layout()
```

Kết quả:




```
# Đối chiếu 28 biến cũ với bản sao ở đầu mục 2.4; không thay đổi giá trị
gốc.
pd.testing.assert_frame_equal(df_fe[df.columns], df_input,
check_exact=True)
assert df_fe.shape == (148670, 33)
assert not np.isinf(df_fe[['loan_income_ratio', 'log_loan_amount',
'log_income']].to_numpy()).any()
assert int(df_fe['loan_income_ratio'].isna().sum()) ==
int(df['income'].isna().sum() + df['income'].eq(0).sum())
assert df_fe['credit_score_band'].notna().all()
compare_final = pd.DataFrame({
    'Chỉ tiêu': ['Số hồ sơ', 'Số biến', 'Thiếu ở 28 biến cũ',
```

```
                'Thiếu phát sinh trong 5 biến mới', 'Tổng ô thiếu trên
toàn bảng'],
    'Đầu vào sau 2.3': [len(df_input), df_input.shape[1],
int(df_input.isna().sum().sum()),
                        0, int(df_input.isna().sum().sum())],
    'Đầu ra sau 2.5': [len(df_fe), df_fe.shape[1],
                       int(df_fe[df.columns].isna().sum().sum()),
                       int(df_fe[features].isna().sum().sum()),
                       int(df_fe.isna().sum().sum())]
})
display(compare_final)
print('Dữ liệu sau mục 2.5 nằm trong biến df_fe:', df_fe.shape)
```

Kết quả:

	Chỉ tiêu	Đầu vào sau 2.3	Đầu ra sau 2.5
0	Số hồ sơ	148670	148670
1	Số biến	28	33
2	Thiếu ở 28 biến cũ	176187	176187
3	Thiếu phát sinh trong 5 biến mới	0	20820
4	Tổng ô thiếu trên toàn bảng	176187	197007

Dữ liệu sau mục 2.5 nằm trong biến df_fe: (148670, 33)

Giải thích. Chỉ tạo năm biến mới, mỗi biến có lý do rõ:

- `loan_income_ratio` = `loan_amount` / `income` (chỉ tính khi `income` > 0), phản ánh gánh nặng vay so với thu nhập.

- `log_loan_amount` = log(1 + `loan_amount`) và `log_income` = log(`income`) khi `income` > 0, để giảm độ lệch phải.

- `term_360_flag`: cờ kỳ hạn 360 tháng, vì 81,9% khoản vay có kỳ hạn này.

- `credit_score_band`: bốn khoảng điểm (500–599, 600–699, 700–799, 800–900) để trình bày EDA, không phải chuẩn tín dụng.

Nhận xét. Độ lệch giảm rõ: `loan_amount` từ 1,667 xuống −0,397 sau log, `income` từ 17,308 xuống 0,006. Bốn nhóm điểm tín dụng có quy mô gần bằng nhau (37.190; 37.200; 36.811; 37.469 hồ sơ). `loan_income_ratio` có trung vị 52,29 và P99 là 127,63, nhưng giá trị lớn nhất đạt 25.275, một ngoại lai rất xa. 10.410 hồ sơ không tính được `loan_income_ratio` và `log_income`, gồm 9.150 hồ sơ thiếu `income` và 1.260 hồ

sơ `income` bằng 0. Nhờ vậy, số biến tăng từ 28 lên 33, tổng ô thiếu tăng lên 197.007 do 20.820 ô thiếu mới. 28 biến cũ không bị thay đổi.

Kết luận. Các biến mới giảm độ lệch và bổ sung thông tin tỷ lệ mà không sửa dữ liệu gốc. Ô thiếu mới được giữ để Pipeline xử lý ở 2.6.

2.6. Mã hóa và chuẩn hóa dữ liệu

2.6.1. Xác định nhu cầu mã hóa

```
import joblib
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler,
RobustScaler
df_final = df_fe.copy()
assert df_final.shape == (148670, 33)
assert df_final['status'].notna().all() and
set(df_final['status'].unique()) == {0, 1}
print('Đầu vào mục 2.6', df_final.shape)
print('Các biến phân loại:',
len(df_final.select_dtypes(include=['object', 'string',
'category']).columns))
print('Tổng ô thiếu:', int(df_final.isna().sum().sum()))
```

Kết quả:

Đầu vào mục 2.6 (148670, 33)

Các biến phân loại: 18

Tổng ô thiếu: 197007

```
# Chỉ loại biến ở nhánh phục vụ xây dựng mô hình; vẫn lưu chúng trong dữ
liệu EDA.
# Loại các biến có nguy cơ hé lộ nhãn/không xác định thời điểm ghi nhận.
leakage_excluded =
['interest_rate_spread','rate_of_interest','upfront_charges',
                   'property_value','ltv','dtir1',
                   'age','submission_of_application','credit_type']

# Giữ biến log thay tiền gốc và giữ credit_score thay nhóm điểm trùng
lặp.
redundancy_excluded = ['loan_amount','income','credit_score_band']
excluded = ['status'] + leakage_excluded + redundancy_excluded
feature_cols = [col for col in df_final.columns if col not in excluded]
```

```
# Phân loại biến chữ cần One-Hot; không mã hóa status và không gán thứ tự
tùy tiện cho nhãn chữ.
all_categorical = df_final.select_dtypes(include=['object', 'string',
'category']).columns.tolist()
cat_cols = [col for col in feature_cols if col in all_categorical]
cat_summary = pd.DataFrame({
   'Biến phân loại': all_categorical,
   'Số nhãn': [df_final[col].nunique(dropna=True) for col in
all_categorical],
   'Phương pháp': ['Không đưa vào mô hình (đã có credit_score)' if
col=='credit_score_band'
                   else ('Loại khỏi mô hình do unknown ↔ status' if col in
['age','submission_of_application','credit_type']
                   else 'One-Hot Encoding') for col in all_categorical]
})
assert len(all_categorical)==18 and len(cat_cols)==14
print('Tổng biến dạng chữ:',len(all_categorical), '| One-Hot trong mô
hình:',len(cat_cols))
print('Số biến đầu vào dự kiến:',len(feature_cols))
display(cat_summary)
```

Kết quả:

Tổng biến dạng chữ: 18 | One-Hot trong mô hình: 14

Số biến đầu vào dự kiến: 20

	Biến phân loại	Số nhãn	Phương pháp
0	loan_limit	3	One-Hot Encoding
1	gender	4	One-Hot Encoding
2	approv_in_adv	3	One-Hot Encoding
3	loan_type	3	One-Hot Encoding
4	loan_purpose	5	One-Hot Encoding
5	credit_worthiness	2	One-Hot Encoding
6	open_credit	2	One-Hot Encoding
7	neg_ammortization	3	One-Hot Encoding
8	interest_only	2	One-Hot Encoding
9	lump_sum_payment	2	One-Hot Encoding

10	occupancy_type	3	One-Hot Encoding
11	total_units	4	One-Hot Encoding
12	credit_type	4	Loại khỏi mô hình do unknown ↔ status
13	co_applicant_credit_type	2	One-Hot Encoding
14	age	8	Loại khỏi mô hình do unknown ↔ status
15	submission_of_application	3	Loại khỏi mô hình do unknown ↔ status
16	region	4	One-Hot Encoding
17	credit_score_band	4	Không đưa vào mô hình (đã có credit_score)

Giải thích. Dữ liệu có 18 biến dạng chữ. Với nhánh mô hình, các biến sau bị loại:

- Chín biến có nguy cơ lộ nhãn hoặc không rõ thời điểm ghi nhận: `interest_rate_spread`, `rate_of_interest`, `upfront_charges`, `property_value`, `ltv`, `dtir1`, `age`, `submission_of_application`, `credit_type`.
- Ba biến trùng lặp: `loan_amount` và `income` (đã có bản log), `credit_score_band` (đã có `credit_score`).

14 biến phân loại còn lại được mã hóa One-Hot, không gán thứ tự tùy ý cho nhãn chữ. Dữ liệu EDA vẫn giữ đầy đủ các biến.

Nhận xét. Còn 20 biến đầu vào: 14 phân loại, 4 biến chuẩn hóa chuẩn, 1 biến chuẩn hóa bền vững và 1 biến nhị phân. `credit_type` bị loại vì nhóm `EQUI` (15.298 hồ sơ) gần như toàn bộ là vỡ nợ (15.297 hồ sơ `status = 1`, chỉ 1 hồ sơ `status = 0`), cho thấy nguy cơ rò rỉ.

Kết luận. Nhánh mô hình dùng 20 biến đầu vào đã loại các biến nguy cơ rò rỉ và biến trùng lặp.

2.6.2. Xác định nhu cầu Scaling

```
# Phân nhóm thang đo theo đặc điểm phân phối
standard_cols = ['term','credit_score','log_loan_amount','log_income']
robust_cols = ['loan_income_ratio']
binary_cols = ['term_360_flag']
```

```
assert set(feature_cols) == set(cat_cols + standard_cols + robust_cols +
binary_cols)
# Bảng quyết định scaling và ý nghĩa.
scaling_policy = pd.DataFrame([
   {'Nhóm':'term, credit_score, log_loan_amount, log_income',
    'Phương pháp':'StandardScaler','Giải thích':'Đưa về trung bình 0, độ
lệch chuẩn 1'},
   {'Nhóm':'loan_income_ratio','Phương pháp':'RobustScaler',
    'Giải thích':'Dùng median và IQR để giảm phụ thuộc của thang đo vào
ngoại lai'},
   {'Nhóm':'term_360_flag','Phương pháp':'Giữ nguyên 0/1',
    'Giải thích':'Biến nhị phân không cần scaling'},
   {'Nhóm':f'{len(cat_cols)} biến phân loại','Phương pháp':'One-Hot
Encoding',
    'Giải thích':'Biến chữ, mỗi nhóm là cột giả 0/1'},
])
display(scaling_policy)
print('Ngoại lai: max loan_income_ratio
=',round(df_final['loan_income_ratio'].max(),2))
print(f'Ô thiếu trước khi đưa vào pipeline (trên {len(feature_cols)} ứng
viên):',
     int(df_final[feature_cols].isna().sum().sum()))
```

Kết quả:

	Nhóm	Phương pháp	Giải thích
0	term, credit_score, log_loan_amount, log_income	StandardScaler	Đưa về trung bình 0, độ lệch chuẩn 1
1	loan_income_ratio	RobustScaler	Dùng median và IQR để giảm phụ thuộc của thang...
2	term_360_flag	Giữ nguyên 0/1	Biến nhị phân không cần scaling
3	14 biến phân loại	One-Hot Encoding	Biến chữ, mỗi nhóm là cột giả 0/1

Ngoại lai: max loan_income_ratio = 25275.0

Ô thiếu trước khi đưa vào pipeline (trên 20 ứng viên): 20820

Giải thích. Phương pháp chuẩn hóa được chọn theo đặc điểm phân phối: `StandardScaler` cho `term`, `credit_score`, `log_loan_amount`, `log_income` (đưa về trung bình 0, độ lệch chuẩn 1); `RobustScaler` cho `loan_income_ratio` vì dùng trung vị và IQR nên ít bị ngoại lai (max 25.275) chi phối; `term_360_flag` giữ nguyên 0/1. Trước đó các biến được điền thiếu bên trong Pipeline: trung vị cho biến số, giá trị phổ biến nhất cho biến phân loại và biến nhị phân. Tất cả chỉ học tham số trên Train.

Nhận xét. Trước khi vào Pipeline còn 20.820 ô thiếu trên 20 biến ứng viên, nên bước điền thiếu bên trong Pipeline là cần thiết.

Kết luận. Mỗi nhóm biến được xử lý đúng với đặc điểm của nó, và các tham số chỉ học từ Train để tránh rò rỉ từ Test.

2.7. Hoàn tất và lưu dữ liệu sạch

Code:

```
# Tách biến đầu vào X và biến mục tiêu y
X = df_final[feature_cols].copy()
y = df_final['status'].astype('int8')
# Kiểm tra tính nhất quán với dữ liệu gốc
assert df_raw['loan_amount'].eq(df_final['loan_amount']).all()
assert df_raw['Status'].eq(y).all()
assert df_raw['term'].isna().sum() == 41
# Khôi phục giá trị term thiếu gốc để tránh Data Leakage
X['term'] = df_raw['term'].to_numpy()
X['term_360_flag'] =
X['term'].eq(360).astype(float).mask(X['term'].isna())
# Chuyển biến phân loại sang object để tương thích sklearn
X[cat_cols] = X[cat_cols].astype(object)
# Chia Train/Test theo tỷ lệ 80/20, phân tầng theo status
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=123, stratify=y
)
# Tổng hợp kết quả chia dữ liệu
split_table = pd.DataFrame(
    [
        [name, len(s), (s == 0).sum(), (s == 1).sum(), 100 * s.mean()]
        for name, s in [('Toàn bộ', y), ('Train', y_train), ('Test',
y_test)]
    ],
    columns=['Tập', 'Số dòng', 'Status=0', 'Status=1', 'Tỷ lệ status=1
(%)']
)
display(split_table.round(3))
```

Kết quả:

	Tập	Số dòng	Status=0	Status=1	Tỷ lệ status=1 (%)
0	Toàn bộ	148670	112031	36639	24.645
1	Train	118936	89625	29311	24.644
2	Test	29734	22406	7328	24.645

```
# Tương thích tên tham số dense output với các phiên bản scikit-learn.
try:
   encoder = OneHotEncoder(handle_unknown='ignore',sparse_output=False)
except TypeError:
   encoder = OneHotEncoder(handle_unknown='ignore',sparse=False)
# Mỗi nhóm được điền thiếu (khi cần) rồi mới mã hóa hoặc chuẩn hóa.
preprocessor = ColumnTransformer(transformers=[

('categorical',Pipeline(steps=[('imputer',SimpleImputer(strategy='most_fre
quent')),
                                    ('onehot',encoder)]),cat_cols),
 ('standard',Pipeline(steps=[('imputer',SimpleImputer(strategy='median')),

('scaler',StandardScaler())]),standard_cols),
   ('robust',Pipeline(steps=[('imputer',SimpleImputer(strategy='median')),
                             ('scaler',RobustScaler())]),robust_cols),
   ('binary', Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent'))]), binary_cols),
],remainder='drop',sparse_threshold=0,verbose_feature_names_out=False)
# RẤT QUAN TRỌNG: fit_transform trên Train, chỉ transform trên Test.
train_values = preprocessor.fit_transform(X_train)
test_values = preprocessor.transform(X_test)
feature_names = list(preprocessor.get_feature_names_out())
train_ready =
pd.DataFrame(train_values,columns=feature_names,index=X_train.index)
test_ready =
pd.DataFrame(test_values,columns=feature_names,index=X_test.index)
# Kiểm tra ma trận đầu ra có đúng các cột và không chứa số vô hạn / NaN.
assert len(feature_names)==len(set(feature_names))
assert train_ready.columns.equals(test_ready.columns)
assert np.isfinite(train_ready.to_numpy()).all()
assert np.isfinite(test_ready.to_numpy()).all()
assert not set(leakage_excluded).intersection(feature_names)
assert 'status' not in feature_names
print('Train sau mã hóa/scaling:',train_ready.shape)
print('Test sau mã hóa/scaling:',test_ready.shape)
```

```
print('Số biến
giả:',len(preprocessor.named_transformers_['categorical'].named_steps['one
hot'].get_feature_names_out(cat_cols)))
print('NaN ở Train /
Test:',int(train_ready.isna().sum().sum()),'/',int(test_ready.isna().sum()
.sum()))
```

Kết quả:

Train sau mã hóa/scaling: (118936, 48)

Test sau mã hóa/scaling: (29734, 48)

Số biến giả: 42

NaN ở Train / Test: 0 / 0

Nhận xét. Train có 118.936 dòng và Test có 29.734 dòng, cùng 48 đặc trưng. Trong đó có 42 biến giả (từ 14 biến phân loại) và 6 biến còn lại: 4 biến `StandardScaler`, 1 biến `RobustScaler` và 1 biến nhị phân `term_360_flag`. Số NaN ở cả hai tập là 0, nghĩa là các bước điền thiếu trong Pipeline đã xử lý hết 20.820 ô thiếu của các biến ứng viên. Các `assert` bảo đảm hai tập có cùng cột, không có số vô hạn, không có biến bị loại ở 2.4.3 lọt vào và `status` không nằm trong đặc trưng. Tùy chọn `handle_unknown='ignore'` giúp Pipeline không lỗi nếu Test có nhãn không xuất hiện ở Train.

Kết luận. Hai ma trận đầu vào có cấu trúc giống hệt nhau, đầy đủ, không còn ô thiếu và không chứa biến nguy cơ rò rỉ.

```
# Kiểm tra đúng tính chất StandardScaler/RobustScaler trên Train, không
kiểm trên Test.
check_scaling = pd.DataFrame([
   {'Biến':col,'Mean Train':train_ready[col].mean(),
    'Std Train (ddof=0)':train_ready[col].std(ddof=0),
    'Median Train':train_ready[col].median(),
    'IQR
Train':train_ready[col].quantile(.75)-train_ready[col].quantile(.25)}
   for col in standard_cols+robust_cols
])
display(check_scaling.round(5))
# StandardScaler: mean ≈ 0, std ≈ 1 (khi dữ liệu không hằng số).
for col in standard_cols:
   assert abs(train_ready[col].mean())<1e-10
   assert abs(train_ready[col].std(ddof=0)-1)<1e-10
# RobustScaler: median ≈ 0, IQR ≈ 1 trên Train.
assert abs(train_ready['loan_income_ratio'].median())<1e-10
```

```
assert
abs((train_ready['loan_income_ratio'].quantile(.75)-train_ready['loan_inco
me_ratio'].quantile(.25))-1)<0.002
```

Kết quả:

	Biến	Mean Train	Std Train (ddof=0)	Median Train	IQR Train
0	term	-0.000	1.000	0.427	0.000
1	credit_score	0.000	1.000	-0.008	1.735
2	log_loan_amount	0.000	1.000	0.075	1.391
3	log_income	0.000	1.000	0.016	1.236
4	loan_income_ratio	0.123	1.977	0.000	1.000

Nhận xét.

- Bốn biến dùng `StandardScaler` có trung bình 0 và độ lệch chuẩn 1, đúng tính chất mong đợi. Trung vị của ba biến `credit_score`, `log_loan_amount` và `log_income` gần 0, cho thấy phân phối khá cân đối.
- `term` có IQR bằng 0 và trung vị 0,427. Điều này không phải lỗi: 81,9% khoản vay có kỳ hạn 360 tháng (Q1 = Q3 = 360), nên sau chuẩn hóa phần lớn hồ sơ cùng nhận một giá trị là 0,427.
- `loan_income_ratio` dùng `RobustScaler` nên trung vị đúng bằng 0 và IQR bằng 1. Tuy vậy trung bình (0,123) và độ lệch chuẩn (1,977) vẫn lệch khỏi chuẩn, vì đuôi phải dài (giá trị lớn nhất 25.275) vẫn còn. `RobustScaler` không loại bỏ ngoại lai mà chỉ giúp chúng không chi phối thang đo.

Kết luận. Các scaler hoạt động đúng thiết kế và được học chỉ trên Train. Việc dùng `RobustScaler` cho `loan_income_ratio` là hợp lý.

```
# Dữ liệu EDA giữ đủ các biến và missing có chủ đích, không ảnh hưởng
bảng Train/Test.
# Chỉ ghép nhãn status khi cần xuất dữ liệu huấn luyện.
train_export = train_ready.copy()
test_export = test_ready.copy()
train_export['status'] = y_train
test_export['status'] = y_test
# Kiểm tra đầu ra: số dòng, số đặc trưng, số NaN, khớp nhãn.
```

```
assert len(train_export) + len(test_export) == len(df_final)
assert train_export.shape[0] == 118936 and test_export.shape[0] == 29734
assert train_export.shape[1] == test_export.shape[1]
assert train_export['status'].sum() + test_export['status'].sum() ==
df_final['status'].sum()
assert df_final.shape == (148670, 33)
assert int(df_final.isna().sum().sum()) == 197007
assert not train_export.isna().any().any()
assert not test_export.isna().any().any()
quality = pd.DataFrame([
   {'Bộ dữ liệu':'EDA (df_final)', 'Số dòng':len(df_final), 'Số
cột':df_final.shape[1],
    'Số ô thiếu':int(df_final.isna().sum().sum())},
   {'Bộ dữ liệu':'Train (train_export)', 'Số dòng':len(train_export), 'Số
cột':train_export.shape[1],
    'Số ô thiếu':int(train_export.isna().sum().sum())},
   {'Bộ dữ liệu':'Test (test_export)', 'Số dòng':len(test_export), 'Số
cột':test_export.shape[1],
    'Số ô thiếu':int(test_export.isna().sum().sum())},
])
display(quality)
```

Kết quả:

	Bộ dữ liệu	Số dòng	Số cột	Số ô thiếu
0	EDA (df_final)	148670	33	197007
1	Train (train_export)	118936	49	0
2	Test (test_export)	29734	49	0

Nhận xét. Hai nhánh dữ liệu có mục đích khác nhau. Nhánh EDA giữ đủ 33 biến và các ô thiếu có chủ đích (197.007 ô, gồm 176.187 ô ở các biến gốc và 20.820 ô phát sinh ở `loan_income_ratio`, `log_income`), phục vụ phân tích mô tả ở Chương 3. Nhánh mô hình (Train và Test) có 49 cột, gồm 48 đặc trưng và `status`, không còn ô thiếu. Số dòng của Train và Test cộng lại bằng đúng số hồ sơ ban đầu, và tổng số hồ sơ vỡ nợ được bảo toàn (đã được `assert` kiểm tra).

Kết luận. Chương 2 bàn giao hai bộ dữ liệu rõ ràng: bộ EDA cho phân tích khám phá, và bộ Train/Test sạch, đã mã hóa, chuẩn hóa cho xây dựng mô hình.

Lưu ý: các cell 55 đến 58 chưa có lệnh ghi file ra đĩa (`to_csv`, `joblib.dump`). Nếu đề bài yêu cầu "lưu dữ liệu sạch", bạn nên thêm bước này, kèm lưu `preprocessor` để dùng lại ở giai đoạn mô hình.

CHƯƠNG 3: PHÂN TÍCH KHÁM PHÁ DỮ LIỆU VÀ TRỰC QUAN HÓA

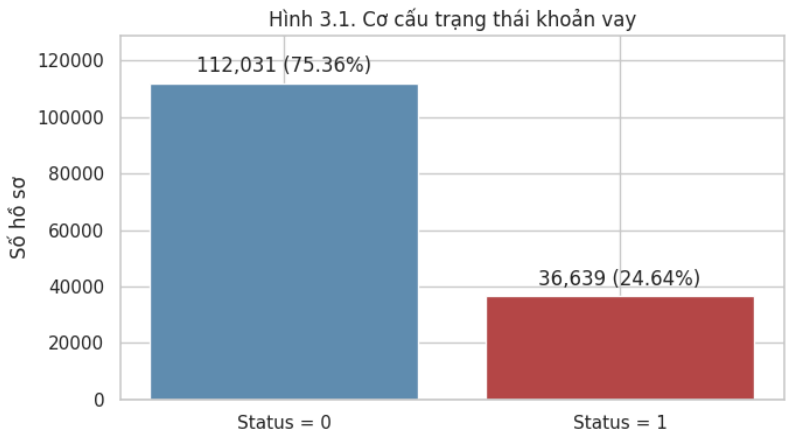
```
# Sử dụng dữ liệu EDA từ Chương 2
eda = df_final
required = {"status", "loan_amount", "term", "property_value", "income",
            "credit_score", "age", "loan_type", "loan_purpose", "gender",
"ltv", "dtir1"}
assert required.issubset(eda.columns) and eda.shape == (148670, 33)
assert set(eda["status"].unique()) == {0, 1}
```

Chương 3 sử dụng df_final với 148.670 hồ sơ, giữ nguyên dữ liệu thiếu và các giá trị ngoại lai hợp lệ. Các thống kê được tính theo số quan sát có dữ liệu của từng biến; giới hạn P99 chỉ phục vụ hiển thị biểu đồ, không làm thay đổi dữ liệu nghiên cứu.

3.1. Thống kê mô tả tổng quan

3.1.1. Phân phối biến mục tiêu

```
# Sử dụng toàn bộ hồ sơ vì biến status không thiếu.
status_n = eda["status"].value_counts().reindex([0, 1], fill_value=0)
status_pct = status_n.div(len(eda)).mul(100)
display(pd.DataFrame({"Số hồ sơ": status_n, "Tỷ lệ (%)":
status_pct.round(2)}))
# Hình 3.1: tần suất và tỷ lệ status.
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(["Status = 0", "Status = 1"], status_n, color=["#618eaf",
"#b44747"])
ax.bar_label(bars, [f"{n:,} ({p:.2f}%)" for n, p in zip(status_n,
status_pct)], padding=4)
ax.set_ylim(0, status_n.max() * 1.15)
ax.set_ylabel("Số hồ sơ")
ax.set_title("Hình 3.1. Cơ cấu trạng thái khoản vay")
```



Nhận xét: Có 36.639 hồ sơ `status = 1` (24,64%) và 112.031 hồ sơ `status = 0` (75,36%), tỷ lệ xấp xỉ 1 : 3

Kết luận: Biến mục tiêu mất cân bằng ở mức vừa phải, nên khi so sánh phải dùng tỷ lệ thay vì số hồ sơ. Mốc 24,64% là đường tham chiếu cho các phần sau, nhưng mốc này thay đổi khi mẫu thay đổi

3.1.2. Đặc điểm quy mô khoản vay

```
# Bảng thống kê trên dữ liệu quan sát được của từng biến.
display(bang_mo_ta(["loan_amount", "property_value", "term"]))
print(f"Hồ sơ kỳ hạn 360 tháng: {eda['term'].eq(360).sum():,} "
      f"({eda['term'].eq(360).mean():.2%})")
```

	N hợp lệ	Thiếu (%)	Trung bình	Trung vị	Q1	Q3	P99	Tối đa
loan_amount	148670	0.000	331,117.740	296,500.000	196,500.000	436,500.000	856,500.000	3,576,500.000
property_value	133572	10.160	497,893.470	418,000.000	268,000.000	628,000.000	1,808,000.000	16,508,000.000
term	148670	0.000	335.140	360.000	360.000	360.000	360.000	360.000

Hồ sơ kỳ hạn 360 tháng: 121,726 (81.88%)

Nhận xét: Khoản vay có trung vị 296.500, thấp hơn trung bình 331.117,74; 81,88% khoản vay có kỳ hạn 360 tháng. Giá trị tài sản thế chấp thiếu 10,16% hồ sơ.
Kết luận: Quy mô vay có xu hướng lệch phải, còn thống kê tài sản chỉ đại diện cho mẫu có dữ liệu.

3.1.3. Đặc điểm tài chính người vay

```
# Bảng N hợp lệ và tỷ lệ thiếu bắt buộc đi cùng các thống kê.
display(bang_mo_ta(["income", "credit_score", "ltv", "dtir1"]))
print("Số hồ sơ income = 0:", f"{eda['income'].eq(0).sum():,}")
print("Số hồ sơ LTV > 1.000%:", f"{eda['ltv'].gt(1000).sum():,}")
```

	N hợp lệ	Thiếu (%)	Trung bình	Trung vị	Q1	Q3	P99	Tối đa
income	139520	6.150	6,957.340	5,760.000	3,720.000	8,520.000	26,640.000	578,580.000
credit_score	148670	0.000	699.790	699.000	599.000	800.000	897.000	900.000
ltv	133572	10.160	72.750	75.140	60.470	86.180	102.760	7,831.250
dtir1	124549	16.220	37.730	39.000	31.000	45.000	60.000	61.000

Số hồ sơ income = 0: 1,260
Số hồ sơ LTV > 1.000%: 6

Nhận xét:

- Biến Income có trung vị là 5.760 thấp hơn trung bình là 6.957,34; có 1.260 hồ sơ ghi thu nhập bằng 0.
- LTV và DTI thiếu lần lượt 10,16% và 16,22%; có 6 LTV vượt 1.000%.

Kết luận: Cần chú ý giá trị cực trị và mẫu hợp lệ khi diễn giải các chỉ số tài chính

3.1.4. Đặc điểm phân loại

```
# Bảng cơ cấu ba biến phân loại theo tổng số 148.670 hồ sơ.
cat_rows = []
for label, col in [("Loại vay", "loan_type"),
                   ("Mục đích vay", "loan_purpose"),
                   ("Giới tính/đồng vay", "gender")]:
    for group, n in eda[col].value_counts(dropna=False).items():
        cat_rows.append([label, str(group), n, 100 * n / len(eda)])
cat_table = pd.DataFrame(cat_rows, columns=["Biến", "Nhóm", "Số hồ sơ",
"Tỷ lệ (%)"])
display(cat_table.round(2))
```

	Biến	Nhóm	Số hồ sơ	Tỷ lệ (%)

0	Loại vay	type1	113173	76.120
1	Loại vay	type2	20762	13.970
2	Loại vay	type3	14735	9.910
3	Mục đích vay	p3	55934	37.620
4	Mục đích vay	p4	54799	36.860
5	Mục đích vay	p1	34529	23.230
6	Mục đích vay	p2	3274	2.200
7	Mục đích vay	unknown	134	0.090
8	Giới tính/đồng vay	male	42346	28.480
9	Giới tính/đồng vay	joint	41399	27.850
10	Giới tính/đồng vay	sex not available	37659	25.330
11	Giới tính/đồng vay	female	27266	18.340

Nhận xét: type1 chiếm 76,12% số khoản vay; hai nhóm mục đích p3 và p4 chiếm tổng cộng 74,48%. Mục đích vay có 134 hồ sơ unknown. Về giới tính/đồng vay, sex not available chiếm 25,33%, tức khoảng một phần tư hồ sơ không có thông tin giới tính.

Kết luận: Cơ cấu khoản vay không đồng đều và biến giới tính có tỷ lệ không xác định đáng kể, nên cần xét tỷ lệ trên từng nhóm khi phân tích tình trạng vỡ nợ và không diễn giải nhóm sex not available như một nhóm khách hàng thực.

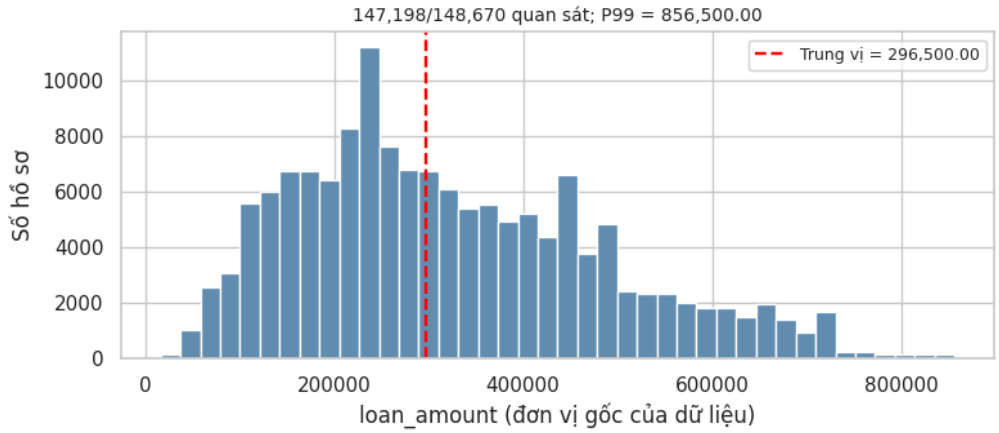
3.2. Phân tích phân phối từng biến

3.2.1. Phân phối loan_amount

```
# Hình 3.2: Chỉ phóng vùng đến P99, không loại hồ sơ khỏi dữ liệu.
fig, ax = plt.subplots(figsize=(8, 4))
ve_hist(ax, "loan_amount", bins=40, p99=True)
ax.set_xlabel("loan_amount (đơn vị gốc của dữ liệu)")
fig.suptitle("Hình 3.2. Phân phối quy mô khoản vay", fontsize=12)
plt.tight_layout()
print(f"Độ lệch (skewness) toàn bộ: {eda['loan_amount'].skew():.2f}")
```

Độ lệch (skewness) toàn bộ: 1.67

Hình 3.2. Phân phối quy mô khoản vay



Nhận xét: Phân phối loan_amount lệch phải (skewness = 1,67), với trung vị 296.500 thấp hơn trung bình 331.117,74.

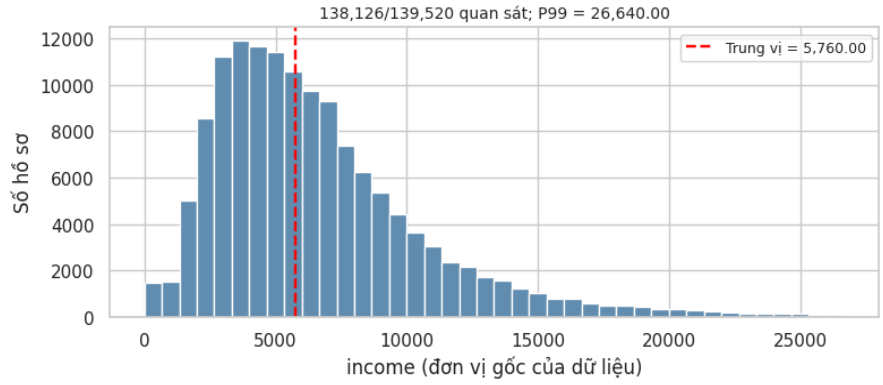
Kết luận: Một số khoản vay lớn kéo đuôi phân phối; biểu đồ chỉ phóng vùng đến P99, không loại bỏ các hồ sơ lớn.

3.2.2. Phân phối income

```
# Hình 3.3: Bỏ NaN riêng khi vẽ; không đổi các khoản income = 0.
fig, ax = plt.subplots(figsize=(8, 4))
ve_hist(ax, "income", bins=40, p99=True)
ax.set_xlabel("income (đơn vị gốc của dữ liệu)")
fig.suptitle("Hình 3.3. Phân phối thu nhập", fontsize=12)
plt.tight_layout()
print(f"Độ lệch (skewness) trên N hợp lệ: {eda['income'].skew():.2f}")
```

Độ lệch (skewness) trên N hợp lệ: 17.31

Hình 3.3. Phân phối thu nhập



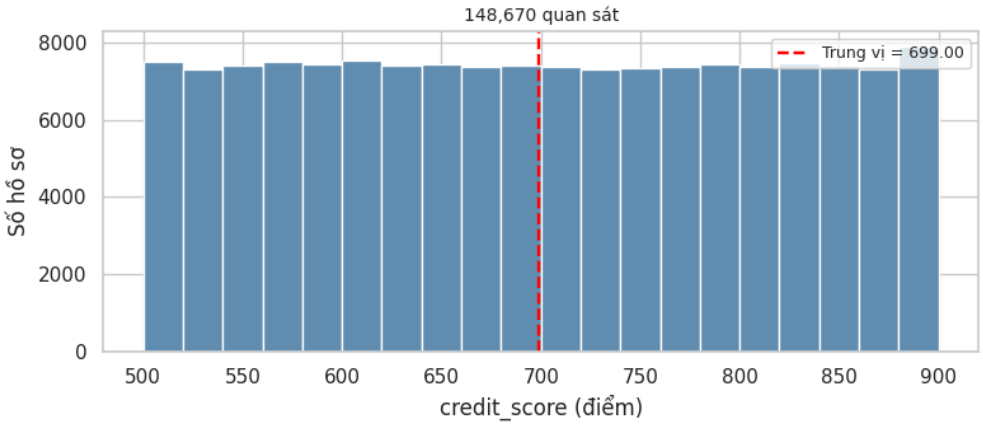
Nhận xét: income lệch phải rất mạnh (skewness = 17,31); trung vị 5.760 trong khi giá trị lớn nhất đạt 578.580, gấp khoảng 22 lần P99 (26.640).

Kết luận: Trung vị phù hợp hơn trung bình để phản ánh thu nhập điển hình. Các giá trị bằng 0 (1.260 hồ sơ) vẫn được giữ để phân tích riêng, vì ở 3.4.2 nhóm này có tỷ lệ status = 1 bất thường.

3.2.3. Phân phối Credit_Score

```
# Hình 3.4: Hiển thị toàn bộ dải 500-900, không cắt ngưỡng P99.
fig, ax = plt.subplots(figsize=(8, 4))
ve_hist(ax, "credit_score", bins=20)
ax.set_xlabel("credit_score (điểm)")
fig.suptitle("Hình 3.4. Phân phối điểm tín dụng", fontsize=12)
plt.tight_layout()
plt.show()
print("Tần suất theo nhóm điểm đã tạo ở Chương 2:")
display(eda['credit_score_band'].value_counts(sort=False).to_frame("Số hồ sơ"))
```

Hình 3.4. Phân phối điểm tín dụng



Tần suất theo nhóm điểm đã tạo ở Chương 2:

	Số hồ sơ
credit_score_band	
500–599	37190
600–699	37200

700–799	36811
800–900	37469

.

Nhận xét: Bốn khoảng điểm tín dụng có quy mô gần bằng nhau (36.811 - 37.469 hồ sơ/nhóm, mỗi nhóm khoảng 25%), với trung vị chung 699.

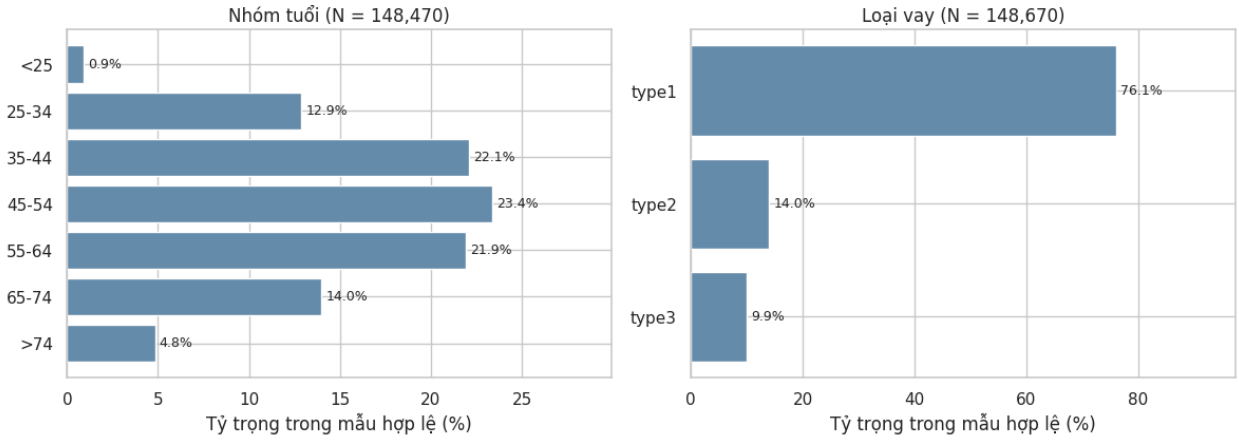
Kết luận: Phân phối gần như phẳng trong khoảng 500–900, khác với điểm tín dụng thực tế thường có dạng hình chuông. Đây là dấu hiệu dữ liệu có thể là mô phỏng hoặc đã ẩn danh. Phân phối đơn biến này chưa cho biết điểm tín dụng có phân biệt được status hay không; vấn đề này được kiểm tra ở 3.4.3 và 3.5 (H3)

3.2.4. Phân phối theo nhóm tuổi và loại khoản vay

```
# Hình 3.5: age=unknown không phải nhóm tuổi nên loại khỏi mẫu số biểu đồ
tuổi.
age_order = ["<25", "25-34", "35-44", "45-54", "55-64", "65-74", ">74"]
age_n = eda.loc[eda['age'].isin(age_order),
'age'].value_counts().reindex(age_order, fill_value=0)
loan_n = eda['loan_type'].value_counts()

fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
for ax, counts, title in [
    (axes[0], age_n, f"Nhóm tuổi (N = {age_n.sum():,})"),
    (axes[1], loan_n, f"Loại vay (N = {loan_n.sum():,})")]:
    percentage = counts.div(counts.sum()).mul(100)
    bars = ax.barh(counts.index, percentage, color='#668dad')
    ax.bar_label(bars, fmt="%.1f%%", padding=3, fontsize=9)
    ax.set_xlim(0, percentage.max() * 1.28)
    ax.invert_yaxis()
    ax.set_xlabel("Tỷ trọng trong mẫu hợp lệ (%)")
    ax.set_title(title)
fig.suptitle("Hình 3.5. Cơ cấu theo tuổi và loại khoản vay", fontsize=12)
plt.tight_layout()
plt.show()
print("Hồ sơ age=unknown (không đưa vào biểu đồ tuổi):",
eda['age'].eq('unknown').sum())
```

Hình 3.5. Cơ cấu theo tuổi và loại khoản vay



Hồ sơ age=unknown (không đưa vào biểu đồ tuổi): 200

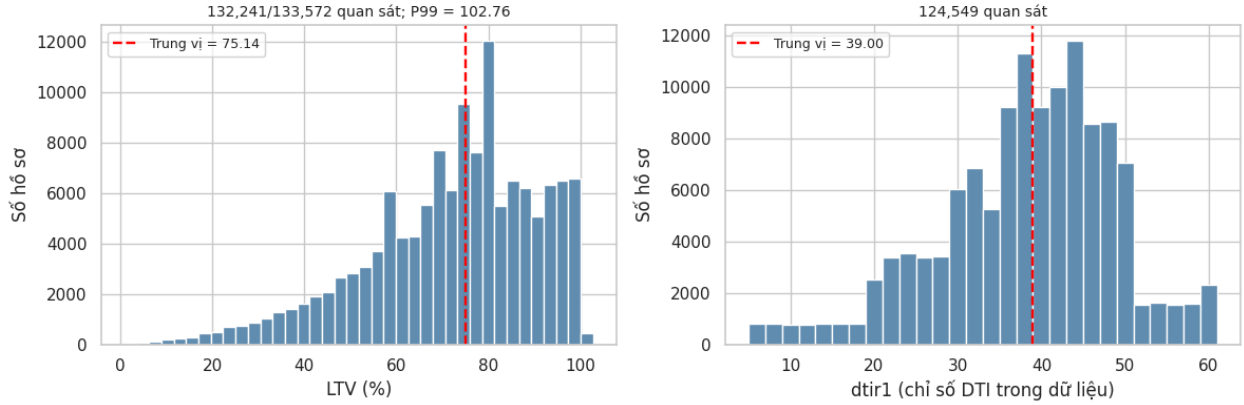
Nhận xét: Nhóm tuổi 45–54 có nhiều hồ sơ nhất (34.720); ba nhóm 35–64 tuổi chiếm 67,4% số hồ sơ có tuổi, trong khi nhóm dưới 25 tuổi chỉ chiếm 0,9%. type1 chiếm 76,12% khoản vay. Có 200 hồ sơ age = unknown được loại riêng khỏi biểu đồ tuổi.

Kết luận: Danh mục tập trung vào người vay trung niên; cần phân biệt sự phổ biến của một nhóm với tỷ lệ vỡ nợ của nhóm đó.

3.2.5. Phân phối các chỉ số LTV và DTI

```
# Hình 3.6: LTV chỉ phóng đến P99 do có 6 giá trị > 1.000%; DTI hiện toàn bộ.
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
ve_hist(axes[0], 'ltv', bins=38, p99=True)
ve_hist(axes[1], 'dtir1', bins=28)
axes[0].set_xlabel("LTV (%)")
axes[1].set_xlabel("dtir1 (chỉ số DTI trong dữ liệu)")
fig.suptitle("Hình 3.6. Phân phối LTV và DTI", fontsize=12)
plt.tight_layout()
plt.show()
print(f"LTV > 1.000% (giữ nguyên): {eda['ltv'].gt(1000).sum():,} hồ sơ")
print("Tỷ lệ thiếu LTV / DTI:",
      f"{eda['ltv'].isna().mean():.2%} / {eda['dtir1'].isna().mean():.2%}")
```

Hình 3.6. Phân phối LTV và DTI



LTV > 1.000% (giữ nguyên): 6 hồ sơ
Tỷ lệ thiếu LTV / DTI: 10.16% / 16.22%

Nhận xét: LTV có trung vị 75,14%, DTI có trung vị 39; tỷ lệ thiếu tương ứng 10,16% và 16,22%. Có 1.799 hồ sơ (1,35% mẫu có LTV) với LTV trên 100%, tức khoản vay lớn hơn giá trị tài sản. Sáu hồ sơ LTV trên 1.000% đều có giá trị tài sản bằng 8.000 và thiếu thu nhập, nên nghi là lỗi nhập liệu. DTI có giá trị lớn nhất chỉ 61 nên bị chặn ở phía trên.

Kết luận: Biểu đồ LTV phóng vùng đến P99 để dễ quan sát nhưng vẫn giữ các giá trị cực trị trong dữ liệu; nhóm LTV trên 100% là nhóm cần theo dõi riêng.

3.3. Phân tích chất lượng dữ liệu theo trạng thái vỡ nợ

3.3.1. So sánh Missing Values giữa hai nhóm Status

```
# 7 biến số còn NaN; 2 biến phân loại thiếu gốc đã được gán nhãn
'unknown' ở Chương 2.
missing_vars = ['interest_rate_spread', 'rate_of_interest',
'upfront_charges',
                'property_value', 'ltv', 'dtir1', 'income']
assert all(eda[c].isna().any() for c in missing_vars

missing_flags = eda[missing_vars].isna().copy()
for col in ['age', 'submission_of_application']:
    # Phân biệt thiếu gốc đã chuyển thành 'unknown' với NaN còn tồn tại.
    assert eda[col].eq('unknown').sum() == df_raw[col].isna().sum()
    missing_flags[f'{col} (unknown)'] = eda[col].eq('unknown')

miss0 = missing_flags.loc[eda.status.eq(0)].mean() * 100
```

```
miss1 = missing_flags.loc[eda.status.eq(1)].mean() * 100
bang_33 = pd.DataFrame({'Thiếu | status=0 (%)': miss0,
                        'Thiếu | status=1 (%)': miss1,
                        'Chênh lệch (đpt; 1-0)': miss1 - miss0})
print('Bảng 3.3. Missing theo status (7 biến còn NaN, 2 biến thiếu gốc đã gán unknown):')
display(bang_33.round(2))
#Lưu ý: age và submission_of_application không còn là NaN sau làm sạch;'
hai dòng cuối phản ánh thiếu dữ liệu gốc, không phải nhóm tuổi/kênh nộp thực.'
```

Bảng 3.3. Missing theo status (7 biến còn NaN, 2 biến thiếu gốc đã gán unknown):

	Thiếu | status=0 (%)	Thiếu | status=1 (%)	Chênh lệch (đpt; 1−0)
interest_rate_spread	0.000	100.000	100.000
rate_of_interest	0.000	99.450	99.450
upfront_charges	2.820	99.580	96.770
property_value	0.000	41.200	41.200
ltv	0.000	41.200	41.200
dtir1	6.970	44.520	37.540
income	7.060	3.380	-3.680
age (unknown)	0.000	0.550	0.550
submission_of_application (unknown)	0.000	0.550	0.550

Nhận xét: interest_rate_spread thiếu ở 100% hồ sơ status = 1 nhưng không thiếu ở nhóm status = 0; LTV thiếu 41,20% ở nhóm 1 so với 0% ở nhóm 0. Thu nhập lại thiếu nhiều hơn ở nhóm 0 (7,06% so với 3,38%).

Kết luận: Dữ liệu thiếu có liên hệ rõ với status, không phải thiếu ngẫu nhiên. Ba biến lãi suất, chênh lệch lãi suất và phí ban đầu gần như xác định nhãn, nên là dấu hiệu rò rỉ thông tin; chỉ phân tích hồ sơ đầy đủ sẽ gây thiên lệch.

3.3.2. Phân tích sự khác biệt về mẫu quan sát

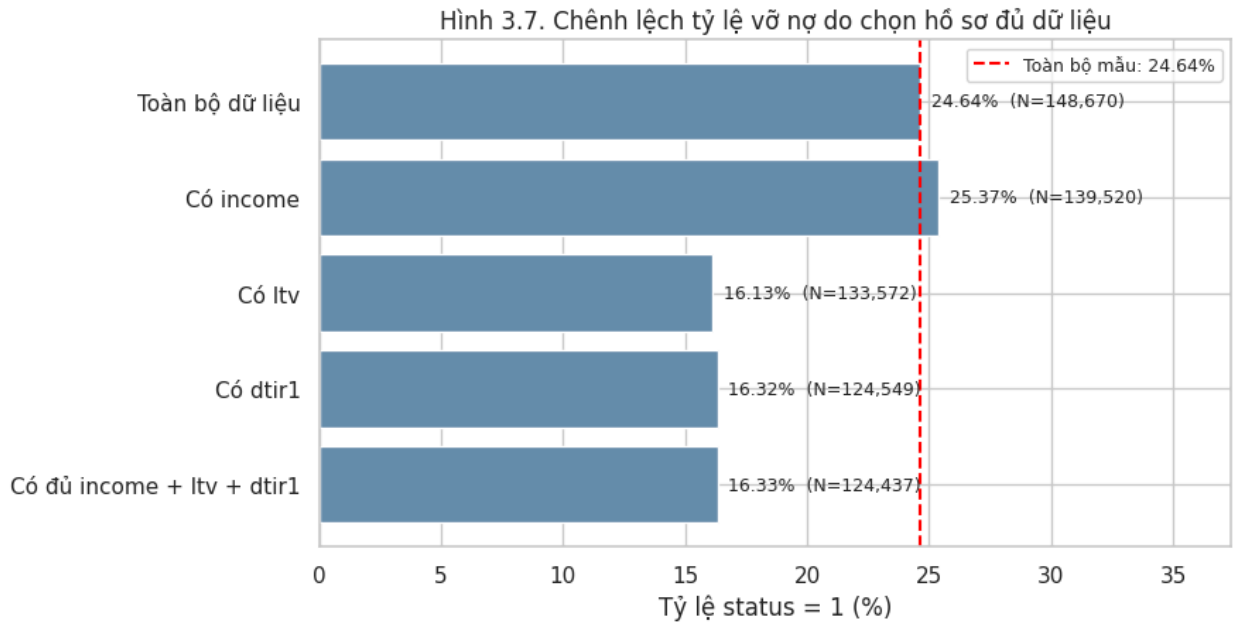
```
# Không xóa dữ liệu: mỗi mask chỉ dùng tính chỉ số trên một tập con.
mau_day_du = {
   'Toàn bộ dữ liệu': pd.Series(True, index=eda.index),
   'Có income': eda['income'].notna(),
   'Có ltv': eda['ltv'].notna(),
   'Có dtir1': eda['dtir1'].notna(),
   'Có đủ income + ltv + dtir1': eda[['income', 'ltv',
'dtir1']].notna().all(axis=1)
}
nen_chung = eda.status.mean() * 100
bang_mau = pd.DataFrame([
   {'Mẫu': name, 'N giữ lại': int(mask.sum()), 'Tỷ lệ giữ lại (%)': 100 *
mask.mean(),
    'Số status=1': int(eda.loc[mask, 'status'].sum()),
    'Status=1 trong mẫu (%)': 100 * eda.loc[mask, 'status'].mean(),
    'Lệch so với toàn bộ (đpt)': 100 * eda.loc[mask, 'status'].mean() -
nen_chung}
   for name, mask in mau_day_du.items()
]).set_index('Mẫu')
display(bang_mau.round(2))
```

Mẫu	N giữ lại	Tỷ lệ giữ lại (%)	Số status=1	Status=1 trong mẫu (%)	Lệch so với toàn bộ (đpt)
Toàn bộ dữ liệu	148670	100.000	36639	24.640	0.000
Có income	139520	93.850	35400	25.370	0.730
Có ltv	133572	89.840	21543	16.130	-8.520
Có dtir1	124549	83.780	20329	16.320	-8.320
Có đủ income + ltv + dtir1	124437	83.700	20319	16.330	-8.320

Nhận xét: Tỷ lệ status = 1 là 24,64% trên toàn bộ mẫu, nhưng còn 16,13% trong mẫu có LTV và 16,33% trong mẫu đủ thu nhập, LTV và DTI (giảm khoảng 8,3–8,5 điểm phần trăm).

Kết luận: Chọn mẫu chỉ gồm hồ sơ đủ dữ liệu làm thay đổi đáng kể cơ cấu kết quả, các tỷ lệ không nên so sánh khi bỏ qua mẫu số.

```
# Hình 3.7: chỉ so tỷ lệ trên các mẫu quan sát khác nhau, kèm N giữ lại.
fig, ax = plt.subplots(figsize=(9, 4.7))
vals = bang_mau['Status=1 trong mẫu (%)']
bars = ax.barh(vals.index, vals, color='#668dad')
ax.axvline(nen_chung, color='red', ls='--', lw=1.5,
           label=f'Toàn bộ mẫu: {nen_chung:.2f}%')
for bar, pct, n in zip(bars, vals, bang_mau['N giữ lại']):
    ax.text(pct + .45, bar.get_y() + bar.get_height()/2,
            f'{pct:.2f}%  (N={n:,})', va='center', fontsize=9)
ax.set_xlim(0, vals.max() + 12)
ax.invert_yaxis()
ax.set_xlabel('Tỷ lệ status = 1 (%)')
ax.set_title('Hình 3.7. Chênh lệch tỷ lệ vỡ nợ do chọn hồ sơ đủ dữ liệu')
ax.legend(fontsize=9)
plt.tight_layout()
plt.show()
print('Tuổi không xác định (age=unknown):',
int(eda.age.eq('unknown').sum()),
      '| status=1:', int(eda.loc[eda.age.eq('unknown'), 'status'].sum()))
# Đối chiếu nguồn gốc thiếu LTV/property_value theo credit_type (không
sửa dữ liệu).
nguon_thieu = pd.crosstab(eda['credit_type'],
eda['ltv'].isna()).reindex(columns=[False, True], fill_value=0)
nguon_thieu.columns = ['LTV có dữ liệu', 'LTV thiếu']
nguon_thieu['Tỷ lệ thiếu LTV (%)'] = (100 * nguon_thieu['LTV thiếu'] /
nguon_thieu.sum(axis=1))
print('Bảng 3.4a. Tình trạng thiếu LTV theo credit_type:')
display(nguon_thieu.round(2))
print(f"Trong {int(eda.ltv.isna().sum()):,} hồ sơ thiếu LTV, "
      f"có {int((eda.credit_type.eq('equi') & eda.ltv.isna()).sum()):,} hồ
sơ loại equi.")
```



Tuổi không xác định (age=unknown): 200 | status=1: 200

Bảng 3.4a. Tình trạng thiếu LTV theo credit_type:

	LTV có dữ liệu	LTV thiếu	Tỷ lệ thiếu LTV (%)
credit_type			
cib	48151	1	0.000
crif	43900	1	0.000
equi	202	15096	98.680
exp	41319	0	0.000

Trong 15,098 hồ sơ thiếu LTV, có 15,096 hồ sơ loại equi.

```
# Đối chiếu cơ cấu mẫu: sự thiếu LTV gần như loại nhóm EQUI khỏi phân
tích LTV.
equi = eda['credit_type'].eq('equi')
co_ltv = eda['ltv'].notna()
bang_doi_chieu_equi = pd.DataFrame([
   {'Mẫu': ten, 'N': int(mask.sum()), 'Số status=1': int(eda.loc[mask,
'status'].sum()),
    'Tỷ lệ status=1 (%)': 100 * eda.loc[mask, 'status'].mean()}
```

```
    for ten, mask in [('Toàn bộ dữ liệu', pd.Series(True,
index=eda.index)),
                      ('Không gồm EQUI', ~equi),
                      ('Có dữ liệu LTV', co_ltv)]
]).set_index('Mẫu')
print('Bảng 3.4b. Đối chiếu tỷ lệ status=1 và cơ cấu EQUI khi chọn mẫu:')
display(bang_doi_chieu_equi.round(2))
print(f"EQUI: {equi.sum():,} hồ sơ, {int(eda.loc[equi, 'status'].sum()):,}
status=1; "
      f"chiếm {100 * (equi & eda.status.eq(1)).sum() /
eda.status.sum():.2f}% "
      'tổng số status=1.')
print(f"EQUI chiếm {(equi & ~co_ltv).sum():,}/{(~co_ltv).sum():,} "
      f"({100*(equi & ~co_ltv).sum()/(~co_ltv).sum():.2f}%) hồ sơ thiếu
LTV.")
```

Bảng 3.4b. Đối chiếu tỷ lệ status=1 và cơ cấu EQUI khi chọn mẫu:

	N	Số status=1	Tỷ lệ status=1 (%)
Mẫu			
Toàn bộ dữ liệu	148670	36639	24.640
Không gồm EQUI	133372	21342	16.000
Có dữ liệu LTV	133572	21543	16.130

EQUI: 15,298 hồ sơ, 15,297 status=1; chiếm 41.75% tổng số status=1.
EQUI chiếm 15,096/15,098 (99.99%) hồ sơ thiếu LTV.

Nhận xét:
Tỷ lệ status=1 trong mẫu có LTV gần tỷ lệ khi bỏ EQUI; sự giảm so với toàn mẫu chủ yếu phản ánh thay đổi cơ cấu credit_type, không chứng minh LTV là nguyên nhân làm thay đổi tình trạng vỡ nợ.
Trong 15.098 hồ sơ thiếu LTV có 15.096 hồ sơ thuộc `EQUI` (99,99%); nhóm `EQUI` có 15.297/15.298 hồ sơ mang `status = 1`. Khi bỏ `EQUI`, tỷ lệ `status = 1` còn 16,00%, gần với 16,13% ở mẫu có LTV.

Kết luận: Chênh lệch tỷ lệ chủ yếu gắn với thay đổi cơ cấu `credit_type`, không chứng minh LTV gây ra vỡ nợ. Nhóm EQUI gần như 100% vỡ nợ là bất thường, cần xác minh định nghĩa biến trước khi dùng.

3.3.3. Ý nghĩa đối với kết quả phân tích

Phát hiện: Dữ liệu thiếu không ngẫu nhiên mà gắn chặt với status (Bảng 3.3).

- interest_rate_spread thiếu ở 100% hồ sơ status = 1 và gần như 0% ở status = 0
- rate_of_interest và upfront_charges thiếu ở hơn 99% nhóm 1.
- dtir1 thiếu 44,52% ở nhóm 1 so với 6,97% ở nhóm 0
- LTV thiếu 41,20% ở nhóm 1 so với gần 0% ở nhóm 0
- income thiếu nhiều hơn ở nhóm 0 (7,06% so với 3,38%).

Tác động 1: Lệch cơ cấu mẫu

Nếu chỉ giữ hồ sơ có LTV, tỷ lệ status = 1 giảm từ 24,64% (148.670 hồ sơ) xuống 16,13% (133.572 hồ sơ). Lý do chủ yếu là nhóm credit_type = EQUI: nhóm này có 15.297/15.298 hồ sơ status = 1 (chiếm 41,75% tổng số hồ sơ nhóm 1) và chiếm 15.096/15.098 hồ sơ thiếu LTV. Khi bỏ EQUI, tỷ lệ còn 16,00%, gần bằng 16,13%. Như vậy mức giảm do thay đổi cơ cấu mẫu, không phải bằng chứng LTV ảnh hưởng đến vỡ nợ.

Tác động 2: Nguy cơ rò rỉ thông tin

Ba biến lãi suất, chênh lệch lãi suất và phí ban đầu gần như báo trước nhãn nên không được dùng để giải thích hay dự báo, và Chương 3 không phân tích chúng theo status.

Bài học:

- Mọi tỷ lệ phải ghi rõ số quan sát hợp lệ và mẫu số
- Kết quả về LTV và DTI chỉ đúng cho hồ sơ có dữ liệu, trên thực tế gần như không gồm EQUI
- Không điền hay coi giá trị thiếu là 0 chỉ để vẽ biểu đồ.

3.4. Phân tích mối quan hệ giữa các biến và tình trạng vỡ nợ

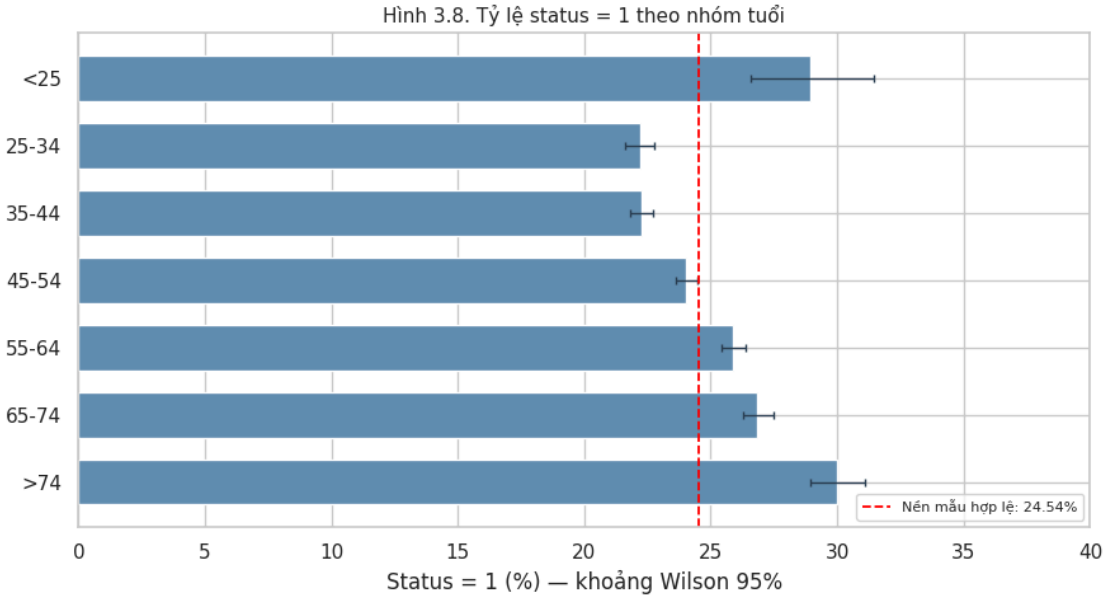
3.4.1. Tỷ lệ vỡ nợ theo nhóm tuổi

```
tuoi_hop_le = eda['age'].isin(age_order)
bang_tuoi = bang_ty_le(eda.loc[tuoi_hop_le, 'age'], age_order)
display(bang_tuoi.round(2))
fig, ax = plt.subplots(figsize=(9, 5))
ve_ty_le(ax, bang_tuoi, 100*eda.loc[tuoi_hop_le, 'status'].mean(),
         'Hình 3.8. Tỷ lệ status = 1 theo nhóm tuổi')
```

```
plt.tight_layout(); plt.show()
```

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
<25	1337	387	28.950	26.580	31.430
25-34	19142	4248	22.190	21.610	22.790
35-44	32818	7309	22.270	21.820	22.720
45-54	34720	8349	24.050	23.600	24.500
55-64	32534	8422	25.890	25.410	26.370
65-74	20744	5571	26.860	26.260	27.460
>74	7175	2153	30.010	28.960	31.080

Hình 3.8. Tỷ lệ status = 1 theo nhóm tuổi



Nhận xét: Tỷ lệ status = 1 thấp nhất ở nhóm 25–34 tuổi (22,19%) và cao nhất ở nhóm trên 74 tuổi (30,01%), chênh khoảng 7,8 điểm phần trăm. Hai nhóm 25–34 và 35–44 có khoảng tin cậy chồng nhau nên không khác biệt; tỷ lệ chỉ tăng đều từ 45 tuổi trở lên. Nhóm dưới 25 tuổi đạt 28,95% nhưng chỉ có 1.337 hồ sơ.

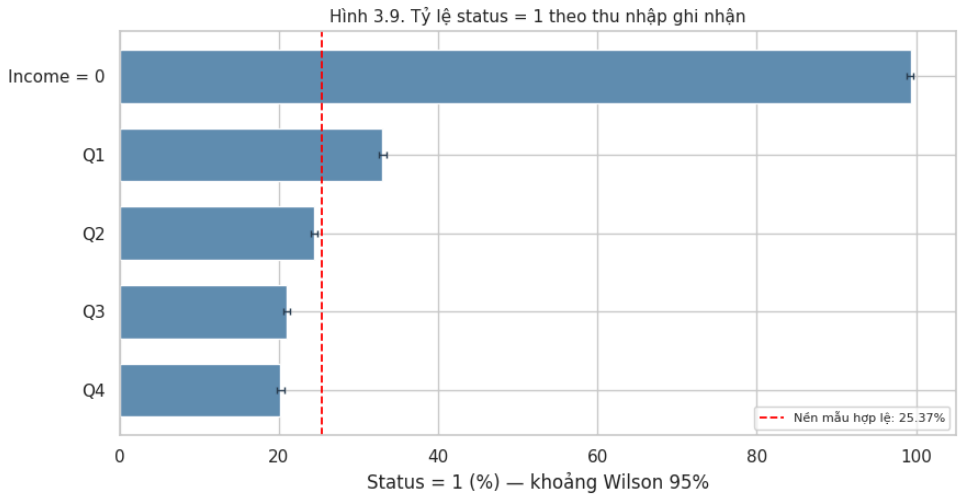
Kết luận: Tuổi có liên hệ nhưng yếu (Cramér's V = 0,050 ở 3.5); cần xét cỡ mẫu và không suy ra tuổi là nguyên nhân gây vỡ nợ.

3.4.2. Tỷ lệ vỡ nợ theo thu nhập

```
# Q1-Q4 chỉ trên income > 0; giữ nhóm income = 0 riêng, không gộp với Q1.
thu_nhap = pd.Series(pd.NA, index=eda.index, dtype='string')
co_thu_nhap = eda['income'].gt(0)
thu_nhap.loc[co_thu_nhap] = pd.qcut(eda.loc[co_thu_nhap, 'income'], 4,
                                    labels=['Q1', 'Q2', 'Q3',
'Q4']).astype('string')
thu_nhap.loc[eda['income'].eq(0)] = 'Income = 0'
thu_tu_income = ['Income = 0', 'Q1', 'Q2', 'Q3', 'Q4']
bang_income = bang_ty_le(thu_nhap, thu_tu_income)
nguong_income = pd.qcut(eda.loc[co_thu_nhap, 'income'], 4,
retbins=True)[1]
print('Các mốc tứ phân vị thu nhập dương:', [f'{x:,.0f}' for x in
nguong_income])
print('Mẫu income hợp lệ:', int(eda.income.notna().sum()), '| thiếu:',
int(eda.income.isna().sum()))
display(bang_income.round(2))
fig, ax = plt.subplots(figsize=(9, 4.8))
ve_ty_le(ax, bang_income, 100*eda.loc[eda.income.notna(),
'status'].mean(),
         'Hình 3.9. Tỷ lệ status = 1 theo thu nhập ghi nhận')
plt.tight_layout(); plt.show()
```

Các mốc tứ phân vị thu nhập dương: ['60', '3,780', '5,760', '8,580', '578,580']
Mẫu income hợp lệ: 139520 | thiếu: 9150

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
Income = 0	1260	1252	99.370	98.750	99.680
Q1	35075	11568	32.980	32.490	33.470
Q2	34293	8382	24.440	23.990	24.900
Q3	34791	7303	20.990	20.570	21.420
Q4	34101	6895	20.220	19.800	20.650



Nhận xét: Trong các hồ sơ thu nhập dương, tỷ lệ status = 1 giảm từ 32,98% (Q1) xuống 20,22% (Q4), giảm rõ từ Q1 đến Q3 rồi gần như đi ngang (20,99% → 20,22%). Riêng 1.260 hồ sơ income = 0 có tỷ lệ lên đến 99,37% (1.252 hồ sơ).

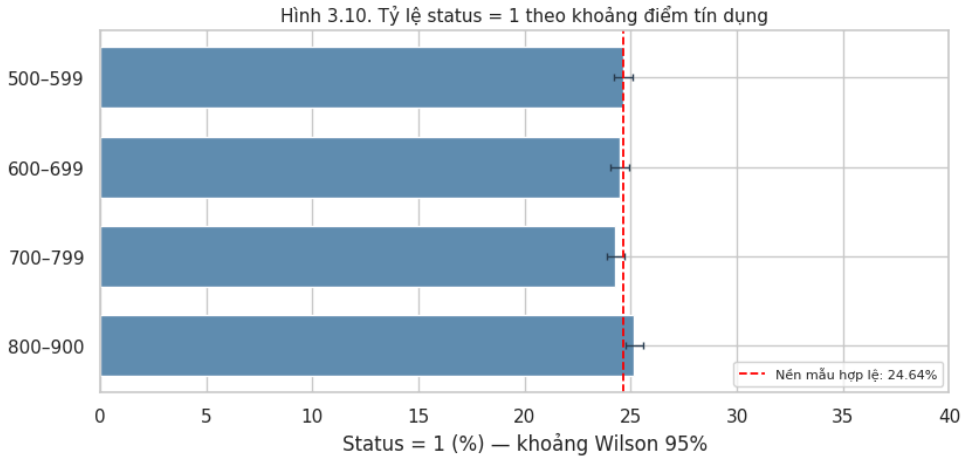
Kết luận: Thu nhập thấp liên hệ với tỷ lệ nhóm 1 cao hơn. Nhóm income = 0 rất bất thường, có thể là cách mã hóa của hồ sơ lỗi hoặc vỡ nợ chứ không phải thu nhập thật, nên phải xem riêng, không gộp vào Q1.

3.4.3. Điểm tín dụng và tình trạng vỡ nợ

```
# Credit score đủ dữ liệu; so sánh tỷ lệ status=1 ở bốn khoảng điểm
score_order = ['500-599', '600-699', '700-799', '800-900']
bang_score = bang_ty_le(eda['credit_score_band'], score_order)
display(bang_score.round(2))
fig, ax = plt.subplots(figsize=(8.5, 4.2))
ve_ty_le(ax, bang_score, 100*eda.status.mean(),
         'Hình 3.10. Tỷ lệ status = 1 theo khoảng điểm tín dụng')
plt.tight_layout(); plt.show()
```

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
500–599	37190	9160	24.630	24.200	25.070
600–699	37200	9108	24.480	24.050	24.920
700–799	36811	8940	24.290	23.850	24.730

800–900	37469	9431	25.170	24.730	25.610



Nhận xét: Tỷ lệ status = 1 ở bốn nhóm điểm tín dụng chỉ dao động 24,29%–25,17%, chênh tối đa 0,88 điểm phần trăm; các khoảng tin cậy chồng nhau và nhóm điểm cao nhất (800–900) lại có tỷ lệ cao nhất.

Kết luận: Điểm tín dụng không cho thấy khả năng phân biệt hai trạng thái, trái với kỳ vọng lý thuyết. Cùng với phân phối gần phẳng ở 3.2.3, điều này cho thấy biến này ít giá trị trong bộ dữ liệu hiện tại; kết quả kiểm định ở 3.5 (H3) sẽ xác nhận thêm.

3.4.4. Quy mô khoản vay và tình trạng vỡ nợ

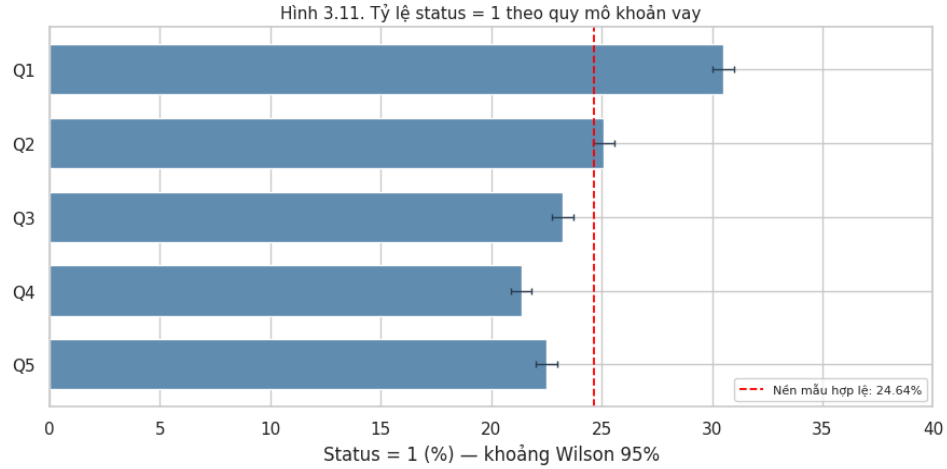
```
# loan_amount đủ dữ liệu: chia năm phân vị
nhom_vay = pd.qcut(eda['loan_amount'], 5, labels=['Q1', 'Q2', 'Q3', 'Q4',
'Q5'])
nguong_vay = pd.qcut(eda['loan_amount'], 5, retbins=True)[1]
print('Các mốc phân vị loan_amount:', [f'{x:,.0f}' for x in nguong_vay])
bang_vay = bang_ty_le(nhom_vay, ['Q1', 'Q2', 'Q3', 'Q4', 'Q5'])
display(bang_vay.round(2))
fig, ax = plt.subplots(figsize=(9, 4.6))
ve_ty_le(ax, bang_vay, 100*eda.status.mean(),
         'Hình 3.11. Tỷ lệ status = 1 theo quy mô khoản vay')
plt.tight_layout(); plt.show()
```

Các mốc phân vị loan_amount: ['16,500', '176,500', '256,500', '346,500', '476,500', '3,576,500']

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)

nhom					
Q1	31920	9737	30.500	30.000	31.010
Q2	29910	7503	25.090	24.600	25.580
Q3	28557	6633	23.230	22.740	23.720
Q4	30419	6499	21.360	20.910	21.830
Q5	27864	6267	22.490	22.000	22.990

Hình 3.11. Tỷ lệ status = 1 theo quy mô khoản vay



Nhận xét: Nhóm khoản vay nhỏ nhất Q1 (đến 176.500) có tỷ lệ status = 1 là 30,50%, cao hơn Q4 (21,36%); Q5 tăng nhẹ lên 22,49%. Mức giảm mạnh nhất nằm ở bước Q1 sang Q2 (5,4 điểm phần trăm).

Kết luận: Mối liên hệ có dạng giảm rồi đi ngang, không tăng đều theo số tiền vay, nên không thể kết luận khoản vay lớn hơn luôn rủi ro hơn. Khoản vay nhỏ thường đi cùng thu nhập thấp (ρ = 0,642 ở 3.6.1) nên hai hiệu ứng này khó tách riêng.

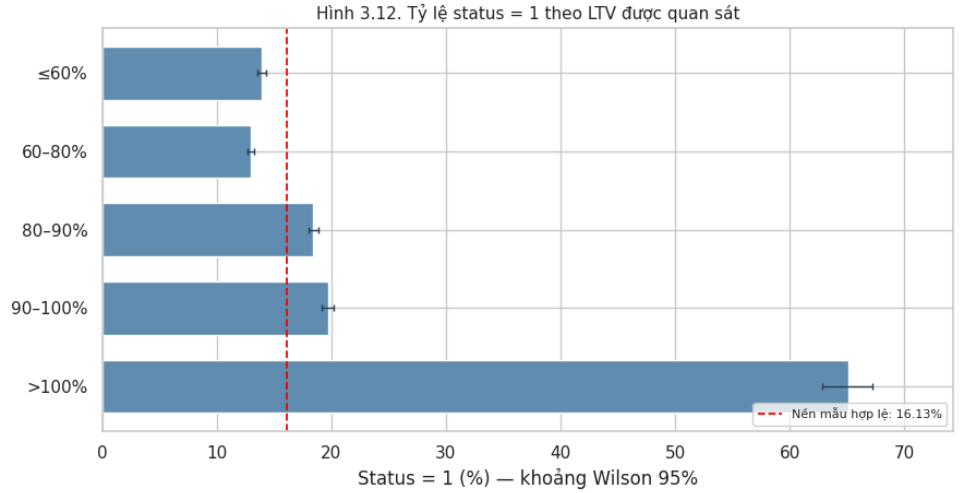
3.4.5. Tỷ lệ LTV và tình trạng vỡ nợ

```
# LTV là tỷ lệ phần trăm theo dữ liệu; loại NaN khỏi mẫu số nhưng giữ mọi
giá trị cực trị.
nhom_ltv = pd.cut(eda['ltv'], [-np.inf, 60, 80, 90, 100, np.inf],
                  labels=['≤60%', '60-80%', '80-90%', '90-100%', '>100%'])
ltv_order = ['≤60%', '60-80%', '80-90%', '90-100%', '>100%']
bang_ltv = bang_ty_le(nhom_ltv, ltv_order)
display(bang_ltv.round(2))
fig, ax = plt.subplots(figsize=(9, 4.8))
ve_ty_le(ax, bang_ltv, 100*eda.loc[eda.ltv.notna(), 'status'].mean(),
```

```
        'Hình 3.12. Tỷ lệ status = 1 theo LTV được quan sát')
plt.tight_layout(); plt.show()

# Kiểm tra độ nhạy riêng nhóm >100% nếu không tính sáu giá trị cực trị
>1000%.
ltv_100_1000 = eda['ltv'].gt(100) & eda['ltv'].le(1000)
print(f'Nhóm 100% < LTV ≤ 1000%: N={ltv_100_1000.sum():,}, '
      f'status=1={eda.loc[ltv_100_1000, "status"].mean():.2%}; '
      f'LTV >1000%: {eda.ltv.gt(1000).sum():,} hồ sơ.')
```

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
≤60%	32565	4534	13.920	13.550	14.300
60–80%	49895	6456	12.940	12.650	13.240
80–90%	26153	4822	18.440	17.970	18.910
90–100%	23160	4560	19.690	19.180	20.210
>100%	1799	1171	65.090	62.860	67.260



Nhóm 100% < LTV ≤ 1000%: N=1,793, status=1=65.25%; LTV >1000%: 6 hồ sơ.

Nhận xét: Nhóm LTV trên 100% có tỷ lệ status = 1 đạt 65,09% (1.799 hồ sơ), cao hơn rõ rệt nhóm LTV 60–80% (12,94%). Nhóm ≤60% (13,92%) không thấp hơn nhóm 60–80%, tức quan

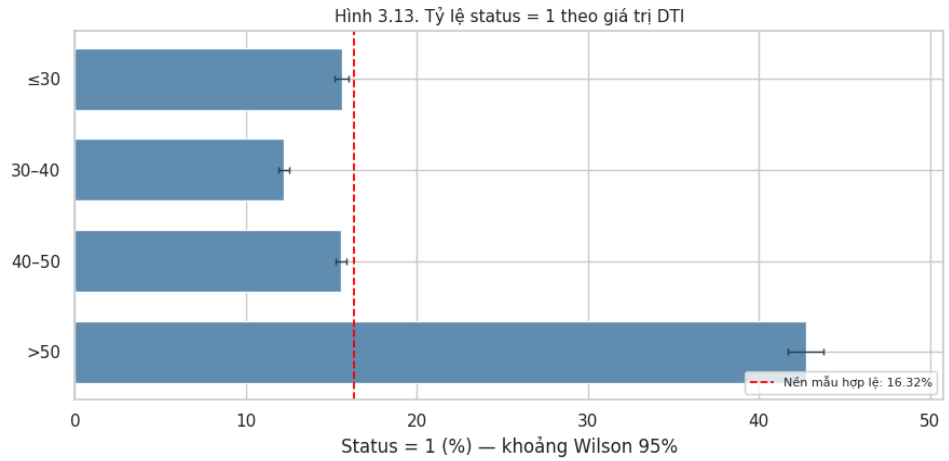
hệ không đơn điệu; bước nhảy chủ yếu nằm ở mốc 100%. Bỏ sáu giá trị LTV trên 1.000% thì tỷ lệ nhóm trên 100% vẫn là 65,25%.

Kết luận: Khi khoản vay vượt giá trị tài sản, rủi ro tăng mạnh. Nhóm này chỉ chiếm 1,35% mẫu có LTV nhưng chứa 5,4% số hồ sơ vỡ nợ trong mẫu đó. Đây là tín hiệu liên hệ trong mẫu có LTV (gần như không gồm EQUI), không phải bằng chứng nhân quả.

3.4.6. DTI và tình trạng vỡ nợ

```
# So sánh phân tầng DTI trên đúng tập có dtir1 quan sát được.
nhom_dti = pd.cut(eda['dtir1'], [-np.inf, 30, 40, 50, np.inf],
                  labels=['≤30', '30–40', '40–50', '>50'])
dti_order = ['≤30', '30–40', '40–50', '>50']
bang_dti = bang_ty_le(nhom_dti, dti_order)
display(bang_dti.round(2))
fig, ax = plt.subplots(figsize=(9, 4.5))
ve_ty_le(ax, bang_dti, 100*eda.loc[eda.dtir1.notna(), 'status'].mean(),
         'Hình 3.13. Tỷ lệ status = 1 theo giá trị DTI')
plt.tight_layout(); plt.show()
```

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
≤30	27970	4364	15.600	15.180	16.030
30–40	41861	5109	12.200	11.890	12.520
40–50	46079	7164	15.550	15.220	15.880
>50	8639	3692	42.740	41.700	43.780



Nhận xét: Nhóm DTI trên 50 có tỷ lệ status = 1 là 42,74%, so với 12,20% ở nhóm DTI 30–40; các nhóm dưới 50 không tăng đều theo DTI (15,60%, 12,20%, 15,55%). Nhóm DTI trên 50 chiếm 6,9% mẫu nhưng chứa 18,2% số hồ sơ vỡ nợ trong mẫu có DTI.

Kết luận: Nhóm DTI trên 50 có tỷ lệ status = 1 cao nổi bật (42,74%) so với các nhóm DTI còn lại (12,20%–15,60%). Đây là ngưỡng phân nhóm đáng quan tâm khi phân tích rủi ro, nhưng chưa được kiểm định là điểm gãy hoặc ngưỡng dự báo tối ưu. Kết quả chỉ đại diện cho 124.549 hồ sơ có dữ liệu DTI hợp lệ.

3.4.7. Loại khoản vay và mục đích vay

```
# Bảng: Loại khoản vay và mục đích vay
loan_type_order = ['type1', 'type2', 'type3']
purpose_order = ['p1', 'p2', 'p3', 'p4', 'unknown']
bang_loai_vay = bang_ty_le(eda['loan_type'], loan_type_order)
bang_muc_dich = bang_ty_le(eda['loan_purpose'], purpose_order)
print('Loại khoản vay:'); display(bang_loai_vay.round(2))
print('Mục đích vay:'); display(bang_muc_dich.round(2))

# Hình 3.14: Tỷ lệ status = 1 theo loại và mục đích vay
fig, axes = plt.subplots(1, 2, figsize=(13.5, 4.7))
ve_ty_le(axes[0], bang_loai_vay, 100*eda.status.mean(), 'Loại khoản vay')
ve_ty_le(axes[1], bang_muc_dich, 100*eda.status.mean(), 'Mục đích vay')
fig.suptitle('Hình 3.14. Tỷ lệ status = 1 theo loại và mục đích vay',
fontsize=12)
plt.tight_layout(); plt.show()
```

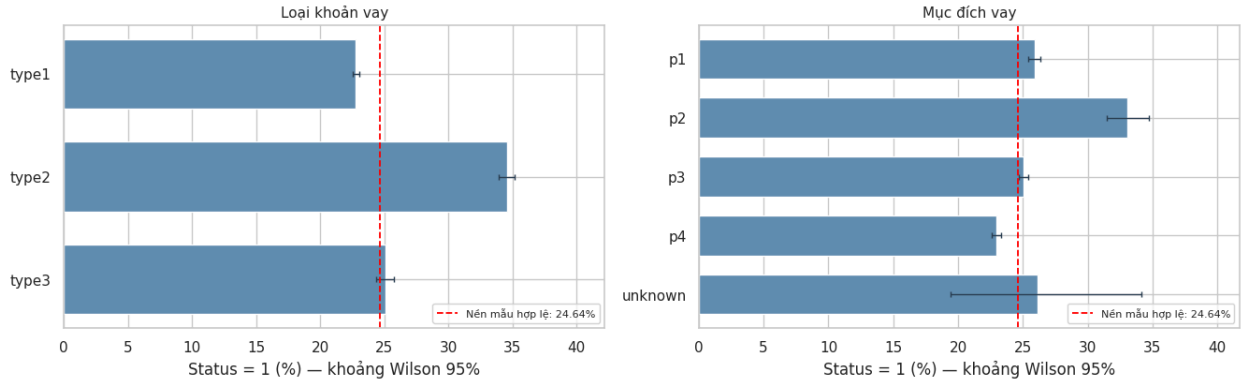
Loại khoản vay:

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
type1	113173	25775	22.770	22.530	23.020
type2	20762	7172	34.540	33.900	35.190
type3	14735	3692	25.060	24.360	25.760

Mục đích vay:

	N	Status=1	Tỷ lệ (%)	CI thấp (%)	CI cao (%)
nhom					
p1	34529	8935	25.880	25.420	26.340
p2	3274	1083	33.080	31.490	34.710
p3	55934	13996	25.020	24.670	25.380
p4	54799	12590	22.970	22.620	23.330
unknown	134	35	26.120	19.420	34.150

Hình 3.14. Tỷ lệ status = 1 theo loại và mục đích vay



Nhận xét: type2 có tỷ lệ status = 1 cao nhất trong ba loại vay (34,54%), cao hơn type1 (22,77%) 11,77 điểm phần trăm; type2 chính là nhóm khoản vay kinh doanh/thương mại và chiếm 13,97% hồ sơ. Theo mục đích, chỉ p2 nổi bật (33,08%, chỉ 3.274 hồ sơ), trong khi p1, p3, p4 chênh nhau nhỏ (22,97%–25,88%); nhóm unknown (134 hồ sơ) có khoảng tin cậy quá rộng để kết luận.

Kết luận: Khoản vay kinh doanh có rủi ro cao hơn khoản vay thông thường, còn mục đích vay ít phân biệt. Không suy ra quan hệ nhân quả.

3.5. Kiểm định các giả thuyết nghiên cứu

```
from scipy.stats import chi2_contingency, mannwhitneyu,
false_discovery_control
from itertools import combinations

# 1. Kiểm định Chi-square và tính Cramér's V
def kiem_chi2(ma, ten, nhom):
   tmp = pd.DataFrame({'nhom': nhom, 'status': eda['status']}).dropna()
   bang = pd.crosstab(tmp['nhom'], tmp['status']).reindex(columns=[0, 1],
fill_value=0
   chi2, p, do_tu_do, expected = chi2_contingency(bang, correction=False)
   assert expected.min() >= 5, f'Ô tần số kỳ vọng nhỏ ở {ma}; cần thay
kiểm định.'
   v = np.sqrt(chi2 / (len(tmp) * min(bang.shape[0]-1, bang.shape[1]-1)))
   return [ma, ten, len(tmp), f'χ²({do_tu_do}) = {chi2:,.1f}', p, f'V =
{v:.3f}', v]

# 2. Kiểm định Mann-Whitney U và tính Cliff's delta
def kiem_mann_whitney(ma, ten, bien):
   mau_1 = eda.loc[eda['status'].eq(1), bien].dropna()
   mau_0 = eda.loc[eda['status'].eq(0), bien].dropna()
   u, p = mannwhitneyu(mau_1, mau_0, alternative='two-sided',
method='asymptotic')
   delta = 2*u / (len(mau_1)*len(mau_0)) - 1
   return [ma, ten, len(mau_1)+len(mau_0), f'U = {u:,.0f}', p,
           f'Cliff δ = {delta:+.3f}', delta]

# 3. Thực hiện 7 kiểm định
kiem_dinh = [
   kiem_chi2('H1a', 'Nhóm tuổi × status',
eda['age'].where(eda['age'].isin(age_order))),
   kiem_chi2('H1b', 'Loại vay × status', eda['loan_type']),
   kiem_mann_whitney('H2', 'Income theo status', 'income'),
   kiem_mann_whitney('H3', 'Credit score theo status', 'credit_score'),
   kiem_chi2('H4', '5 phân vị loan_amount × status', nhom_vay),
```

```
    kiem_chi2('H5a', 'Nhóm LTV × status', nhom_ltv),
    kiem_chi2('H5b', 'Nhóm DTI × status', nhom_dti)
]
ket_qua_35 = pd.DataFrame(kiem_dinh, columns=[
    'Mã', 'Giả thuyết', 'N', 'Thống kê', 'p_thuc', 'Hiệu ứng', 'Hiệu ứng
số'
])
# 4. Hiệu chỉnh FDR bằng Benjamini-Hochberg
ket_qua_35['q_fdr'] = false_discovery_control(
    ket_qua_35['p_thuc'].to_numpy(), method='bh'
)
# 5. Trình bày bảng kết quả.
```

Bảng 3.5. Kết quả 7 phép kiểm định (p=0 do tràn số chỉ biểu thị là p rất nhỏ):

	Mã	Giả thuyết	N	Thống kê	Hiệu ứng	p	q (FDR)	Kết luận (α=5%)
0	H1a	Nhóm tuổi × status	148470	$\chi^2(6) = 374.5$	V = 0.050	8.44e-78	9.85e-78	Có bằng chứng liên hệ
1	H1b	Loại vay × status	148670	$\chi^2(2) = 1,310.0$	V = 0.094	3.52e-285	6.16e-285	Có bằng chứng liên hệ
2	H2	Income theo status	139520	U = 1,502,807,176	Cliff δ = -0.185	<1e-300	<1e-300	Có bằng chứng liên hệ
3	H3	Credit score theo status	148670	U = 2,063,311,078	Cliff δ = +0.005	0.1243	0.1243	Chưa đủ bằng chứng
4	H4	5 phân vị loan_amount × status	148670	$\chi^2(4) = 870.0$	V = 0.076	5.38e-187	7.53e-187	Có bằng chứng liên hệ
5	H5a	Nhóm LTV × status	133572	$\chi^2(4) = 4,000.8$	V = 0.173	<1e-300	<1e-300	Có bằng chứng liên hệ
6	H5b	Nhóm DTI × status	124549	$\chi^2(3) = 4,963.7$	V = 0.200	<1e-300	<1e-300	Có bằng chứng liên hệ

Đối chiếu trung vị:

	income	credit_score
status		
Status = 0	6,000.000	699.000
Status = 1	4,860.000	700.000

Phương pháp: Mức ý nghĩa 5%. Dùng Chi-square cho biến phân loại và Mann–Whitney U cho biến định lượng. P-value được hiệu chỉnh nhiều phép thử bằng Benjamini–Hochberg (FDR) và kết luận theo q < 0,05. Vì mẫu rất lớn nên p-value gần như luôn nhỏ, nên (Cramér's V < 0,10 là rất yếu, 0,10–0,30 là yếu; |Cliff's δ| < 0,147 là không đáng kể, 0,147–0,33 là nhỏ). Với mỗi giả thuyết, H0 là "không có liên hệ với `status`"

Mã	Biến kiểm định	Hiệu ứng	Kết luận
H1a	Nhóm tuổi	V = 0,050 (rất yếu)	Bác bỏ H0. Cao nhất nhóm >74 (30,01%), thấp nhất 25–34 (22,19%).
H1b	Loại vay	V = 0,094 (rất yếu)	Bác bỏ H0. type2 34,54% so với type1 22,77%.
H2	Thu nhập	δ = −0,185 (nhỏ)	Bác bỏ H0. Trung vị 4.860 (nhóm 1) so với 6.000 (nhóm 0). Hiệu ứng nhạy với income = 0 (δ = −0,155 nếu bỏ nhóm này, xem 3.6.5).
H3	Điểm tín dụng	δ = +0,005 (không đáng kể)	**Không bác bỏ H0.** Trung vị 700 so với 699.
H4	Quy mô vay (5 nhóm)	V = 0,076 (rất yếu)	Bác bỏ H0, nhưng không đơn điệu: Q1 30,50% → Q4 21,36% → Q5 22,49%.
H5a	LTV	V = 0,173 (yếu)	Bác bỏ H0. LTV > 100%: 65,09%, so với 12,94% ở nhóm 60–80%.
H5b	DTI	V = 0,200 (yếu)	Bác bỏ H0. DTI > 50: 42,74%, so với 12,20% ở nhóm 30–40.

Kết luận: Có 6/7 giả thuyết bác bỏ được H0. Tuy nhiên, chỉ DTI và LTV đạt mức liên hệ "yếu". Tuổi, loại vay và quy mô vay chỉ "rất yếu". Điểm tín dụng không phân biệt được hai nhóm.

- Ý nghĩa thống kê không đồng nghĩa với liên hệ mạnh. Cần đọc p-value cùng mức hiệu ứng.
- Nhóm DTI > 50 và LTV > 100% ghi nhận tỷ lệ `status = 1` cao nổi bật so với các nhóm còn lại. Đây là các ngưỡng phục vụ phân tích phân khúc, chưa phải điểm gãy thống kê hoặc ngưỡng dự báo tối ưu đã được xác thực. Kết quả này chỉ áp dụng cho hồ sơ có dữ liệu (133.572 hồ sơ có LTV, 124.549 hồ sơ có DTI), gần như không gồm EQUI.
- `credit_type` có liên hệ mạnh nhất (V = 0,592, mục 3.6) nhưng không đưa vào 7 giả thuyết vì nghi rò rỉ nhãn (EQUI).
- Các kết quả là liên hệ thống kê, không chứng minh nhân quả.

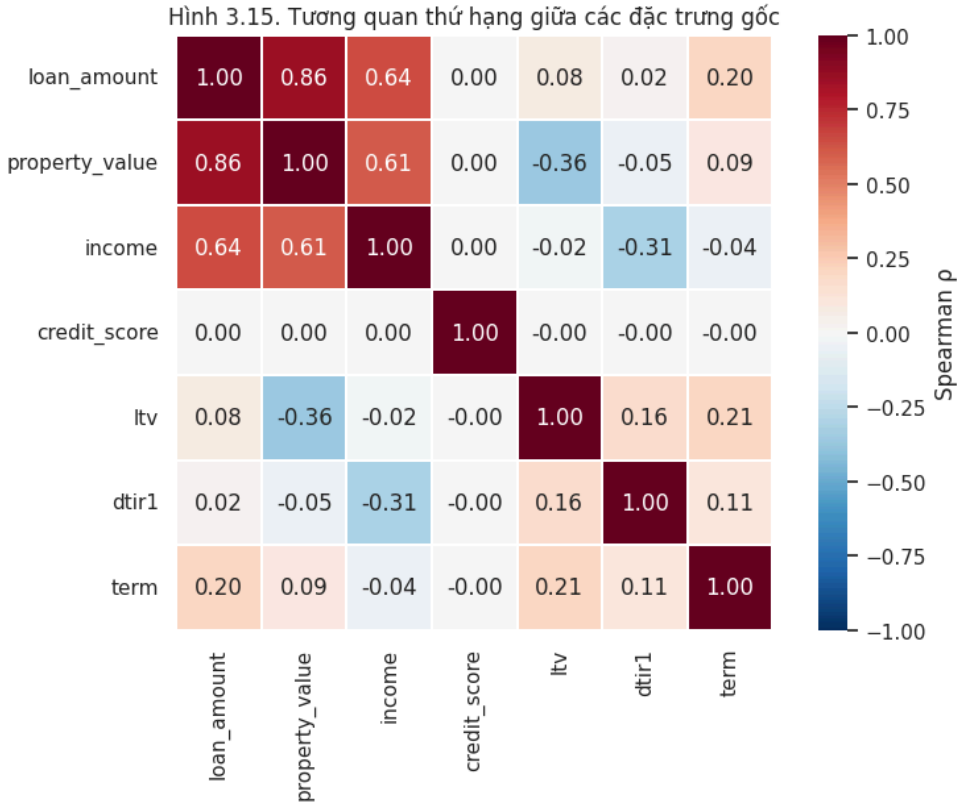
3.6. Phân tích mối quan hệ đa biến

3.6.1. Ma trận tương quan

```
# Hình 3.15 — Tương quan Spearman giữa các biến gốc (không đưa status vào ma trận).
bien_tuong_quan = ['loan_amount', 'property_value', 'income', 'credit_score',
                   'ltv', 'dtir1', 'term']
ma_tran_spearman = eda[bien_tuong_quan].corr(method='spearman')
ma_n = eda[bien_tuong_quan].notna().astype(int).T.dot(
   eda[bien_tuong_quan].notna().astype(int))

fig, ax = plt.subplots(figsize=(8.4, 6.4))
sns.heatmap(ma_tran_spearman, ax=ax, annot=True, fmt='.2f',
            vmin=-1, vmax=1, center=0, cmap='RdBu_r', square=True,
            linewidths=.35, cbar_kws={'label': 'Spearman ρ'})
ax.set_title('Hình 3.15. Tương quan thứ hạng giữa các đặc trưng gốc')
plt.tight_layout(); plt.show()
print('Số cặp quan sát hợp lệ (N) theo từng ô:')
display(ma_n)
```

Hình 3.15. Tương quan thứ hạng giữa các đặc trưng gốc



Số cặp quan sát hợp lệ (N) theo từng ô:

	loan_amount	property_value	income	credit_score	ltv	dtir1	term
loan_amount	148670	133572	139520	148670	133572	124549	148670
property_value	133572	133572	124437	133572	133572	124547	133572
income	139520	124437	139520	139520	124437	124439	139520
credit_score	148670	133572	139520	148670	133572	124549	148670
ltv	133572	133572	124437	133572	133572	124547	133572
dtir1	124549	124547	124439	124549	124547	124549	124549
term	148670	133572	139520	148670	133572	124549	148670

Nhận xét: Heatmap cho thấy khoản vay và giá trị tài sản tương quan thuận mạnh (Spearman ρ ≈ 0,857); thu nhập tương quan thuận với quy mô vay (ρ ≈ 0,642) và với giá trị tài sản (ρ ≈ 0,606). Tương quan giữa ltv và property_value (−0,365) là quan hệ cơ học vì LTV = khoản vay / giá trị tài sản. credit_score không nằm trong cặp nào có |ρ| ≥ 0,30.

Kết luận: Ba biến loan_amount, property_value, income có thông tin chồng lặp, dễ gây đa cộng tuyến khi xây mô hình. Điểm tín dụng độc lập với mọi biến tài chính, củng cố nghi vấn dữ liệu mô phỏng. Tương quan giữa các đặc trưng không phải bằng chứng về tác động gây vỡ nợ.

3.6.2. Phát hiện mối liên hệ mạnh giữa các đặc trưng

```
# Liệt kê duy nhất các cặp đạt |ρ| >= 0,30, kèm N quan sát của từng cặp
cap_manh = pd.DataFrame([
    {'Cặp biến': f'{a} ↔ {b}', 'Spearman ρ': ma_tran_spearman.loc[a, b],
     'N cặp hợp lệ': int(ma_n.loc[a, b])}
    for a, b in combinations(bien_tuong_quan, 2)
    if abs(ma_tran_spearman.loc[a, b]) >= .30
]).sort_values('Spearman ρ', key=lambda s: s.abs(), ascending=False)
print('Bảng 3.6. Các cặp có |ρ| ≥ 0,30:')
display(cap_manh.round(3))
```

Bảng 3.6. Các cặp có |ρ| ≥ 0,30:

	Cặp biến	Spearman ρ	N cặp hợp lệ
0	loan_amount ↔ property_value	0.857	133572
1	loan_amount ↔ income	0.642	139520
2	property_value ↔ income	0.606	124437
3	property_value ↔ ltv	-0.365	133572
4	income ↔ dtir1	-0.307	124439

Nhận xét: Có 5 cặp biến đạt |ρ| ≥ 0,30; mạnh nhất là loan_amount - property_value (0,857). income - dtir1 tương quan nghịch ở mức −0,307, tức thu nhập cao thường đi cùng DTI thấp.

Kết luận: Khi phân tích đồng thời các đặc trưng, cần lưu ý sự trùng lặp thông tin, đồng thời báo cáo N hợp lệ khác nhau theo từng cặp.

3.6.3. Liên hệ biến phân loại với status

```
# Loại 'unknown' ở age và submission_of_application để không nhầm thiếu gốc với nhóm thực.
rows_v = []
for col in all_categorical:
    if col not in eda.columns:
        continue
    tmp = eda[[col, 'status']].dropna()
    if col in ['age', 'submission_of_application']:
        tmp = tmp.loc[tmp[col].ne('unknown')]
    tab = pd.crosstab(tmp[col], tmp['status']).reindex(columns=[0, 1], fill_value=0)
    if min(tab.shape) < 2:
        continue
    chi2 = chi2_contingency(tab, correction=False)[0]
    v = np.sqrt(chi2 / (len(tmp) * min(tab.shape[0]-1, tab.shape[1]-1)))
    rows_v.append({'Biến phân loại': col, 'N hợp lệ': len(tmp), 'Cramér V': v})
bang_v_phan_loai = pd.DataFrame(rows_v).sort_values(
    'Cramér V', ascending=False).reset_index(drop=True)
print('Bảng 3.6b. Liên hệ với status: N tính riêng cho từng biến;'
      ' loại unknown của tuổi và hình thức nộp hồ sơ:')
display(bang_v_phan_loai.round(3))
# Đối chiếu mục 3.5: cùng định nghĩa nhóm tuổi -> cùng kích thước hiệu ứng.
v_tuoi = bang_v_phan_loai.loc[bang_v_phan_loai['Biến phân loại'].eq('age'), 'Cramér V'].iloc[0]
v_h1a = ket_qua_35.loc[ket_qua_35['Mã'].eq('H1a'), 'Hiệu ứng số'].iloc[0]
assert np.isclose(v_tuoi, v_h1a), 'Cramér V tuổi ở 3.5 và 3.6.2 không khớp.'
```

Bảng 3.6b. Liên hệ với status: N tính riêng cho từng biến; loại unknown của tuổi và hình thức nộp hồ sơ:

	Biến phân loại	N hợp lệ	Cramér V
0	credit_type	148670	0.592
1	lump_sum_payment	148670	0.188
2	neg_ammortization	148670	0.156
3	co_applicant_credit_type	148670	0.144
4	submission_of_application	148470	0.121
5	loan_type	148670	0.094
6	gender	148670	0.084
7	loan_limit	148670	0.054
8	region	148670	0.051
9	age	148470	0.050
10	loan_purpose	148670	0.040
11	approv_in_adv	148670	0.037
12	credit_worthiness	148670	0.035
13	occupancy_type	148670	0.030
14	total_units	148670	0.029
15	interest_only	148670	0.014
16	open_credit	148670	0.010
17	credit_score_band	148670	0.008

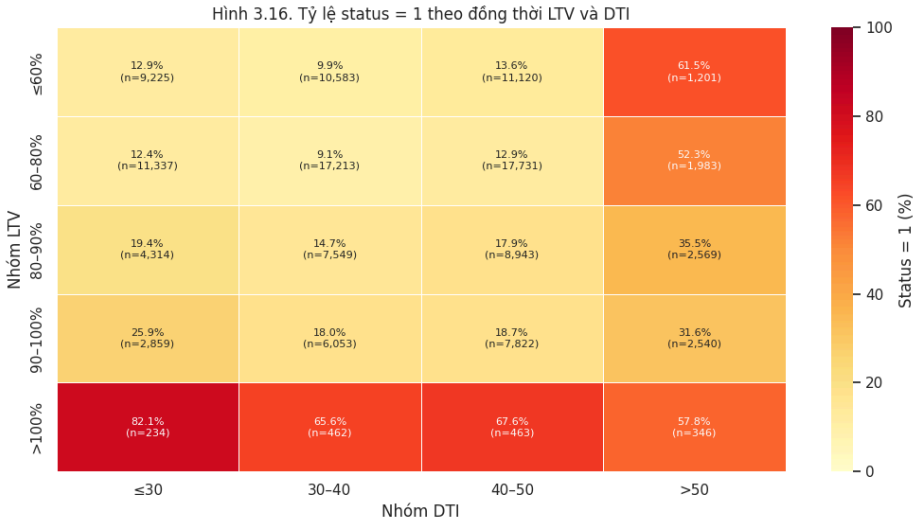
Nhận xét: credit_type có liên hệ mạnh nhất với status trong các biến phân loại (Cramér's V = 0,592), tiếp đến là lump_sum_payment (0,188) và neg_ammortization (0,156). Riêng nhóm tuổi

có V = 0,050 khi loại unknown, thống nhất với mục 3.5. credit_score_band có V thấp nhất (0,008), phù hợp kết quả H3.

Kết luận: Tín hiệu của credit_type quá mạnh so với các biến còn lại, cùng với nhóm EQUI gần như 100% vỡ nợ, cho thấy biến này nhiều khả năng phản ánh quy trình ghi nhận hoặc rò rỉ nhãn, cần xác minh trước khi dùng để dự báo.

3.6.4. Phân tích phân khúc kết hợp

```
# Hình 3.16 — Phân khúc kết hợp LTV × DTI trên cùng một tập hồ sơ đủ hai chỉ số.
phan_khuc = pd.DataFrame({'LTV': nhom_ltv, 'DTI': nhom_dti,
                          'status': eda['status']}).dropna()
grid_36 = phan_khuc.groupby(['LTV', 'DTI'], observed=True)['status'].agg(
   N='size', So_1='sum', Ty_le='mean')
grid_pct = grid_36['Ty_le'].unstack('DTI').reindex(index=ltv_order,
columns=dti_order) * 100
grid_n = grid_36['N'].unstack('DTI').reindex(index=ltv_order,
columns=dti_order)
# Chỉ hiện ô đủ 100 hồ sơ; ô ít hơn không đủ ổn định để so sánh trực quan.
nhan = grid_pct.copy().astype(object)
for i in range(len(nhan.index)):
   for j in range(len(nhan.columns)):
       n = grid_n.iloc[i, j]
       nhan.iloc[i, j] = f'{grid_pct.iloc[i, j]:.1f}%\n(n={int(n):,})' if
pd.notna(n) and n >= 100 else 'N < 100'
fig, ax = plt.subplots(figsize=(10.5, 5.8))
sns.heatmap(grid_pct.where(grid_n >= 100), annot=nhan, fmt='', ax=ax,
           cmap='YlOrRd', vmin=0, vmax=100, linewidths=.4,
           annot_kws={'fontsize': 8}, cbar_kws={'label': 'Status = 1
(%)'})
ax.set_title('Hình 3.16. Tỷ lệ status = 1 theo đồng thời LTV và DTI')
ax.set_xlabel('Nhóm DTI'); ax.set_ylabel('Nhóm LTV')
plt.tight_layout(); plt.show()
print(f'Mẫu có cả LTV và DTI: {len(phan_khuc):,} hồ sơ; '
     f'tỷ lệ status=1: {100*phan_khuc.status.mean():.2f}%; '
     f'nhóm nhỏ nhất: {int(grid_n.min().min()):,} hồ sơ.')
assert int(grid_n.sum().sum()) == len(phan_khuc)
```



Mẫu có cả LTV và DTI: 124,547 hồ sơ; tỷ lệ status=1: 16.32%; nhóm nhỏ nhất: 234 hồ sơ.

Nhận xét: Trên 124.547 hồ sơ có đầy đủ LTV và DTI, tỷ lệ status = 1 giữa các phân khúc dao động từ 9,1% đến 82,1%, cho thấy sự khác biệt đáng kể giữa các nhóm. Nhóm LTV trên 100% ghi nhận tỷ lệ vỡ nợ cao ở các mức DTI (57,8%–82,1%). Tuy nhiên, tỷ lệ này không tăng đều khi DTI tăng: nhóm LTV trên 100% và DTI không quá 30 có tỷ lệ 82,1%, cao hơn nhóm đồng thời có LTV trên 100% và DTI trên 50 (57,8%). Tương tự, mức độ liên hệ của DTI trên 50 cũng khác nhau giữa các nhóm LTV.

Kết luận: Phân tích kết hợp LTV và DTI giúp nhận diện những phân khúc có tỷ lệ vỡ nợ khác nhau, nhưng chưa đủ bằng chứng để kết luận về hiệu ứng cộng dồn hoặc tương tác giữa hai chỉ số. Kết quả chỉ mang tính khám phá trên mẫu có dữ liệu hợp lệ và chưa kiểm soát các yếu tố gây nhiễu.

3.6.5. Kiểm tra độ nhạy của kết quả

```
# Bảng 3.7 — Sensitivity analysis: luôn đối chiếu hai nhóm trên cùng mẫu.
mau_chung = eda[['ltv', 'dtir1']].notna().all(axis=1)
mau_duong = mau_chung & eda['income'].gt(0)

cac_kich_ban = [
    ('LTV', 'Có LTV', eda['ltv'].notna(), 'ltv', 100),
    ('LTV', 'Có LTV, loại 6 cực trị >1.000%', eda['ltv'].notna() &
eda['ltv'].le(1000), 'ltv', 100),
    ('LTV', 'Có cả LTV và DTI', mau_chung, 'ltv', 100),
    ('LTV', 'Cả hai, income dương', mau_duong, 'ltv', 100),
    ('DTI', 'Có DTI', eda['dtir1'].notna(), 'dtir1', 50),
    ('DTI', 'Có cả LTV và DTI', mau_chung, 'dtir1', 50),
```

```
    ('DTI', 'Cả hai, income dương', mau_duong, 'dtir1', 50),
]
dong_do_nhay = []
for ten, kich_ban, mask, bien, nguong in cac_kich_ban:
    nhom_cao = mask & eda[bien].gt(nguong)
    nhom_thap = mask & eda[bien].le(nguong)
    p_cao = 100*eda.loc[nhom_cao, 'status'].mean()
    p_thap = 100*eda.loc[nhom_thap, 'status'].mean()
    dong_do_nhay.append({'Biến': ten, 'Kịch bản': kich_ban,
                         'N mẫu': int(mask.sum()), 'N nhóm cao':
int(nhom_cao.sum()),
                         'Tỷ lệ nhóm cao (%)': p_cao, 'Tỷ lệ còn lại (%)':
p_thap,
                         'Chênh lệch (đpt)': p_cao-p_thap})
bang_nhay_36 = pd.DataFrame(dong_do_nhay)
display(bang_nhay_36.round(2))
print('Sáu hồ sơ có LTV >1.000%:', int(eda.ltv.gt(1000).sum()),
      '| số hồ sơ trong mẫu có cả LTV và DTI:', int((mau_chung &
eda.ltv.gt(1000)).sum()))

# Đối chiếu hiệu ứng thu nhập khi không xét giá trị 0 và khi chỉ còn mẫu
đủ LTV/DTI.
thu_nhap_nhay = []
for ten, mask in [('Income ghi nhận', eda.income.notna()),
                  ('Income dương', eda.income.gt(0)),
                  ('Income dương + đủ LTV/DTI', mau_duong)]:
    s1 = eda.loc[mask & eda.status.eq(1), 'income']
    s0 = eda.loc[mask & eda.status.eq(0), 'income']
    u, _ = mannwhitneyu(s1, s0, method='asymptotic')
    thu_nhap_nhay.append({'Mẫu': ten, 'N': int(mask.sum()),
                          'Median status=0': s0.median(), 'Median
status=1': s1.median(),
                          'Cliff δ': 2*u/(len(s1)*len(s0)) - 1})
print('Độ nhạy của liên hệ thu nhập với status:')
display(pd.DataFrame(thu_nhap_nhay).round(3))
```

	Biến	Kịch bản	N mẫu	N nhóm cao	Tỷ lệ nhóm cao (%)	Tỷ lệ còn lại (%)	Chênh lệch (đpt)
0	LTV	Có LTV	133572	1799	65.090	15.460	49.630

1	LTV	Có LTV, loại 6 cực trị >1.000%	133566	1793	65.250	15.460	49.790
2	LTV	Có cả LTV và DTI	124547	1505	66.980	15.700	51.270
3	LTV	Cả hai, income dương	124089	1499	66.910	15.480	51.430
4	DTI	Có DTI	124549	8639	42.740	14.350	28.380
5	DTI	Có cả LTV và DTI	124547	8639	42.740	14.350	28.380
6	DTI	Cả hai, income dương	124089	8635	42.730	14.110	28.620

Sáu hồ sơ có LTV >1.000%: 6 | số hồ sơ trong mẫu có cả LTV và DTI: 0
Độ nhạy của liên hệ thu nhập với status:

	Mẫu	N	Median status=0	Median status=1	Cliff δ
0	Income ghi nhận	139520	6,000.000	4,860.000	-0.185
1	Income dương	138260	6,000.000	5,040.000	-0.155
2	Income dương + đủ LTV/DTI	124089	6,000.000	4,680.000	-0.213

Nhận xét: Chênh lệch tỷ lệ status = 1 giữa LTV > 100% và phần còn lại giữ ở khoảng 49,63–51,43 điểm phần trăm qua các kịch bản; với DTI > 50 là 28,38–28,62 điểm. Cliff's δ của thu nhập thay đổi từ - 0,155 đến - 0,213 tùy mẫu (mạnh hơn khi có income = 0: −0,185).

Kết luận: Tín hiệu LTV/DTI ổn định trong các phép lọc đã thử, còn mức độ liên hệ thu nhập nhạy với cách chọn mẫu, nên cần báo cáo kèm mẫu sử dụng.

3.7. Tổng hợp Deep Insights

1. Chất lượng dữ liệu quyết định cách đọc mọi kết quả

Tỷ lệ status = 1 là 24,64% trên toàn bộ nhưng chỉ còn 16,13% trong mẫu có LTV. Nguyên nhân là nhóm EQUI (15.297/15.298 hồ sơ vỡ nợ, thiếu LTV gần như toàn bộ), cộng với interest_rate_spread thiếu đúng bằng status = 1.

Hàm ý: kết quả chỉ đúng với hồ sơ có dữ liệu; các biến lãi suất, phí và credit_type không nên dùng dự báo nếu chưa xác minh thời điểm ghi nhận

2. DTI > 50 và LTV > 100% là hai ngưỡng rủi ro rõ nhất

Tỷ lệ vỡ nợ là 42,74% (DTI > 50) so với 12,20% (DTI 30–40), và 65,09% (LTV > 100%) so với 12,94% (LTV 60–80%). Chênh lệch ổn định qua các kịch bản kiểm tra độ nhạy (28,4–28,6 và 49,6–51,4 đpt). Hai nhóm chỉ chiếm 6,9% và 1,35% mẫu nhưng chứa 18,2% và 5,4% số hồ sơ vỡ nợ.

Hàm ý: nên rà soát có trọng tâm các nhóm này, không dùng để tự động từ chối

3. Phân tích kết hợp LTV và DTI cho thấy sự khác biệt giữa các phân khúc

Tỷ lệ status = 1 giữa các phân khúc LTV × DTI dao động từ 9,1% đến 82,1%. Nhóm LTV trên 100% có tỷ lệ vỡ nợ cao ở các mức DTI (57,8%–82,1%), trong khi tỷ lệ của nhóm DTI trên 50 thay đổi đáng kể theo LTV. Đáng chú ý, tỷ lệ vỡ nợ không tăng đều khi cả hai chỉ số cùng tăng, cho thấy mối liên hệ giữa đặc điểm tài chính và trạng thái vỡ nợ có tính không đồng nhất giữa các phân khúc.

Hàm ý: Nên xem xét đồng thời LTV và DTI khi phân tích danh mục khoản vay, thay vì chỉ dựa vào một chỉ số. Tuy nhiên, cần kiểm định bổ sung trước khi kết luận về hiệu ứng tương tác hoặc xây dựng quy tắc chấm điểm rủi ro

4. Thu nhập thấp, khoản vay nhỏ và type2 có liên hệ nhưng yếu hoặc không đơn điệu

Tỷ lệ vỡ nợ giảm từ 32,98% (thu nhập Q1) xuống 20,22% (Q4); khoản vay Q1 là 30,50% so với 21,36% ở Q4 và 22,49% ở Q5; type2 là 34,54% so với 22,77% ở type1 (V = 0,094). Nhóm income = 0 (99,37% vỡ nợ) rất bất thường nên cần xem riêng. Vì loan_amount và income tương quan cao (ρ = 0,642), hai hiệu ứng này khó tách rời

5. Điểm tín dụng không phân biệt được hai nhóm trong bộ dữ liệu này

Tỷ lệ vỡ nợ theo bốn khoảng điểm chỉ dao động 24,29–25,17%, p = 0,1243, δ ≈ 0,005. Phân phối điểm gần phẳng trong 500–900 và không tương quan mạnh với biến tài chính nào, nên có thể đây là dữ liệu mô phỏng hoặc ẩn danh

Hàm ý: không kết luận điểm tín dụng vô dụng ngoài thực tế, chỉ kết luận bộ dữ liệu này chưa cho thấy tín hiệu.

Kết luận: Chương 3 xác định được các phân khúc rủi ro cao (DTI > 50, LTV > 100%) và các giới hạn lớn của dữ liệu (thiếu theo status, EQUI, nghi rò rỉ). Chương 4 sẽ tập trung vào ba khuyến nghị: kiểm soát chất lượng và thời điểm ghi nhận dữ liệu; rà soát có trọng tâm LTV > 100%, DTI > 50, income = 0. Đây là mối liên hệ thống kê, chưa phải nhân quả hay mô hình dự báo.

CHƯƠNG 4. ĐÁNH GIÁ TÁC ĐỘNG VÀ KHUYẾN NGHỊ

4.1. Tổng hợp những phát hiện chính

4.1.1. Phát hiện về chất lượng dữ liệu

Quá trình rà soát, làm sạch và đối chiếu chất lượng dữ liệu trên 148.670 hồ sơ vay (34 thuộc tính gốc, thu gọn còn 28 thuộc tính sau khi loại 6 biến dư thừa hoặc gần như hằng số) chỉ ra ba vấn đề cốt lõi:

Dữ liệu khuyết thiếu quy mô lớn, tập trung theo cụm và có liên hệ thống kê chặt với trạng thái vỡ nợ: Toàn bộ tập dữ liệu có 33,96% số hồ sơ (50.483 dòng) thiếu ít nhất một giá trị, tập trung chủ yếu ở 7 biến tài chính và thường khuyết đồng thời theo cụm nghiệp vụ (như cặp ltv - property_value cùng thiếu trên 15.098 hồ sơ). Kết quả kiểm định cho thấy tình trạng khuyết dữ liệu không độc lập ngẫu nhiên mà có mối liên hệ thống kê rất mạnh với biến mục tiêu status. Đáng chú ý nhất, ba biến chi phí vay (interest_rate_spread, rate_of_interest, upfront_charges) bị khuyết từ 99,45% đến 100,00% ở nhóm status = 1 nhưng chỉ khuyết 0,00%-2,82% ở nhóm status = 0 (Cramér' *V* từ 0,943 đến 1,000), trong khi dtir1 và ltv cũng khuyết ở nhóm status = 1 cao gấp nhiều lần nhóm status = 0 (lần lượt 44,52% so với 6,97% và 41,20% so với 0,002%). Do bộ dữ liệu không có các mốc thời gian chi tiết về tiếp nhận, phê duyệt và giải ngân, nghiên cứu chưa thể xác định đầy đủ nguyên nhân thiếu dữ liệu. Tuy nhiên, mối liên hệ rất mạnh giữa trạng thái thiếu và `status` cho thấy nguy cơ rò rỉ thông tin mục tiêu nếu các biến này được sử dụng cho dự báo mà chưa xác minh thời điểm ghi nhận.

Sự bất thường của nhóm credit_type = equi và hiện tượng lệch cơ cấu mẫu quan sát: Trong 15.098 hồ sơ thiếu ltv và property_value, có tới 15.096 hồ sơ (99,99%) thuộc về nhóm equi. Bản thân nhóm equi (15.298 hồ sơ) có tới 99,99% hồ sơ mang status = 1 (15.297 hồ sơ), chiếm 41,75% tổng số ca vỡ nợ của toàn bộ dữ liệu. Hệ quả trực tiếp là khi lọc các hồ sơ có dữ liệu hợp lệ để phân tích ltv (N=133.572) hoặc dtir1 (N=124.549), nhóm equi gần như bị loại khỏi mẫu, kéo tỷ lệ status = 1 giảm từ 24,64% (toàn mẫu) xuống còn 16,13%-16,32% (tương đương mức 16,00% của tập dữ liệu không gồm equi). Sự sụt giảm này phản ánh sự thay đổi cơ cấu mẫu quan

sát chứ không phải tác động giảm rủi ro của LTV hay DTI, đồng thời giới hạn phạm vi đại diện của các phân tích đòn bẩy chỉ trong tập hồ sơ có dữ liệu.

Các dấu hiệu bất thường trong quy ước ghi nhận và nhập liệu: Bộ dữ liệu không có bản ghi trùng lặp hay vi phạm miền giá trị âm, nhưng tồn tại các cụm quan sát dị biệt mang tính quy ước hệ thống hoặc lỗi nhập liệu tiềm ẩn: 200 hồ sơ khuyết đồng thời age và submission_of_application (gán nhãn unknown) đều có status = 1 (100%); 1.260 hồ sơ có income = 0 ghi nhận tỷ lệ status = 1 lên tới 99,37%; và 6 hồ sơ có LTV > 1.000% (cao nhất 7.831,25%) đều có property_value bằng đúng 8.000 kèm khuyết thu nhập. Bên cạnh đó, việc 25,33% hồ sơ thiếu thông tin giới tính (sex not available) và biến credit_score phân bố phẳng đồng đều trên toàn dải 500-900 điểm gợi ý dữ liệu có thể đã qua xử lý ẩn danh hoặc mô phỏng một phần.

4.1.2. Phát hiện về đặc điểm các khoản vay

Thống kê mô tả đơn biến và đa biến giúp rút ra được ba đặc trưng cấu trúc nổi bật của danh mục tín dụng:

Danh mục mang đặc thù cho vay thế chấp dài hạn với mức độ tập trung cao ở một số phân khúc chuẩn: Khoản vay không vỡ nợ chiếm 75,36% và vỡ nợ chiếm 24,64% (tỷ lệ xấp xỉ 3:1). Về cấu trúc sản phẩm, danh mục tập trung áp đảo vào kỳ hạn 360 tháng (81,88%), loại khoản vay type1 (76,12%), hai mục đích vay p3 và p4 (cộng gộp chiếm 74,48%) và nhóm khách hàng trung niên 35-64 tuổi (chiếm 67,4% số hồ sơ có tuổi hợp lệ). Quy mô các phân nhóm chênh lệch lớn đòi hỏi việc đánh giá rủi ro phải dựa trên tỷ lệ vỡ nợ nội bộ của từng nhóm thay vì tần suất tuyệt đối.

Sự đối lập giữa phân phối lệch phải mạnh của các biến tiền tệ và tính tập trung của các tỷ số đòn bẩy: Các biến đo lường quy mô tiền tệ tuyệt đối gồm loan_amount (trung vị 296.500; skewness = 1,67), property_value (trung vị 418.000; skewness = 4,59) và đặc biệt là income (trung vị 5.760; skewness = 17,31) đều có đuôi phải kéo dài bởi các quan sát giá trị lớn hợp lệ, khiến trung bình số học luôn cao hơn đáng kể so với trung vị. Ngược lại, trên mẫu quan sát hợp lệ, các chỉ số tương đối như ltv (trung vị 75,14%, khoảng tứ phân vị 60,47%-86,18% khi tách riêng 6 điểm cực trị > 1.000%) và dtir1 (trung vị 39,00, khoảng tứ phân vị 31,00-45,00) lại tập trung khá cân đối quanh vùng trung tâm. Đặc điểm này khẳng định sự cần thiết của việc sử dụng trung vị, khoảng tứ phân vị trong phân tích mô tả cũng như phép biến đổi logarit và chuẩn hóa bền vững (RobustScaler) khi chuẩn bị dữ liệu mô hình.

Hiện tượng chồng lặp thông tin giữa các biến quy mô tài chính: Phân tích tương quan thứ hạng Spearman ghi nhận mối liên hệ thuận khá mạnh giữa loan_amount với property_value ($\rho = 0,857$) và với income ($\rho = 0,642$), cũng như giữa property_value với income ($\rho = 0,606$). Điều này phản ánh thực tế nghiệp vụ rằng khách hàng thu nhập cao thường vay khoản tiền lớn để mua tài sản giá trị cao; hệ quả là hiệu ứng riêng lẻ của quy mô vay và thu nhập

khó tách rời hoàn toàn trong phân tích đơn biến và dễ gây đa cộng tuyến nếu đưa đồng thời các biến gốc vào mô hình tuyến tính.

4.1.3. Các yếu tố liên hệ rõ nhất với tình trạng vỡ nợ

Do kích thước mẫu lớn làm p-value gần như luôn rất nhỏ, việc đối chiếu kết quả kiểm định giả thuyết (hiệu chỉnh FDR ở mức alpha = 5%) kết hợp cùng kích thước hiệu ứng (Cramér's V, Cliff's δ) và phân tích độ nhạy đã làm rõ bốn phát hiện trọng tâm về mối liên hệ thống kê với status:

Mối liên hệ nổi bật của các nhóm DTI > 50 và LTV > 100% với tình trạng vỡ nợ: Trong các giả thuyết chính, DTI (Cramér's V = 0,200) và LTV (Cramér's V = 0,173) có mức độ liên hệ đáng chú ý nhất với status. Nhóm DTI trên 50 ghi nhận tỷ lệ status = 1 là 42,74%, cao hơn các nhóm DTI còn lại (12,20%–15,60%). Tương tự, nhóm LTV trên 100% có tỷ lệ status = 1 là 65,09%, trong khi các nhóm LTV không quá 100% dao động từ 12,94% đến 19,69%. Chênh lệch giữa nhóm vượt ngưỡng và phần còn lại tương đối ổn định qua các kịch bản kiểm tra độ nhạy. Kết quả cho thấy đây là hai phân khúc đáng quan tâm trong mẫu có dữ liệu hợp lệ, nhưng chưa đủ căn cứ xác định các ngưỡng này là điểm gãy rủi ro hoặc có hiệu quả dự báo trên hồ sơ mới.

Sự khác biệt giữa các phân khúc khi kết hợp LTV và DTI: Trên 124.547 hồ sơ có đầy đủ hai chỉ số, tỷ lệ status = 1 giữa các phân khúc dao động từ 9,1% đến 82,1%. Nhóm LTV trên 100% ghi nhận tỷ lệ vỡ nợ cao ở các mức DTI, nhưng tỷ lệ không tăng đều khi cả hai chỉ số cùng tăng. Chẳng hạn, nhóm LTV trên 100% và DTI không quá 30 có tỷ lệ vỡ nợ 82,1%, cao hơn nhóm đồng thời có LTV trên 100% và DTI trên 50 (57,8%). Kết quả cho thấy phân khúc kết hợp giúp làm rõ sự khác biệt về tỷ lệ vỡ nợ, nhưng chưa đủ cơ sở xác định tính cộng dồn hoặc tương tác thống kê giữa LTV và DTI.

Mối liên hệ yếu hoặc phi đơn điệu của thu nhập, quy mô vay và đặc trưng hợp đồng: Thu nhập trung vị của nhóm status = 1 thấp hơn nhóm status = 0 (4.860 so với 6.000; Cliff's δ = -0.185, giảm về -0.155 khi loại cụm dị biệt income = 0), với tỷ lệ vỡ nợ trên mẫu thu nhập dương giảm dần từ 32,98% ở tứ phân vị Q1 xuống 20,22% ở Q4. Do tương quan thuận với thu nhập, nhóm khoản vay quy mô nhỏ nhất (Q1 $\leq$ 176.500) lại có tỷ lệ vỡ nợ cao nhất (30,50%) so với mức 21,36%-22,49% ở Q4 và Q5 (V = 0.076). Ở các biến phân loại (sau khi tách riêng yếu tố credit_type có V = 0.592 do hiệu ứng equi), mức liên hệ đáng kể hơn cả thuộc về các điều khoản cấu trúc trả nợ như lump_sum_payment (V = 0.188), neg_ammortization (V = 0.156) và phân khúc vay kinh doanh/thương mại type2 (tỷ lệ vỡ nợ 34,54% so với 22,77% ở type1, V = 0.094), trong khi các biến nhân khẩu học như age (V = 0.050) hay region (V = 0.051) chỉ có hiệu ứng rất yếu.

Sự vắng mặt của mối liên hệ thống kê ở biến Điểm tín dụng (credit_score): Kiểm định thực nghiệm xác nhận credit_score không có khả năng phân biệt trạng thái vỡ nợ trong bộ dữ liệu này ($p = 0,1243$, Cliff's $\delta = +0,005$, Cramér's V = 0.008), khi tỷ lệ status = 1 ở cả bốn khoảng

điểm từ 500-599 đến 800-900 đều xoay quanh mức 24,29%-25,17%. Kết hợp với việc credit_score hoàn toàn độc lập với các biến tài chính khác ($\rho \approx 0,00$), phát hiện này phản ánh hạn chế đặc thù của dữ liệu nguồn hơn là bằng chứng bác bỏ vai trò của điểm tín dụng trong thực tiễn ngân hàng

4.2. Đánh giá tác động trong quản trị rủi ro tín dụng

Những phát hiện từ dữ liệu đặt ra các hệ lụy và thách thức trực tiếp đối với hoạt động quản trị rủi ro tín dụng trên ba khía cạnh: thẩm định hồ sơ, giám sát danh mục và quản trị chất lượng dữ liệu.

4.2.1. Đối với quá trình tiếp nhận và thẩm định hồ sơ

Rủi ro sai lệch thẩm định khi phụ thuộc vào các tiêu chí đơn lẻ hoặc trực giác truyền thống: Thực tế biến credit_score không mang tín hiệu phân loại rủi ro và phân khúc khoản vay nhỏ nhất (Q1) lại có tỷ lệ vỡ nợ cao nhất cho thấy những giả định quen thuộc như "khách hàng điểm tín dụng cao thì ít vỡ nợ" hay "khoản vay nhỏ thì rủi ro thấp" không nghiệm đúng trên tập dữ liệu này.Những kết quả này gợi ý rằng việc đánh giá hồ sơ chỉ dựa vào điểm tín dụng hoặc quy mô khoản vay có thể bỏ sót một số khác biệt về đặc điểm tài chính trong dữ liệu nghiên cứu. Vì vậy, quy trình thẩm định nên xem xét đồng thời nhiều tiêu chí, nhưng cần kiểm chứng thêm trước khi xác định mức độ ảnh hưởng của từng yếu tố đối với quyết định phê duyệt thực tế.

Ý nghĩa của LTV và DTI trong đánh giá tình hình tài chính người vay: Kết quả nghiên cứu cho thấy các nhóm có LTV trên 100% và DTI trên 50 ghi nhận tỷ lệ status = 1 cao hơn đáng kể so với các nhóm còn lại. Về mặt nghiệp vụ, LTV cao có thể phản ánh mức độ bảo đảm bằng tài sản thấp hơn so với giá trị khoản vay, trong khi DTI cao có thể gợi ý áp lực thực hiện nghĩa vụ trả nợ lớn hơn. Vì vậy, hai chỉ số này có thể hỗ trợ xác định những hồ sơ cần rà soát bổ sung. Tuy nhiên, nghiên cứu chưa chứng minh được quan hệ nhân quả hoặc xác nhận đây là các ngưỡng quyết định rủi ro tối ưu trong thực tế.

Rủi ro tiềm ẩn từ cấu trúc trả nợ đặc thù và khoản vay thương mại: Việc các khoản vay kinh doanh/thương mại (type2) hoặc có điều khoản trả gốc gộp một lần (lump_sum_payment), khấu hao âm (neg_ammortization) đi kèm với tỷ lệ vỡ nợ cao hơn hẳn mặt bằng chung cho thấy cấu trúc dòng tiền trả nợ không đều tạo ra áp lực thanh khoản lớn vào kỳ đáo hạn hoặc làm dư nợ gốc tăng thêm theo thời gian, khiến rủi ro tín dụng phức tạp hơn nhiều so với khoản vay thế chấp trả góp tiêu chuẩn (typel).

4.2.2. Đối với phân khúc và giám sát danh mục khoản vay

Ý nghĩa của việc giám sát danh mục theo nhiều đặc điểm tài chính: Kết quả phân tích LTV × DTI cho thấy tỷ lệ vỡ nợ khác biệt đáng kể giữa các phân khúc và không tăng đều theo mức của cả hai chỉ số. Nhóm LTV trên 100% ghi nhận tỷ lệ `status = 1` cao ở các mức DTI, trong khi mức độ liên hệ của DTI trên 50 thay đổi tùy theo nhóm LTV. Điều này gợi ý rằng việc giám sát

danh mục nên xem xét đồng thời nhiều đặc điểm tài chính để nhận diện sự khác biệt giữa các phân khúc, thay vì chỉ dựa vào một tiêu chí riêng lẻ. Tuy nhiên, nghiên cứu chưa xây dựng hoặc kiểm định hệ thống chấm điểm rủi ro, do đó chưa thể kết luận về hiệu quả hay hạn chế của cơ chế cộng điểm tuyến tính trong thực tế. Các kết quả hiện tại chỉ phù hợp để định hướng phân khúc và ưu tiên rà soát sơ bộ.

Ảnh hưởng của cơ cấu mẫu đến kết quả giám sát danh mục: Nhóm credit_type = equi có cấu trúc dữ liệu bất thường, với gần như toàn bộ hồ sơ mang status = 1 và phần lớn thiếu thông tin LTV. Nhóm này chiếm 41,75% tổng số hồ sơ status = 1 trong bộ dữ liệu nghiên cứu. Khi chỉ xét các hồ sơ có dữ liệu LTV hoặc DTI hợp lệ, EQUI gần như bị loại khỏi mẫu, khiến tỷ lệ status = 1 giảm từ 24,64% ở toàn mẫu xuống lần lượt 16,13% và 16,32%. Sự khác biệt này chủ yếu phản ánh thay đổi về thành phần mẫu quan sát, không chứng minh rằng việc có dữ liệu LTV hoặc DTI giúp giảm nguy cơ vỡ nợ. Do chưa xác minh được bản chất nghiệp vụ của EQUI và tính đại diện của dữ liệu, không thể mặc định những tỷ lệ trên phản ánh chất lượng tín dụng thực tế của một tổ chức cho vay. Vì vậy, các báo cáo phân tích cần ghi rõ phạm vi mẫu, quy mô và cơ cấu nhóm tín dụng để tránh diễn giải sai lệch.

Hiện tượng nhiễu phân tầng do chồng lặp đặc trưng quy mô: Tương quan mạnh giữa số tiền vay, giá trị tài sản bảo đảm và thu nhập (ρ từ 0,606 - 0,857) khiến việc phân khúc danh mục đồng thời theo các biến quy mô tuyệt đối này bị trùng lặp thông tin, làm giảm hiệu quả phân tách rủi ro so với việc giám sát bằng các tỷ số đã chuẩn hóa theo quy mô

4.2.3. Đối với quản lý chất lượng dữ liệu

Nguy cơ sử dụng thông tin không phù hợp thời điểm trong mô hình dự báo: Trạng thái thiếu của các biến chi phí vay như interest_rate_spread, rate_of_interest, upfront_charges, cùng sự bất thường của nhóm EQUI và một số nhãn unknown, có mối liên hệ rất mạnh với `status`. Nếu những thông tin này được ghi nhận hoặc thay đổi sau thời điểm cần dự báo, việc sử dụng chúng có thể gây rò rỉ thông tin mục tiêu và làm kết quả đánh giá mô hình trở nên quá lạc quan. Tuy nhiên, do dữ liệu chưa có thông tin thời gian đầy đủ và nghiên cứu chưa triển khai mô hình dự báo, chưa đủ căn cứ để kết luận đã xảy ra Data Leakage hoặc xác định mức độ ảnh hưởng thực tế. Vì vậy, cần ưu tiên xác minh thời điểm ghi nhận và tính sẵn có của từng thuộc tính trước khi đưa vào mô hình.

Tác động làm biến dạng cấu trúc dữ liệu của các thao tác tiền xử lý cơ học: Với hơn 1/3 số bản ghi có dữ liệu khuyết và cơ chế khuyết gắn chặt với biến mục tiêu, việc xóa dòng cơ học sẽ làm thất thoát lượng lớn thông tin và làm biến dạng phân phối của nhóm vỡ nợ. Ngược lại, việc điền khuyết đơn giản bằng trung bình hoặc trung vị toàn mẫu cho các biến khuyết lệch như property_value, ltv hay dtir1 sẽ gán các giá trị đặc trưng của nhóm không vỡ nợ vào hàng chục

nghìn hồ sơ thuộc nhóm vỡ nợ (ví dụ làm trung bình property_value của nhóm status = 1 bị kéo lệch từ 457.786 xuống 441.393), làm suy giảm tính chân thực của dữ liệu phân tích.

Hệ lụy từ việc thiếu kiểm soát tính toàn vẹn dữ liệu đầu vào: Sự xuất hiện của các quan sát cực đoan hoặc bất thường về ngữ nghĩa nghiệp vụ như 1.260 hồ sơ income = 0 (vỡ nợ 99,37%), 6 hồ sơ LTV > 1.000% làm hệ số skewness của LTV bị đẩy vọt lên 120,62, cùng tình trạng thiếu đồng nhất quy tắc đặt tên, viết hoa/thường và lỗi chính tả (Indriect) cho thấy những lỗ hổng trong kiểm soát dữ liệu tại điểm nhập liệu có thể trực tiếp làm sai lệch các thống kê tổng hợp và làm tăng rủi ro sai số trong toàn bộ chuỗi phân tích tín dụng.

4.3. Đề xuất các giải pháp ứng dụng

4.3.1. Nhóm giải pháp cho tiếp nhận và thẩm định hồ sơ

Giải pháp 1 – Thiết lập bộ cờ cảnh báo rà soát tăng cường.

Dựa trên kết quả phân tích, nghiên cứu đề xuất sử dụng các cờ cảnh báo riêng cho LTV, DTI và những giá trị dữ liệu bất thường nhằm hỗ trợ xác định hồ sơ cần kiểm tra bổ sung. Các cờ có thể được xem xét đồng thời, nhưng chưa nên chuyển thành một điểm rủi ro tổng hợp khi chưa kiểm định phương pháp kết hợp và hiệu quả phân loại. Đây chỉ là tiêu chí hỗ trợ rà soát, không phải căn cứ độc lập để phê duyệt hoặc từ chối khoản vay.

Cờ cảnh báo	Điều kiện	Phạm vi mẫu	Tỷ lệ Status = 1 trong nhóm	Phần còn lại	Hành động đề xuất
C1 – Đòn bẩy tài sản	LTV > 100%	133.572 hồ sơ có LTV (1.799 hồ sơ bị gắn cờ)	65,09%	15,46%	Định giá lại tài sản bởi bên độc lập; yêu cầu tài sản bổ sung hoặc giảm số tiền vay
C2 – Gánh nặng trả nợ	DTI > 50	124.549 hồ sơ có DTI (8.639 hồ sơ bị gắn cờ)	42,74%	14,35%	Xác minh lại toàn bộ nghĩa vụ nợ và thu nhập; cân nhắc kỳ hạn hoặc hạn mức phù hợp
C3 – Thu nhập bằng 0	income = 0	139.520 hồ sơ có income (1.260 hồ sơ bị gắn cờ)	99,37%	–	Xác minh nguyên nhân ghi nhận thu nhập bằng 0, đối chiếu chứng từ và nguồn trả nợ. Không mặc định đây là hồ sơ không hợp lệ khi chưa xác định ý nghĩa của giá trị 0.
C4 – Giá trị bất thường	LTV > 1.000%	6 hồ sơ (property_value = 8.000)	–	–	Trả lại bộ phận nhập liệu kiểm tra trước khi xử lý tiếp

Giải pháp 2 – Không nới lỏng thẩm định chỉ vì điểm tín dụng cao hoặc khoản vay nhỏ. Trong bộ dữ liệu, Credit_Score không phân biệt được hai nhóm ($p = 0,1243$; Cliff's $\delta = +0,005$). Khoản vay nhỏ nhất (Q1) lại có tỷ lệ vỡ nợ cao nhất (30,50%). Quy trình phê duyệt nhanh nên dựa cả vào LTV, DTI và thu nhập, không chỉ dựa vào điểm số hay quy mô vay.

Giải pháp 3 – Quy trình thẩm định riêng cho khoản vay kinh doanh và điều khoản trả nợ đặc thù. Khoản vay type2 có tỷ lệ vỡ nợ 34,54%, so với 22,77% ở type1. Trả gộp một lần ($V = 0,188$) và khấu hao âm ($V = 0,156$) là hai điều khoản liên hệ rõ nhất với Status sau credit_type. Với nhóm này, hồ sơ cần có dòng tiền dự phóng và phương án trả nợ khi đến hạn khoản gộp.

4.3.2. Nhóm giải pháp cho phân khúc và giám sát danh mục

Giải pháp 4 – Thử nghiệm giám sát danh mục theo phân khúc LTV × DTI. Đề xuất xây dựng bảng theo dõi tỷ lệ status = 1 theo các nhóm kết hợp LTV và DTI, trong đó ghi rõ quy mô và tỷ lệ dữ liệu hợp lệ của từng phân khúc. Các nhóm có LTV trên 100% hoặc DTI trên 50 có thể được ưu tiên xem xét trong giai đoạn thử nghiệm. Tuy nhiên, do các ngưỡng này chưa được xác thực về hiệu quả dự báo, tần suất giám sát và các biện pháp quản trị cụ thể chỉ nên được xác định sau khi đánh giá tính ổn định của kết quả trên dữ liệu mới.

Giải pháp 5 – Báo cáo tỷ lệ vỡ nợ tách riêng nhóm EQUI. Gộp EQUI vào sẽ làm tỷ lệ toàn danh mục cao hơn nhiều so với nhóm hồ sơ thông thường. Mọi bảng so sánh cần ghi rõ cỡ mẫu hợp lệ.

Giải pháp 6 – Ưu tiên tỷ số thay cho biến quy mô tuyệt đối. Khoản vay, giá trị tài sản và thu nhập trùng thông tin với nhau. Khi phân tầng danh mục hay xây thẻ điểm, nên dùng LTV, DTI và log thu nhập.

4.3.3. Nhóm giải pháp cho quản lý chất lượng dữ liệu

Giải pháp 7 – Quy tắc kiểm tra tự động ngay khi nhập liệu. Bắt buộc nhập giá trị tài sản, thu nhập, DTI và nhóm tuổi. Cảnh báo khi thu nhập bằng 0 hoặc giá trị tài sản nhỏ bất thường. Tự tính LTV thay vì nhập tay. Chọn nhãn từ danh sách chuẩn để tránh lỗi như "Indriect" hay Region lẫn chữ hoa, chữ thường.

Giải pháp 8 – Ghi nhận thời điểm cho từng biến. Cần lưu ngày nộp hồ sơ, ngày phê duyệt, ngày giải ngân và ngày phát sinh Status. Hồ sơ bị từ chối hoặc chưa giải ngân cũng nên tách khỏi khoản vay đã giải ngân. Việc ghi nhận đầy đủ các mốc thời gian giúp xác định thông tin nào đã có sẵn tại thời điểm dự báo, từ đó hỗ trợ phát hiện và hạn chế nguy cơ rò rỉ dữ liệu khi xây dựng mô hình.

Giải pháp 9 – Đối soát luồng dữ liệu EQUI trước khi sử dụng. Nhóm này gần như toàn bộ là hồ sơ vỡ nợ và thiếu LTV. Cần xác minh với đơn vị cung cấp dữ liệu xem đây là rủi ro thật hay hệ quả của cách ghi nhận, trước khi đưa credit_type vào phân tích.

Thứ tự triển khai:

Giai đoạn	Giải pháp	Lý do

Làm trước	7, 8, 9 (dữ liệu)	Khuyến nghị về LTV, DTI chỉ đáng tin khi dữ liệu đầy đủ và đúng thời điểm ghi nhận
Song song	1, 5	Áp dụng được ngay trên hồ sơ có đủ dữ liệu
Sau đó	2, 3, 4, 6	Cần kiểm chứng thêm vì mức liên hệ yếu và chưa kiểm soát biến gây nhiễu

Nhóm giải pháp dữ liệu được xếp làm trước vì mọi khuyến nghị về LTV, DTI chỉ đáng tin khi dữ liệu đầy đủ và đúng thời điểm ghi nhận.

4.4. Đánh giá giới hạn của nghiên cứu

Kết quả của nghiên cứu chỉ nên được dùng trong phạm vi bộ dữ liệu này và trên nhóm hồ sơ có dữ liệu hợp lệ. Bốn giới hạn dưới đây quyết định mức độ có thể áp dụng vào thực tế.

4.4.1. Giới hạn về nguồn gốc và tính đại diện

- Kaggle không nêu tổ chức cho vay, quốc gia, cách chọn mẫu và không có từ điển dữ liệu. Ý nghĩa nhiều mã (type1–type3, p1–p4) và đơn vị thu nhập do nhóm tự suy luận.
- Điểm tín dụng phân bố gần như đều và không tương quan với biến tài chính nào, gợi ý dữ liệu có thể là mô phỏng hoặc đã ẩn danh. Kết luận về điểm tín dụng vì vậy chỉ đúng với bộ dữ liệu này.

4.4.2. Giới hạn về dữ liệu thiếu

- Khoảng 1/3 hồ sơ thiếu dữ liệu, và việc thiếu phụ thuộc vào Status. Không phương pháp điền khuyết nào khôi phục được giá trị thật.
- Phân tích LTV, DTI chỉ dựa trên hồ sơ đủ dữ liệu, gần như không có EQUI, trong khi nhóm này chiếm khoảng 40% số ca vỡ nợ. Các ngưỡng LTV, DTI chưa được kiểm tra trên nhóm này.
- Không đánh giá được vai trò của lãi suất và phí, vì các biến này gần như chỉ có ở nhóm không vỡ nợ.

4.4.3. Giới hạn về quan hệ nhân quả

Các phương pháp Chi-square, Mann–Whitney U và tương quan Spearman chủ yếu đánh giá mức độ liên hệ hoặc sự khác biệt thống kê. Do đó, nghiên cứu chưa chứng minh được rằng một yếu tố như LTV, DTI hay thu nhập trực tiếp gây ra tình trạng vỡ nợ.

Bên cạnh đó, các phân tích phần lớn được thực hiện theo từng biến hoặc phân khúc kết hợp hai chiều, chưa kiểm soát đầy đủ những yếu tố có thể gây nhiễu. Mặc dù sáu trong bảy kiểm định giả thuyết có ý nghĩa thống kê sau hiệu chỉnh FDR, một số kích thước hiệu ứng vẫn tương đối nhỏ. Vì vậy, ý nghĩa thống kê không nên được đồng nhất với mức độ quan trọng trong thực tiễn.

4.4.4. Giới hạn về thời gian và mục tiêu

- Dữ liệu chỉ có năm 2019 và không có mốc ngày. Do đó không xét được xu hướng hay chu kỳ kinh tế, và không xác minh được thứ tự ghi nhận các biến.
- Định nghĩa Status chưa rõ: không biết là quá hạn bao lâu, hay có gộp cả hồ sơ bị từ chối hay không.
- Nghiên cứu dừng ở mô tả và kiểm định, chưa huấn luyện mô hình dự báo. Các ngưỡng do nhóm tự chia, chưa được tối ưu.

4.5. Đề xuất hướng nghiên cứu tiếp theo

Từ những giới hạn đã nêu ở mục 4.4, nhóm đề xuất năm hướng nghiên cứu tiếp theo nhằm nâng cao độ tin cậy và khả năng ứng dụng của kết quả.

Thứ nhất, làm rõ nguồn gốc và cơ chế thiếu dữ liệu. Kết quả Chương 1 và Chương 2 cho thấy việc thiếu các biến chi phí vay (rate_of_interest, Interest_rate_spread, Upfront_charges) gần như trùng khớp với nhóm hồ sơ vỡ nợ. Đây có thể là đặc điểm của quy trình ghi nhận dữ liệu chứ không phải đặc điểm của người vay. Nghiên cứu tiếp theo cần tìm hiểu từ điển dữ liệu gốc hoặc quy trình thu thập của tổ chức cung cấp dữ liệu, nhằm xác định các biến này được ghi nhận vào thời điểm nào của vòng đời khoản vay. Khi đã làm rõ cơ chế thiếu, có thể áp dụng các phương pháp điền khuyết nâng cao như điền khuyết đa trị (Multiple Imputation, MICE), kèm phân tích độ nhạy để đánh giá mức ảnh hưởng của cách xử lý dữ liệu thiếu đến kết luận.

Thứ hai, mở rộng phạm vi dữ liệu theo thời gian và không gian. Bộ dữ liệu hiện tại chỉ ghi nhận năm 2019 và không có trường ngày tháng, nên chưa thể đánh giá sự ổn định của các mối liên hệ qua các chu kỳ kinh tế. Hướng nghiên cứu tiếp theo nên sử dụng dữ liệu nhiều năm để kiểm tra xem các yếu tố như LTV, DTI có giữ nguyên mức độ liên hệ với vỡ nợ trong các giai đoạn lãi suất hoặc thị trường bất động sản khác nhau hay không. Bên cạnh đó, việc kiểm chứng trên dữ liệu tín dụng của các ngân hàng trong nước sẽ giúp đánh giá khả năng áp dụng kết quả vào bối cảnh Việt Nam.

Thứ ba, bổ sung các biến giải thích còn thiếu. Dữ liệu hiện chưa có thông tin về lịch sử thanh toán, nghề nghiệp và mức độ ổn định thu nhập, số khoản vay đang có, cũng như các biến kinh tế vĩ mô (lãi suất thị trường, tỷ lệ thất nghiệp, giá nhà). Ngoài ra, nhóm hồ sơ chấm điểm tín dụng bởi EQUI gần như không có dữ liệu LTV, khiến phân tích LTV thực chất chỉ đại diện cho các nhóm chấm điểm còn lại. Việc bổ sung các biến này và thu thập đầy đủ hơn cho nhóm EQUI sẽ giúp phân tích đa biến toàn diện và giảm thiên lệch do cơ cấu mẫu.

Thứ tư, xây dựng mô hình dự báo có kiểm soát rò rỉ dữ liệu. Trên cơ sở dữ liệu đã làm sạch ở Chương 2, nghiên cứu tiếp theo có thể xây dựng các mô hình phân loại như hồi quy Logistic, cây quyết định, Random Forest hoặc Gradient Boosting để dự báo khả năng vỡ nợ. Điều kiện bắt buộc là loại bỏ các biến có nguy cơ rò rỉ nhãn đã xác định ở mục 2.4.3. Do dữ liệu mất cân bằng (tỷ lệ vỡ nợ 24,64%), mô hình cần được đánh giá bằng các chỉ số phù hợp như Recall, F1-score

và AUC thay vì chỉ dùng độ chính xác (Accuracy). Nếu có dữ liệu theo thời gian, nên kiểm định mô hình trên giai đoạn sau thời điểm huấn luyện để phản ánh đúng điều kiện ứng dụng thực tế.

Thứ năm, hướng tới phân tích nhân quả và đánh giá tính công bằng. Các kết quả trong báo cáo chỉ dừng ở mức liên hệ thống kê. Để chuyển thành căn cứ ra quyết định chính sách tín dụng, cần áp dụng các phương pháp suy luận nhân quả, chẳng hạn so khớp điểm xu hướng (propensity score matching) hoặc thiết kế thử nghiệm có kiểm soát. Đồng thời, do dữ liệu chứa các biến nhạy cảm như giới tính và nhóm tuổi, các nghiên cứu ứng dụng cần đánh giá tính công bằng của mô hình, nhằm đảm bảo quy trình xét duyệt không phân biệt đối xử với các nhóm khách hàng.

4.6. Kết luận chung

Báo cáo đã thực hiện một quy trình phân tích dữ liệu hoàn chỉnh trên bộ dữ liệu Loan Default gồm 148.670 hồ sơ vay và 34 biến: thu thập và đánh giá dữ liệu ban đầu, tiền xử lý và làm sạch, phân tích khám phá, kiểm định giả thuyết, và cuối cùng đánh giá tác động cùng đề xuất khuyến nghị.

Về chất lượng dữ liệu, kết quả cho thấy bộ dữ liệu có quy mô lớn, không có bản ghi trùng lặp và các biến tính toán (như LTV) nhất quán với công thức. Tuy nhiên, dữ liệu tồn tại những hạn chế đáng kể. Có 14 biến chứa giá trị thiếu, ảnh hưởng đến 50.483 hồ sơ. Quan trọng hơn, việc thiếu dữ liệu không xảy ra ngẫu nhiên mà gắn chặt với tình trạng vỡ nợ, đặc biệt ở nhóm biến chi phí vay. Điều này tạo ra nguy cơ rò rỉ dữ liệu và cho thấy đánh giá chất lượng dữ liệu là bước không thể bỏ qua trước khi rút ra bất kỳ kết luận nào.

Về các yếu tố liên quan đến vỡ nợ, phân tích cho thấy tỷ lệ vỡ nợ chung là 24,64%, nhưng khác biệt rõ rệt giữa các nhóm khách hàng. DTI và LTV là hai chỉ số liên hệ mạnh nhất: nhóm có DTI trên 50% có tỷ lệ vỡ nợ 42,74% (so với 12–16% ở các nhóm còn lại), và nhóm có LTV vượt 100% có tỷ lệ vỡ nợ 65,09% (so với 13–20%). Thu nhập liên hệ ngược chiều với vỡ nợ, với tỷ lệ giảm từ 32,98% ở nhóm thu nhập thấp nhất xuống 20,22% ở nhóm cao nhất. Loại khoản vay type2, khoản vay quy mô nhỏ và nhóm tuổi trên 74 cũng có tỷ lệ vỡ nợ cao hơn mức nền. Ngược lại, điểm tín dụng gần như không phân biệt được hai nhóm. Kết quả kiểm định khẳng định 6/7 phép kiểm định thuộc giả thuyết H1–H5 có đủ bằng chứng về mối liên hệ với vỡ nợ, riêng H3 về điểm tín dụng chưa đủ bằng chứng ($p = 0,124$). Tuy nhiên, độ lớn hiệu ứng chỉ ở mức nhỏ đến trung bình (Cramér's V từ 0,050 đến 0,200), cho thấy mỗi yếu tố riêng lẻ chỉ phản ánh một phần khả năng vỡ nợ.

Về ý nghĩa thực tiễn, kết quả gợi ý rằng ngân hàng có thể sử dụng các chỉ số như LTV, DTI và các đặc điểm tín dụng làm tín hiệu cảnh báo sớm trong quá trình thẩm định và giám sát danh mục. Đồng thời, cần cải thiện quy trình ghi nhận dữ liệu để đảm bảo thông tin được thu thập đầy đủ và nhất quán ở mọi hồ sơ, không phụ thuộc vào kết quả khoản vay.

Tuy nhiên, các kết luận trên cần được hiểu đúng phạm vi. Đây là các mối liên hệ thống kê, không phải quan hệ nhân quả. Kết quả chỉ áp dụng trong phạm vi mẫu có dữ liệu hợp lệ, và bị giới hạn bởi dữ liệu thiếu không ngẫu nhiên, sự bất thường của nhóm EQUI, cũng như việc dữ liệu chỉ thuộc một năm duy nhất (2019).